

# Learning long range dependencies through time reversal symmetry breaking

Guillaume Pourcel  
University of Groningen  
g.a.pourcel@rug.nl

Maxence Ernoult  
Google DeepMind  
mernoult@google.com

## Abstract

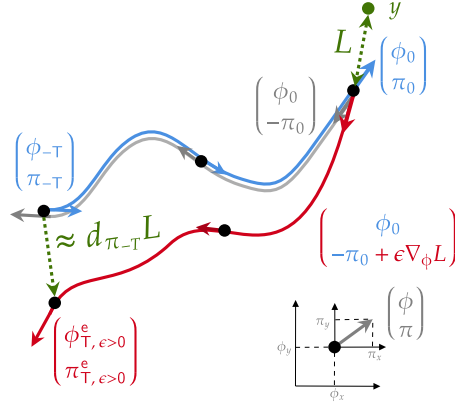
Deep State Space Models (SSMs) reignite physics-grounded compute paradigms, as RNNs could natively be embodied into dynamical systems. This calls for dedicated learning algorithms obeying to core physical principles, with efficient techniques to simulate these systems and guide their design. We propose *Recurrent Hamiltonian Echo Learning* (RHEL), an algorithm which provably computes loss gradients as finite differences of physical trajectories of non-dissipative, *Hamiltonian systems*. In ML terms, RHEL only requires three “forward passes” irrespective of model size, without explicit Jacobian computation, nor incurring any variance in the gradient estimation. Motivated by the physical realization of our algorithm, we first introduce RHEL in *continuous time* and demonstrate its formal equivalence with the continuous adjoint state method. To facilitate the simulation of Hamiltonian systems trained by RHEL, we propose a *discrete-time* version of RHEL which is equivalent to Backpropagation Through Time (BPTT) when applied to a class of recurrent modules which we call *Hamiltonian Recurrent Units* (HRUs). This setting allows us to demonstrate the scalability of RHEL by generalizing these results to hierarchies of HRUs, which we call *Hamiltonian SSMs* (HSSMs). We apply RHEL to train HSSMs with linear and nonlinear dynamics on a variety of time-series tasks ranging from mid-range to long-range classification and regression with sequence length reaching  $\sim 50k$ . We show that RHEL consistently matches the performance of BPTT across all models and tasks. This work opens new doors for the design of scalable, energy-efficient physical systems endowed with self-learning capabilities for sequence modelling.

## 1 Introduction

The resurgence of Recurrent Neural Networks (RNNs) for sequence modeling [1], particularly when integrated with State Space Models (SSMs) [1–6], reopens the debate about the “hardware lottery”. This raises a critical question with high stakes [7]: should future hardware development continue to optimize for the long-dominant GPU/TPU-Transformer-backprop paradigm [8], or should it explore alternative computational approaches alongside novel training algorithms?

This paper embraces an extreme view in the realm of AI hardware by not distinguishing algorithms from hardware [9] and posits that some SSMs could inherently be mapped onto dynamical physical systems and, by designing bespoke temporal credit assignment algorithms, be turned into “self-learning” machines [10, 11]. To this end, such algorithms should fulfill at least two requirements: i) use “forward passes” only (*i.e.* no backward passes), ii) do not use explicit state-Jacobian (*i.e.* Jacobian of the system’s dynamics). The vast majority of existing algorithms endowed with these features revolve around forward-mode automatic differentiation (AD) [12]. The standard forward-mode AD algorithm for temporal processing is Recurrent Real-Time Learning [13] (RTRL). Due to its cubic memory complexity with respect to the number

of neurons, RTRL can only be exactly implemented on architectures of limited size [14] and either requires low-rank approximations [15], or hardcoded [16] or metalearned [17] heuristics. RTRL fails to satisfy criteria ii) because it explicitly uses the Jacobian to compute directional derivatives of the loss  $L$  along each direction  $v_i$  of the canonical basis of parameters  $\theta$  as a supplementary computation during the forward pass. However, it can be reformulated as a zeroth-order procedure by approximating the directional derivatives via  $\nabla_{\theta} L \cdot v = \lim_{\epsilon \rightarrow 0} \epsilon^{-1} (L(\theta + \epsilon v) - L(\theta))$ . This recipe theoretically aligns with our three algorithmic requirements but incurs a prohibitive complexity cost by requiring separate forward passes for each perturbation along every parameter direction. Alternative methods attempt to circumvent this by sampling random directions from the parameter space [18–21] rather than exhaustively computing gradients along the entire canonical basis. However, these approaches suffer from either excessive variance [22] or high bias [23], limiting their effectiveness to small-scale applications [24] or fine-tuning tasks [25]. To avoid the pitfalls of RTRL and other forward-mode AD proxies, a better approach would be to instead emulate *backward-mode* AD, *forward* in time [26]. Yet, such techniques have only developed to emulate backward-mode *implicit differentiation* on energy-based models on static inputs [27] and have not been extended out of equilibrium to sequential data.



**Figure 1: HEB core mechanics [10].** After following a free forward trajectory (blue curve) and undergoing “rebound”, the neurons exactly travel backward (grey curve). Instead, HEB prescribes nudging the “echo” trajectory (red curve) closer to  $y$  before rebound, with the resulting position gap encoding the error gradient with respect to momentum (the left green arrow).

In this work, we take a radically different approach by emulating *backward-mode* AD *forward* in time taking inspiration from the Hamiltonian Echo Backprop (HEB) algorithm [10], which we illustrate on Fig. 1 for a single neuron. Consider a system of neurons described by their position  $\phi_i$  and momentum  $\pi_i$  which do not dissipate energy. This means, for instance for coupled oscillators (Eq. (8)), that the initial energy provided to this system is preserved and transferred across oscillators from kinetic energy to elastic potential or *vice versa*. Such systems can be modelled by the *Hamiltonian formalism* (section 2.1). Let us say that we want to learn the initial conditions on this system such that it reaches some target  $y$  after some time. HEB proceeds in three steps. For the first step, the system evolves freely, but does not reach  $y$ . For the second step, HEB perturbs the trajectory for a brief duration to drive the system slightly closer to  $y$  per some metric  $L$ . Then in the last step, the neurons are “bounced” backwards, causing the system to evolve backward. If the perturbation was omitted, the system would *exactly* retrace its previous trajectory backward in time, a property which is called *time-reversal symmetry*. However, because of the perturbation, the system does not exactly travel backward to its initial state, and this gap encodes the gradient of  $L$  with respect to its initial state.

In spite of its tremendous implications for physical learning, HEB has multiple features which may hinder its broader investigation within the ML community: i) HEB is difficult to compare to standard ML algorithmic baselines as it was derived with dedicated theoretical physics tools and is entangled with fine-grained physics of the systems being trained; ii) HEB theory assumes that model parameters are also dynamical variables and that only their initial state is learnable, which is in stark contrast with standard RNN parametrization; iii) it is unclear how HEB would extend to SSMs, *i.e.* in *discrete time*, on sequential data with losses defined at all timesteps, on *hierarchical* recurrent units; iv) finally, HEB was only evaluated on small static problems (*e.g.* XOR, MNIST) and not proved at larger scale.

Taking inspiration from HEB and using standard ML tools, we propose *Recurrent Hamiltonian Echo Learning* (RHEL) as a simple, general and scalable forward-only proxy of backward-mode AD applying to a broad class of dissipative-free Hamiltonian models with the following key contributions:

- With the primary goal in mind to inform the design of self-learning physical systems and to facilitate its comparison to HEB, we first introduce RHEL in *continuous time*. We show that RHEL generalizes HEB to a broader class of models and problems and demonstrate its equivalence, at *every timestep*, with the *continuous adjoint state method* [28] in the limit of

small trajectory perturbations (Theorem 3.1). We numerically highlight this property on a toy physical model (Eq. (8), Fig. 2).

- To efficiently simulate this algorithm, we extend RHEL to *discrete time* and demonstrate its equivalence with *Backpropagation Through Time* (BPTT) (Theorem 3.2) on *Hamiltonian Recurrent Units* (HRUs) – which are discrete-time, symplectic integrators of *separable* Hamiltonian dynamics. We further generalize this result to learning a *hierarchy* of HRUs, which we call *Hamiltonian State Space Models* (HSSMs, Fig. 2), and propose a *RHL chaining* procedure accordingly (Theorem 3.3).
- With this simulation toolbox in hand, we finally demonstrate the effectiveness of the proposed approach on HSSMs with linear [29] and nonlinear [30] recurrent units. We show that: i) gradients estimates produced by RHEL near perfectly match gradients computed by end-to-end AD (Fig. 4), ii) RHEL remains on par with AD in terms of resulting model performance across classification and regression long-range tasks (Tables 1–2).

## 2 Problem statement

### 2.1 Notations & model description.

**Notations.** Given a differentiable mapping  $H : \mathbb{R}^d \rightarrow \mathbb{R}$ , we denote  $\nabla_{v_1} H \in \mathbb{R}^{d_{v_1} \times 1}$  and  $\nabla_{v_1, v_2}^2 H \in \mathbb{R}^{d_{v_1} \times d_{v_2}}$  the gradient and Hessian of  $H$  respectively. When necessary, we use the Leibniz notation to denote as  $\nabla_i H$  the gradient of  $H$  with respect to the  $i^{\text{th}}$  variable. When considering a differentiable mapping  $\mathcal{M} : \mathbb{R}^d \rightarrow \mathbb{R}^n$ , we denote its Jacobian matrix as  $\partial \mathcal{M} \in \mathbb{R}^{n \times d}$ . We also use the notation  $d_{v_1}$  to denote a *total* derivative with respect to some variable  $v_1$  which accounts for both direct and *indirect* effects of the variable  $v_1$  on the function being differentiated.

**Continuous Hamiltonian model.** Following [10], we model neurons as a vector  $\Phi \in \mathcal{S} \equiv \mathbb{R}^{d_\Phi}$  comprising a position vector  $\phi \in \mathbb{R}^{\frac{d_\Phi}{2}}$  and a momentum vector  $\pi \in \mathbb{R}^{\frac{d_\Phi}{2}}$  such that  $\Phi := (\phi^\top, \pi^\top)^\top$ . We define  $\Sigma_z \in \mathbb{R}^{d_\Phi \times d_\Phi}$  such that  $\Sigma_z \cdot \Phi = (\phi, -\pi)$  and  $\Sigma_x \in \mathbb{R}^{d_\Phi \times d_\Phi}$  such that  $\Sigma_x \cdot \Phi = (\pi, \phi)$ . Denoting  $\theta \in \Theta \equiv \mathbb{R}^{d_\theta}$  and  $u \in \mathcal{U} \equiv \mathbb{R}^{d_u}$  the model parameters and inputs, we define the *Hamiltonian* associated to the model as a mapping  $H : \mathcal{S} \times \Theta \times \mathcal{U} \rightarrow \mathbb{R}$ . Taking  $-T$  as the origin of time and starting from  $\Phi(-T) = x$ , we say that  $\Phi$  follows *Hamiltonian dynamics* under a sequence of inputs  $t \rightarrow u(t)$  and with some parameters  $\theta$  if it satisfies the ordinary differential equation (ODE):

$$\forall t \in [-T, 0] : \partial_t \Phi(t) = J \cdot \nabla_\Phi H[\Phi(t), \theta, u(t)], \quad J := \begin{bmatrix} 0 & I \\ -I & 0 \end{bmatrix}. \quad (1)$$

**Assumptions.** The most fundamental requirement of the whole proposed approach for the Hamiltonian models at use is *time-reversal symmetry*. Heuristically, if we were recording the dynamics of a conservative system described by Eq. (1) and playing the resulting recording forward and backward, we could not distinguish them as both are *physically feasible* (Lemma A.3). A direct consequence of this fact is that if neurons are let to evolve for some time and then bounced back, *i.e.* their momenta are reversed, they will *exactly* travel back to their initial state (Corollary A.3). Namely (see Fig. 1):

$$\left( \Phi(-T) \xrightarrow{\text{Eq. (1)}} \Phi(0) \right) \Rightarrow \left( \Phi^*(0) := \Sigma_z \cdot \Phi(0) \xrightarrow{\text{Eq. (1)}} \Phi^*(-T) \right) \quad (2)$$

### 2.2 Learning as constrained optimization

**Problem formulation.** Our goal can be framed as a constrained optimization problem where we aim to find a set of model parameters  $\theta$  such that, under some input sequence  $t \rightarrow u(t)$ , the model trajectory approaches as much as possible some target trajectory, as measured by a *loss function*  $L$ , under the constraint that the model trajectory is *physically feasible* in the sense of satisfying Eq. (1):

$$\min_{\theta} L := \int_{-T}^0 dt \ell[\Phi(t), t] \quad \text{s.t.} \quad \forall t \in [-T, 0] : \partial_t \Phi(t) = J \cdot \nabla_\Phi H[\Phi(t), \theta, u(t)], \quad (3)$$

where  $L$  reads as the sum of *cost functions*  $t \rightarrow \ell(\cdot, t)$ . The most common approach to solve Eq. (3) is by gradient descent or variants thereof such that the problem boils down to computing  $d_\theta L$ . Note that

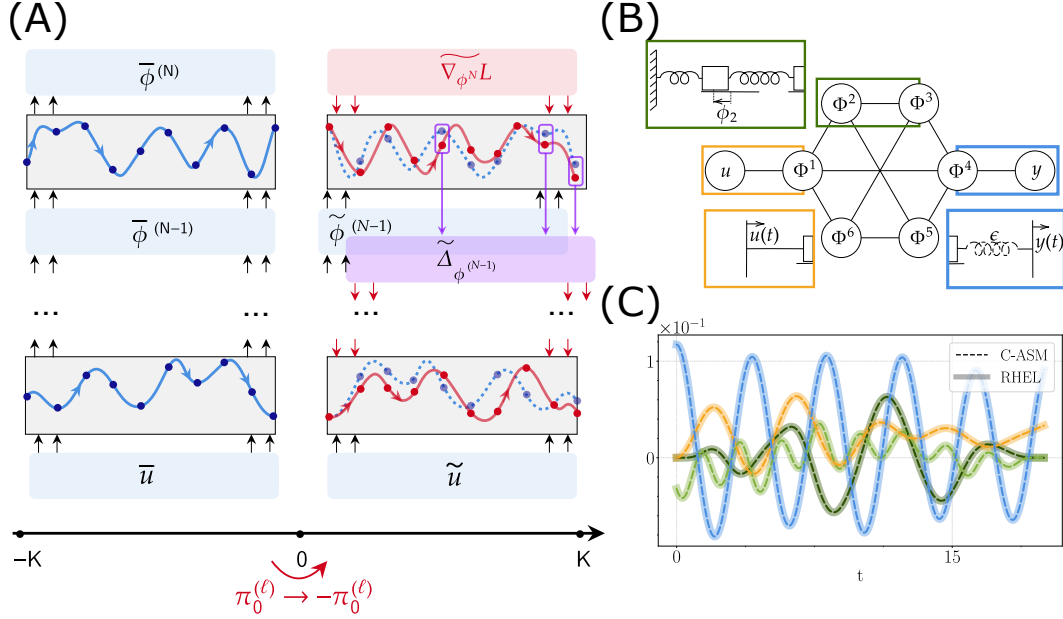


Figure 2: **(A): Learning Hamiltonian SSMs by RHEL.** The forward pass of a HSSM (left, Eq. (15)) reads as a composition of *Hamiltonian Recurrent Units* (HRUs, Eq. (9)). During the backward pass (right), the momenta of the neurons of the top-most HRU are flipped *and* nudged by the first error signal, yielding a perturbed trajectory (red curve). The contrast with the time-reversed trajectory (which would be obtained *without nudging*, dotted blue curve) yields an error signal that is passed backward (purple) to previous HRUs (Alg. 1). **(B)–(C): checking Theorem 3.1 on a toy model.** Six coupled harmonic oscillators with learnable parameters under input  $u$  and target  $y$  (Eq. (8)). Plots show gradients for a spring (dark green) and mass parameter (light green) comparing C-ASM ( $t \rightarrow g_{\theta}(t)$ , dotted) and RHEL ( $t \rightarrow \Delta_{\theta}(t)$ , solid), alongside sensitivities for oscillator positions  $\phi^1$  (orange) and  $\phi^4$  (blue) under both methods ( $t \rightarrow \lambda(t)$ , dotted;  $t \rightarrow \Delta_{\Phi}(t)$ , solid).

the problem defined by Eq. (3) is more general than the one solved by the seminal HEB work [10] as: i) the loss function  $L$  is defined over the *whole* trajectory, ii) the Hamiltonian is *time-dependent* (through  $t \rightarrow u(t)$ ) and parametrized by  $\theta$  which is shared across the whole computational graph.

**Algorithmic baseline.** One standard approach to compute  $d_{\theta}L$  is the *continuous-adjoint state method* (ASM) [28], which can be simply regarded as the continuous counterpart of backward-mode AD. Given some *forward* trajectory  $\{\Phi(t)\}_{t \in [-T, 0]}$  spanned by Eq. (1), this method prescribes solving the following *backward* ODE:

$$\lambda(0) = 0, \quad \partial_t \lambda(t) = \nabla_1^2 H[\Phi(-t), \theta, u(-t)] \cdot J^T \cdot \lambda(t) + \nabla_1 \ell[\Phi(-t), -t], \quad (4)$$

with the gradient of the loss with respect to the initial state of the neurons  $x$  and the model parameters  $\theta$  given by (Theorem A.1):

$$d_x L = \lambda(T), \quad d_{\theta} L = \int_0^T g_{\theta}(t) dt, \quad \text{with} \quad g_{\theta}(t) := \nabla_{1,2}^2 H[\Phi(-t), \theta, u(-t)] \cdot A^T \cdot \lambda(t) \quad (5)$$

Note that in theory, the forward trajectory  $\{\Phi(t)\}_{t \in [-T, 0]}$  can either be stored or recomputed backwards alongside  $\lambda$  yielding  $\mathcal{O}(1)$  memory cost – a property which holds beyond Eq. (1) specifically [31]. However in practice, recomputing variables backward through a *discrete* computational graph is generally inexact and induces bias in the gradient estimation [32], unless a *reversible* discretization scheme is used. Fortunately, *symplectic* integrators associated with Hamiltonian flows are reversible [33], a property which has been leveraged to yield memory savings in neural networks [30]. Therefore, the continuous ASM *as well as* our proposed algorithm (Alg. 1) also naturally inherit this memory efficiency as a *model feature* rather than a feature of the training algorithms themselves.

### 3 Recurrent Hamiltonian Echo Learning (RHEL)

#### 3.1 Definition in continuous time & equivalence with continuous ASM

We are now equipped to introduce *Recurrent Hamiltonian Echo Learning* (RHEL). In comparison to the continuous ASM, RHEL does not require solving a separate adjoint ODE akin to Eq. (4). Instead, RHEL simply prescribes running multiple additional times the forward trajectory Eq. (1) with three modifications: i) the neurons are first *conjugated*, i.e.  $\Phi \rightarrow \Phi^*$ ; ii) the inputs  $t \rightarrow \mathbf{u}(-t)$  are processed *backwards*; iii) the trajectory of the neurons is slightly nudged, with a strength  $\epsilon$ , towards direction of decreasing loss values with cost functions  $t \rightarrow \ell[\cdot, -t]$  processed *backwards* too. More precisely, let us assume a “free” forward trajectory Eq. (1) has been executed between  $-T$  and 0 yielding some neuron state  $\Phi(0)$ . Then, we define the *echo* dynamics of the neurons  $\Phi^e$  through:

$$\Phi^e(0) = \Phi^*(0), \quad \partial_t \Phi^e(t, \epsilon) = \mathbf{J} \nabla_{\Phi^e} H[\Phi^e(t, \epsilon), \boldsymbol{\theta}, \mathbf{u}(-t)] - \epsilon \mathbf{J} \nabla_{\Phi^e} \ell[\Phi^e(t, \epsilon), -t] \quad \forall t \in [0, T] \quad (6)$$

A crucial observation is that when  $\epsilon = 0$ , the echo trajectory simply matches the forward trajectory in reverse:  $\Phi^e(t, \epsilon = 0) = \Phi^*(-t)$  for  $t \in [0, T]$  (Lemma A.3). When  $\epsilon \neq 0$ , the resulting trajectory difference  $\Phi^e(t, \epsilon) - \Phi^e(t, \epsilon = 0)$  implicitly encodes for the error signals carried by the continuous ASM. We introduce this result more formally in Theorem 3.1 with the help of the following quantities capturing the essence of RHEL as a *forward-only*, difference-based gradient estimator:

$$\begin{cases} \Delta_{\boldsymbol{\theta}}^{\text{RHEL}}(t, \epsilon) &:= -\frac{1}{2\epsilon} (\nabla_2 H[\Phi^e(t, \epsilon), \boldsymbol{\theta}, \mathbf{u}(-t)] - \nabla_2 H[\Phi^e(t, -\epsilon), \boldsymbol{\theta}, \mathbf{u}(-t)]), \\ \Delta_{\Phi}^{\text{RHEL}}(t, \epsilon) &:= \frac{1}{2\epsilon} \Sigma_x \cdot (\Phi^e(t, \epsilon) - \Phi^e(t, -\epsilon)), \end{cases} \quad (7)$$

**Theorem 3.1** (Informal). *Under mild assumptions on the Hamiltonian function and with  $t \rightarrow \lambda(t)$  and  $t \rightarrow g_{\boldsymbol{\theta}}(t)$  defined by Eq. (4)–(5), the continuous ASM and RHEL are equivalent at all times in the sense that:*

$$\forall t \in [0, T], \quad \lambda(t) = \lim_{\epsilon \rightarrow 0} \Delta_{\Phi}^{\text{RHEL}}(t, \epsilon), \quad g_{\boldsymbol{\theta}}(t) = \lim_{\epsilon \rightarrow 0} \Delta_{\boldsymbol{\theta}}^{\text{RHEL}}(t, \epsilon)$$

*Proof sketch.* Defining  $\Delta_{\Phi}^{\text{RHEL}}(t) := \lim_{\epsilon \rightarrow 0} \Delta_{\Phi}^{\text{RHEL}}(t, \epsilon)$  and noticing that  $\Delta_{\Phi}^{\text{RHEL}}(t) = \Sigma_x \cdot \partial_{\epsilon}(\Phi^e(t, \epsilon))|_{\epsilon=0}$ , we can show by differentiating Eq. (6) around  $\epsilon = 0$  and with some algebra that  $t \rightarrow \Delta_{\Phi}^{\text{RHEL}}(t)$  satisfies the same ODE as  $t \rightarrow \lambda(t)$  (Eq. (4) and have same initial conditions, therefore are equal at all times. The other two equalities of Theorem 3.1 can then be easily deduced.  $\square$

**Example.** We numerically Theorem 3.1 on a toy model of six coupled harmonic oscillators  $i$  (Fig. 2) with mass  $m_i$  and spring parameters  $k_i, k_{ij}$ . This system is described by the Hamiltonian  $H[\Phi, \boldsymbol{\theta}, \mathbf{u}] = \sum_i \frac{\pi_i^2}{2m_i} + \frac{1}{2} \sum_i k_i \phi_i^2 + \frac{1}{2} \sum_i \sum_{j,j>i} k_{ij} (\phi_j - \phi_i)^2$  with  $\boldsymbol{\theta} = \{m_i, k_i, k_{ij}\}_{i,j}$  that leads to the dynamic and associated gradient estimators (see App. A.3.1):

$$\begin{cases} \forall i \in \llbracket 1, d_{\Phi} \rrbracket & \partial_t \phi_i = \pi_i / m_i, \quad \partial_t \pi_i = -k_i \phi_i + \sum_{j,j \neq i} k_{ij} (\phi_j - \phi_i) \\ g_{k_{ij}}(t) &= \lim_{\epsilon \rightarrow 0} \frac{-1}{2\epsilon} \left( (\phi_j^e(t, \epsilon) - \phi_i^e(t, \epsilon))^2 - (\phi_j^e(t, -\epsilon) - \phi_i^e(t, -\epsilon))^2 \right) \end{cases} \quad (8)$$

#### 3.2 Extension to discrete time & equivalence with BPTT

In this section, we propose a *discrete-time* version of RHEL by first defining a family of discrete models as integrators of the Hamiltonian flow which *exactly* preserve time-reversal symmetry, defining an associated surrogate problem and *then* solving it. This “discretize-then-optimize” approach ensures a better gradient estimation than directly discretizing RHEL theory in continuous time [32].

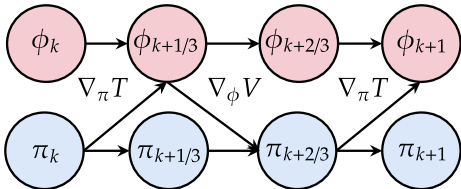


Figure 3: Computational graph of  $\mathcal{M}_{H,\delta}$ .

**Hamiltonian Recurrent Units (HRUs).** We now assume that the neurons  $\Phi$  obey the following discrete-time dynamics, starting from  $\Phi_{-K} = \mathbf{x}$  and:

$$\Phi_{k+1} = \mathcal{M}_{H,\delta}[\Phi_k, \boldsymbol{\theta}, \mathbf{u}_k] \quad \forall k = -K \cdots -1, \quad (9)$$

where  $\mathcal{M}_{H,\delta}$  denotes the *integrator* associated with the Hamiltonian function  $H$  with time discretization  $\delta$ . We classically pick  $\mathcal{M}_{H,\delta}$  as the

*Leapfrog* integrator [33] and restrict ourselves to *separable* Hamiltonians of the form  $H[\Phi] = T[\pi] + V[\phi]$  to yield a reversible *and* explicit integration scheme. One possible parametrization of the Leapfrog integrator in this case is as the composition of *three explicit Euler integrators*, which alternate between updates of the position  $\phi$  (first and third steps) and of the momentum (second step) – see Def. A.3 for a detailed description and Fig. 3 for the associated computational graph. As these two intermediate integrator time steps are needed to accurately define the RHEL learning rule in discrete time, we explicitly denote them as fractional time units:

$$\mathcal{M}_{H,\delta} : \Phi_k \longrightarrow \Phi_{k+1/3} \longrightarrow \Phi_{k+2/3} \longrightarrow \Phi_{k+1} \quad (10)$$

We call such models *Hamiltonian Recurrent Units* (HRUs). Therefore by design, the time-reversal symmetry property defined in Eq. (2) extends in discrete time to HRUs (Corollary A.6).

**Surrogate learning problem.** Given this modelling choice, the continuous-time problem introduced in Eq. (3) naturally translates here in discrete time as:

$$\min_{\theta} L := \sum_{k=-K+1}^0 \ell[\Phi_k, k] \quad \text{s.t.} \quad \Phi_{k+1} = \mathcal{M}_{H,\delta}[\Phi_k, \theta, \mathbf{u}_k] \quad \forall k = -K \cdots -1 \quad (11)$$

**Algorithmic baseline.** Eq. (11) defines a classical RNN learning problem where *Backpropagation Through Time* (BPTT), *i.e.* the instantiation of backward-mode AD in this context, is a natural algorithmic baseline. BPTT simply amounts to apply the “chain rule” backward through the computational graph spanned by the forward pass of a HRU (Eq. (9)). Alternatively, it can also be regarded as the discrete counterpart of the continuous ASM previously introduced (Theorem A.3). For this reason, we re-use the same notations as above and define the following quantities associated to BPTT:

$$d_{\theta} L = \sum_{k=0}^{K-1} g_{\theta}(k), \quad g_{\theta}(k) := d_{\theta_k} L, \quad g_{\mathbf{u}}(k) := d_{\mathbf{u}_{-k}} L, \quad \lambda_k := d_{\Phi_{-k}} L, \quad (12)$$

where  $g_{\theta}(k)$ ,  $g_{\mathbf{u}}(k)$  and  $\lambda_k$  denote the “sensitivity” of the loss  $L$  to  $\theta$  at time step  $k$ ,  $\mathbf{u}_{-k}$  and  $\Phi_{-k}$  respectively – see App. A.2.2 for a more detailed definition and derivation of BPTT.

**RHEL in discrete time.** Finally, we extend RHEL in discrete-time by defining the echo dynamics on HRUs as:

$$\begin{cases} \Phi_0^e(\epsilon) = \Phi_0^* + \epsilon \Sigma_x \cdot \nabla_{\Phi} \ell[\Phi_0, 0], \\ \Phi_{k+1}^e(\epsilon) = \mathcal{M}_{H,\delta}[\Phi_k^e(\epsilon), \theta, \mathbf{u}_{-(k+1)}] - \epsilon \mathbf{J} \cdot \nabla_{\Phi^e} \ell[\Phi_{k+1}^e, -(k+1)] \quad \forall k = 0, \dots, K-1 \end{cases} \quad (13)$$

Denoting:

$$H^{1/2}[\Phi_k^e(\epsilon), \theta, \mathbf{u}_{-(k+1)}] := \frac{1}{2} \left( H[\Phi_{k+1/3}^e(\epsilon), \theta, \mathbf{u}_{-(k+1)}] + H[\Phi_{k+2/3}^e(\epsilon), \theta, \mathbf{u}_{-(k+1)}] \right), \quad (14)$$

we can define the discrete-time counterpart of Eq. (7) as:

$$\begin{cases} \Delta_{\theta}^{\text{RHEL}}(k, \epsilon) &:= -\frac{\delta}{2\epsilon} \left( \nabla_2 H^{1/2}[\Phi_k^e(\epsilon), \theta, \mathbf{u}_{-(k+1)}] - \nabla_2 H^{1/2}[\Phi_k^e(-\epsilon), \theta, \mathbf{u}_{-(k+1)}] \right) \\ \Delta_{\mathbf{u}}^{\text{RHEL}}(k, \epsilon) &:= -\frac{\delta}{2\epsilon} \left( \nabla_3 H^{1/2}[\Phi_k^e(\epsilon), \theta, \mathbf{u}_{-(k+1)}] - \nabla_3 H^{1/2}[\Phi_k^e(-\epsilon), \theta, \mathbf{u}_{-(k+1)}] \right) \\ \Delta_{\Phi}^{\text{RHEL}}(k, \epsilon) &:= \frac{1}{2\epsilon} \Sigma_x \cdot (\Phi_k^e(\epsilon) - \Phi_k^e(-\epsilon)) \end{cases},$$

and consequently extend Theorem 3.1 in discrete time as well.

**Theorem 3.2 (Informal).** *Under mild assumptions on the Hamiltonian function and with  $(\lambda_k)_{k \in [0, K-1]}$ ,  $(g_{\theta}(k))_{k \in [0, K-1]}$  and  $(g_{\mathbf{u}}(k))_{k \in [0, K-1]}$  defined by Eq. (12), BPTT and RHEL are equivalent at all times in the sense that  $\forall k = 0, \dots, K$ :*

$$\lambda_k = \lim_{\epsilon \rightarrow 0} \Delta_{\Phi}^{\text{RHEL}}(k, \epsilon), \quad g_{\theta}(k) = \lim_{\epsilon \rightarrow 0} \Delta_{\theta}^{\text{RHEL}}(k, \epsilon), \quad g_{\mathbf{u}}(k) = \lim_{\epsilon \rightarrow 0} \Delta_{\mathbf{u}}^{\text{RHEL}}(k, \epsilon).$$

### 3.3 Learning stacks of HRUs via RHL chaining

**Vectorized notations.** Given a discrete vector field  $(\mathbf{V}_k)_{k \in [-K, 0]}$ , we denote  $\bar{\mathbf{V}} := (\mathbf{V}_{-K}^{\top}, \dots, \mathbf{V}_0^{\top})^{\top} \in \mathbb{R}^{T \times d_V}$  and  $\tilde{\mathbf{V}} := (\mathbf{V}_0^{\top}, \dots, \mathbf{V}_{-K}^{\top})^{\top} \in \mathbb{R}^{T \times d_V}$  the forward and backward trajectories associated with  $\mathbf{V}$ . We define the vectorized operator  $\mathcal{M}_{H,\delta}$  such that the dynamics of a single HRU unit as defined in Eq. (9) and the nudged dynamics prescribed by RHEL as given in Eq. (13) rewrite more compactly as:

$$\bar{\Phi} = \mathcal{M}_{H,\delta}[\bar{\Phi}, \theta, \bar{\mathbf{u}}], \quad \bar{\Phi}^e = \mathcal{M}_{H,\delta}[\bar{\Phi}^e, \theta, \tilde{\mathbf{u}}] - \epsilon \mathbf{J} \cdot \nabla_{\bar{\Phi}^e} \tilde{L}[\bar{\Phi}^e].$$



**Learning Hamiltonian State Space Models (HSSMs) as multi-level optimization.** We now consider *hierarchical* models reading as a composition of  $N$  HRU units of the form:

$$\Phi^{(0)} := \bar{\mathbf{u}}, \quad \forall \ell \in \llbracket 0, N-1 \rrbracket : \bar{\Phi}^{(\ell+1)} = \mathcal{M}_{H^{(\ell)}, \delta}^{(\ell)}[\bar{\Phi}^{(\ell+1)}, \theta^{(\ell)}, \bar{\Phi}^{(\ell)}], \quad (15)$$

where the trajectory of the  $(\ell)$ -th HRU unit is fed as an input sequence into the  $(\ell+1)$ -th HRU unit (Fig. 2). We call such hierarchies of HRUs *Hamiltonian State Space Models* (HSSMs). The corresponding optimization problem reads:

$$\min_{\theta} L := \sum_{k=-K+1}^0 \ell[\Phi_k^{(N)}, k] \quad \text{s.t.} \quad \bar{\Phi}^{(\ell+1)} = \mathcal{M}_{H^{(\ell)}, \delta}^{(\ell)}[\bar{\Phi}^{(\ell+1)}, \theta^{(\ell)}, \bar{\Phi}^{(\ell)}] \quad \forall \ell \in \llbracket 0, N-1 \rrbracket \quad (16)$$

Alg. 1 prescribes an intuitive recipe to compute  $d_{\theta}L$  for the above optimization problem (Eq. (16)), chaining RHEL backward through HRUs. Namely, the echo dynamics of the top-most HRU read as Eq. (13) using the initial learning signal  $\nabla_{\Phi}L$  to nudge its trajectory. On top of estimating its parameter gradients, we also estimate its *input* gradients which are used to nudge the echo dynamics of the preceding HRU. This procedure is repeated until reaching the first HRU – see Fig. 2.

---

**Algorithm 1** Recurrent Hamiltonian Echo Learning (RHEL) on a single HRU

---

*Inputs:*  $\Phi_0$  (final state of the forward trajectory),  $\Delta_{\Phi}$  (incoming gradient),  $\epsilon$  (nudging strength),  $\delta$  (timestep)

*Outputs:*  $\Delta\theta$  (parameter gradient estimate),  $\Delta_{\mathbf{u}}$  (input gradient estimate)

- 1:  $\Delta\theta \leftarrow \mathbf{0}_{d_{\theta}}$
  - 2:  $\Delta_{\mathbf{u}} := (\Delta_{\mathbf{u}_{-K}}, \dots, \Delta_{\mathbf{u}_0}) \leftarrow \mathbf{0}_{K \times d_{\Phi}}$
  - 3:  $\Phi_{\pm\epsilon}^e \leftarrow \Phi_0^* \pm \epsilon \Sigma_x \cdot \Delta_{\Phi, 0}$  ▷ Compute  $\Phi_{\pm\epsilon}^e$  in parallel
  - 4: **for**  $k$  in  $0, \dots, K$  **do**
  - 5:    $\Phi_{\pm\epsilon}^e \leftarrow \mathcal{M}_{H, \delta}[\Phi_{\pm\epsilon}^e, \theta, \mathbf{u}_{-(k+1)}] \pm \epsilon \Sigma_x \cdot \Delta_{\Phi, -(k+1)}$
  - 6:    $\Delta\theta \leftarrow \Delta\theta - \frac{\delta}{2\epsilon} (\nabla_2 H^{1/2}[\Phi_{\epsilon}^e, \theta, \mathbf{u}_{-(k+1)}] - \nabla_2 H^{1/2}[\Phi_{-\epsilon}^e, \theta, \mathbf{u}_{-(k+1)}])$  ▷ Eq. (14)
  - 7:    $\Delta_{\mathbf{u}_{-k}} \leftarrow -\frac{\delta}{2\epsilon} (\nabla_3 H^{1/2}[\Phi_{\epsilon}^e, \theta, \mathbf{u}_{-(k+1)}] - \nabla_3 H^{1/2}[\Phi_{-\epsilon}^e, \theta, \mathbf{u}_{-(k+1)}])$
  - 8: **end for**
  - 9: **return**  $\Delta\theta, \Delta_{\mathbf{u}}$
- 

**Theorem 3.3** (Informal). *Given  $\bar{\mathbf{u}}$  an input sequence and a HSSM with layer-wise Hamiltonians  $\{H^{(\ell)}\}_{k \in \llbracket 1, N \rrbracket}$ , applying Alg. 1 recursively backward from the top-most HRU solves the multilevel optimization problem defined in Eq. (16) in the sense that:*

$$\forall \ell = 0, \dots, N : \quad d_{\theta^{(\ell)}}L = \lim_{\epsilon \rightarrow 0} \Delta\theta^{(\ell)}(\epsilon)$$

## 4 Experiments

**Foreword.** We first introduce the two types HRUs at use, empirically assess the validity of Theorem 3.3 by statically comparing gradients computed by RHEL and BPTT on HSSMs made up of these two types of HRUs, then perform training experiments across classification and regression sequence tasks. Importantly, the goal of the proposed experiments is *not* to improve the SOTA performance on these tasks. Rather we want to show, on a *given* model satisfying the requirements of RHEL (*i.e.* being a HRU or a stack thereof), that RHEL maintains training performance *with respect to BPTT*.

**Linear HRU block.** Following recent work [29], we introduce the *linear* HRU block with the parametrization  $\theta^{(\ell)} = \{\mathbf{A}^{(\ell)}, \mathbf{B}^{(\ell)}, \mathbf{W}^{(\ell)}\}$  and Hamiltonian:

$$\begin{cases} H^{(\ell)}[\Phi^{(\ell)}, \theta^{(\ell)}, \Phi^{(\ell-1)}] = \frac{1}{2} \|\pi^{(\ell)}\|^2 + \frac{1}{2} \phi^{(\ell)\top} \cdot \mathbf{A}^{(\ell)} \cdot \phi^{(\ell)} - \phi^{(\ell)\top} \cdot \mathbf{B}^{(\ell)} \cdot \mathbf{u}^{(\ell)}, \\ \mathbf{u}^{(\ell)} = F[\phi^{(\ell-1)}, \mathbf{W}^{(\ell)}], \end{cases} \quad (17)$$

where  $\mathbf{A} \in \mathbb{R}^{(d_{\Phi}/2) \times (d_{\Phi}/2)}$  is assumed to be *diagonal* and to have positive entries,  $\mathbf{B}^{(\ell)} \in \mathbb{R}^{(d_{\Phi}/2) \times d_u}$ ,  $\alpha \in \mathbb{R}$ , and  $F$  is the nonlinear spatial building block parametrized by  $\mathbf{W}^{(\ell)}$  (see App. A.3.3). Under these assumptions, it can be shown that such HRUs have a bounded spectrum and yield HSSMs which are universal approximators of continuous and causal operators between time-series [29].

**Nonlinear HRU block.** To demonstrate the validity of our approach beyond linear HRUs, we also introduce a *nonlinear* HRU block [30] with  $\theta^{(\ell)} = \{\mathbf{A}^{(\ell)}, \mathbf{W}^{(\ell)}, \mathbf{B}^{(\ell)}, \mathbf{b}^{(\ell)}, \alpha^{(\ell)}\}$  and:

$$\begin{cases} H^{(\ell)}[\Phi^{(\ell)}, \theta^{(\ell)}, \Phi^{(\ell-1)}] = \frac{1}{2}\|\pi^{(\ell)}\|^2 + \frac{\alpha^{(\ell)}}{2}\|\phi^{(\ell)}\|^2 \\ \quad + (\mathbf{A}^{(\ell)})^{-\top} \cdot \log(\cosh(\mathbf{A}^{(\ell)} \cdot \phi^{(\ell)} + \mathbf{B}^{(\ell)} \cdot \mathbf{u}^{(\ell)} + \mathbf{b}^{(\ell)})), \\ \mathbf{u}^{(\ell)} = F[\phi^{(\ell-1)}, \mathbf{W}^{(\ell)}], \end{cases} \quad (18)$$

where  $\mathbf{A} \in \mathbb{R}^{(d_{\Phi}/2) \times (d_{\Phi}/2)}$  is also assumed to be *diagonal*,  $\mathbf{B}^{(\ell)} \in \mathbb{R}^{(d_{\Phi}/2) \times d_u}$ ,  $\mathbf{b}^{(\ell)} \in \mathbb{R}^{(d_{\Phi}/2)}$ ,  $\alpha \in \mathbb{R}$ , and  $F$  is the nonlinear spatial building block parametrized by  $\mathbf{W}^{(\ell)}$  (see App. A.3.4). It has been shown that these HRUs mitigate by design vanishing and exploding gradients.

#### 4.1 Static gradient comparison

As a sanity check of Theorem 3.3 and preamble to training experiments, we first check that RHEL gradients are computed correctly. Given a HSSM model, we randomly sample a tuple  $(\mathbf{u}_i, \mathbf{y}_i) \sim \mathcal{D}$  from the SCP1 dataset (see App. A.4.1 for details on this dataset) and pass  $\mathbf{u}_i$  through the model. Given  $\mathbf{y}$ , we then run BPTT and RHEL through the model and compare them in terms of cosine similarity and norm ratio of the resulting layer-wise parameter gradients. We run this experiment on a HSSM made up of six linear HRU blocks (which we call “linear HSSM”) and another HSSM comprising six nonlinear HRU blocks (resp. “nonlinear HSSM”) and obtain Fig. 4. We observe that in terms of these two comparison metrics, RHEL and BPTT parameter gradients are near-perfectly aligned for all parameters and across all HRU blocks, for both the linear and nonlinear HSSMs.

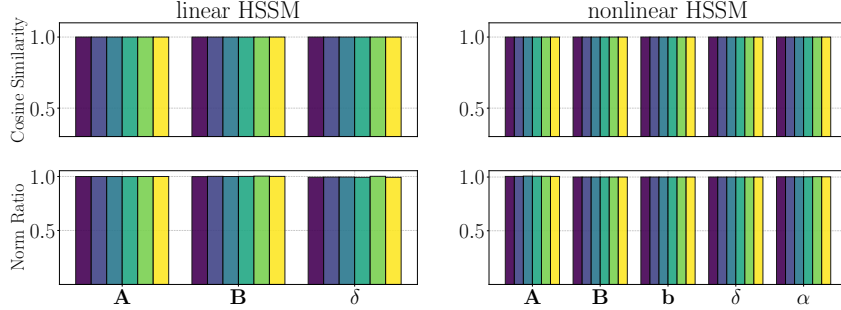


Figure 4: **Static comparison between RHEL and BPTT.** Given some  $(\mathbf{u}_i, \mathbf{y}_i) \sim \mathcal{D}$ , we perform BPTT and RHEL on six blocks-deep linear and nonlinear HSSMs. We measure, layer-wise (first layer in purple), the cosine similarity (top panels) and norm ratio (bottom pannels) between RHEL and BPTT parameter gradients of a linear HSSM (Eq. (17)) and a nonlinear HSSM, Eq. (18)).

#### 4.2 Training experiments

**Classification.** To further check our theoretical guarantees, we now perform training experiments with BPTT and RHEL across six multivariate sequence classification datasets with various sequence lengths (from  $\sim 400$  to  $\sim 18k$ ) and number of classes, on linear and nonlinear HSSMs (see App. A.4.1 for details on these datasets) and display our results in Table 1. This series of tasks was recently introduced [34] as a subset of the University of East Anglia (UEA) datasets [35] with the longest sequences for increased difficulty and recently used to benchmark the linear HSSM previously introduced [29]. We observe that models trained by RHEL almost match, on average across the datasets, those trained by BPTT in terms of resulting performance on the test dataset.

**Regression & scalability.** Finally, to assess the applicability of RHEL beyond classification and scalability to longer sequences, we run training experiments on the PPG-DaLiA dataset, a multivariate time series regression dataset designed for heart rate prediction using data collected from a wrist-worn device [36]. With sequence length of  $\sim 50k$ , this task is considered to be a difficult “long-range” benchmark [29]. We display in Table 2 the results obtained when training linear and nonlinear HSSMs with RHEL and BPTT on this dataset. Here again, we observe that on average and with both models, the performance of RHEL matches that of BPTT.



| Tasks       |      | Worms          | SCP1           | SCP2           | Ethanol        | Heartbeat      | Motor          |                                |
|-------------|------|----------------|----------------|----------------|----------------|----------------|----------------|--------------------------------|
| Seq. length |      | 17,984         | 896            | 1,152          | 1,751          | 405            | 3,000          |                                |
| # classes   |      | 5              | 2              | 2              | 4              | 2              | 2              | <b>Avg</b>                     |
| Lin         | BPTT | 78.3 $\pm$ 7.5 | 86.4 $\pm$ 1.8 | 63.9 $\pm$ 7.3 | 29.9 $\pm$ 0.6 | 73.9 $\pm$ 0.6 | 48.1 $\pm$ 5.7 | <b>63.4<math>\pm</math>3.9</b> |
|             | RHEL | 75.0 $\pm$ 9.9 | 86.1 $\pm$ 2.9 | 61.4 $\pm$ 9.4 | 29.9 $\pm$ 0.6 | 73.5 $\pm$ 1.6 | 51.6 $\pm$ 5.0 | <b>62.9<math>\pm</math>4.9</b> |
| Nonlin      | BPTT | 51.1 $\pm$ 7.2 | 86.8 $\pm$ 3.2 | 54.0 $\pm$ 4.9 | 29.9 $\pm$ 0.6 | 74.5 $\pm$ 2.4 | 56.5 $\pm$ 7.6 | <b>58.8<math>\pm</math>4.3</b> |
|             | RHEL | 50.6 $\pm$ 6.7 | 85.6 $\pm$ 4.4 | 54.0 $\pm$ 2.0 | 29.9 $\pm$ 0.6 | 73.9 $\pm$ 4.3 | 53.0 $\pm$ 5.7 | <b>57.8<math>\pm</math>4.0</b> |

Table 1: Test mean accuracy (% , higher is better) across five different seeds ( $\pm$  indicates standard deviation) using RHEL and BPTT, nonlinear and linear HSSMs, on six UEA time series classification datasets with various sequence length and number of classes.

## 5 Discussion

**Limitations.** We observe on Tables 1–2 that RHEL slightly underperforms BPTT, though this gap is statistically significant only for nonlinear HSSMs on regression tasks. This performance difference can be attributed to: i) the approximation bias introduced by finite nudging in our method, a known issue in related works [37], and ii) numerical precision that requires some careful selection of the nudging strength (see App. A.4.4).

**Related work.** On top of its direct connection with HEB [10], RHEL belongs to a large body Hamiltonian-based ML algorithms dedicated to physics-informed neural networks [38–40], memory-efficient models [30], generative models [41], sampling [42–44] and optimization [45] algorithms. RHEL also extends the design of scalable self-learning machines beyond *dissipative* systems emulating *implicit differentiation* [46–50]. In comparison to these algorithms, RHEL crucially avoids prohibitively long simulation times due to the use of lengthy root-finding algorithms, extends their core mechanics to the broader domain of sequence modelling and possibly larger-scale tasks.

**Future work.** Yet, the restriction of RHEL to non-dissipative systems limits model expressivity [29]. While inducing artificial dissipation through the use dissipative integrators [29] or by embedding a dissipative system within a larger conservative one [10] are convenient workarounds, there is ultimately a need for a temporal credit assignment algorithm tailored for *truly* dissipative systems. Additionally and in the spirit of HEB, an idealized version of RHEL should be entirely “black-box” as it still requires explicit Hamiltonian parameter gradients to implement its learning rule (Alg. 1). This could possibly be achieved for instance using homeostatic control *via* control knobs acting on the model parameters [51]. Another exciting direction of work is to have RHEL operate *online* – it currently requires one forward pass and subsequent ones revisiting the inputs in reverse order. Finally, while HRU hidden units can be recomputed from their final state [30], it still requires storing the input sequence of the HRU so that memory gains are not evident within a stack of HRUs. Looking beyond analog physical systems, endowing RHEL with better memory efficiency and online mechanics could potentially yield a compelling alternative to BPTT on *digital* accelerators like GPUs [14].

**Conclusion.** While we *simulated* Hamiltonian dynamics alongside RHEL on GPUs, the greatest potential of RHEL lies within *real* Hamiltonian systems, namely on-chip photonic circuits [52], superconducting circuits [53], or spintronic systems [54]. We hope this work will incentivize the search for alternative analog hardware capable of training sequence models with much higher energy efficiency, as well as alternative algorithms for digital hardware.

|                   |      | PPG-DaLiA     |
|-------------------|------|---------------|
| Seq. length       |      | 49,920        |
| Input/output dim. |      | 6/1           |
| Lin               | BPTT | 9.1 $\pm$ 1.1 |
|                   | RHEL | 9.5 $\pm$ 1.0 |
| Nonlin            | BPTT | 7.8 $\pm$ 0.5 |
|                   | RHEL | 8.4 $\pm$ 0.5 |

Table 2: Test average mean-square error ( $\times 10^{-2}$ , lower is better) across five different seeds ( $\pm$  indicates standard deviation) applying RHEL and BPTT to train nonlinear and linear HSSMs on the PPG-DaLiA dataset.

## Acknowledgments and Disclosure of Funding

The authors warmly thank (in alphabetic order) Aditya Gilra, Albert Cohen, Debabrota Basu and James Laudon for their support of the project. ME thanks Vincent Roulet for his careful review which greatly helped improve the quality of this manuscript, as well as Amaury Hayat and Alexandre Krajenbrink for useful discussions. GP acknowledges the CHIST-ERA grant for the CausalXRL project (CHIST-ERA-19-XAI-002) by L'Agence Nationale de la Recherche, France (grant reference ANR-21-CHR4-0007), and the ANR JCJC for the REPUBLIC project (ANR-22-CE23-0003-01).

## Author contributions

GP and ME collaborated closely on all aspects of this work. Both authors contributed to study design, with GP focusing primarily on experiments and code development, and ME focusing primarily on theoretical development and writing.

## References

- [1] Antonio Orvieto, Samuel L Smith, Albert Gu, Anushan Fernando, Caglar Gulcehre, Razvan Pascanu, and Soham De. Resurrecting recurrent neural networks for long sequences. In *International Conference on Machine Learning*, pages 26670–26698. PMLR, 2023.
- [2] Albert Gu, Isys Johnson, Karan Goel, Khaled Saab, Tri Dao, Atri Rudra, and Christopher Ré. Combining recurrent, convolutional, and continuous-time models with linear state space layers. *Advances in neural information processing systems*, 34:572–585, 2021.
- [3] Albert Gu, Karan Goel, and Christopher Ré. Efficiently modeling long sequences with structured state spaces. *arXiv preprint arXiv:2111.00396*, 2021.
- [4] Ankit Gupta, Albert Gu, and Jonathan Berant. Diagonal state spaces are as effective as structured state spaces. *Advances in Neural Information Processing Systems*, 35:22982–22994, 2022.
- [5] Jimmy TH Smith, Andrew Warrington, and Scott W Linderman. Simplified state space layers for sequence modeling. *arXiv preprint arXiv:2208.04933*, 2022.
- [6] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*, 2023.
- [7] Sara Hooker. The hardware lottery. *Communications of the ACM*, 64(12):58–65, 2021.
- [8] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.
- [9] Jérémie Laydevant, Logan G Wright, Tianyu Wang, and Peter L McMahon. The hardware is the software. *Neuron*, 112(2):180–183, 2024.
- [10] Victor Lopez-Pastor and Florian Marquardt. Self-learning machines based on hamiltonian echo backpropagation. *Physical Review X*, 13(3):031020, 2023.
- [11] Ali Momeni, Babak Rahmani, Benjamin Scellier, Logan G Wright, Peter L McMahon, Clara C Wanjura, Yuhang Li, Anas Skalli, Natalia G Berloff, Tatsuhiko Onodera, et al. Training of physical neural networks. *arXiv preprint arXiv:2406.03372*, 2024.
- [12] Atilim Gunes Baydin, Barak A Pearlmutter, Alexey Andreyevich Radul, and Jeffrey Mark Siskind. Automatic differentiation in machine learning: a survey. *Journal of machine learning research*, 18(153):1–43, 2018.
- [13] Ronald J Williams and David Zipser. A learning algorithm for continually running fully recurrent neural networks. *Neural computation*, 1(2):270–280, 1989.
- [14] Nicolas Zucchet, Robert Meier, Simon Schug, Asier Mujika, and João Sacramento. Online learning of long-range dependencies. *Advances in Neural Information Processing Systems*, 36: 10477–10493, 2023.

- [15] Corentin Tallec and Yann Ollivier. Unbiased online recurrent optimization. *arXiv preprint arXiv:1702.05043*, 2017.
- [16] Guillaume Bellec, Franz Scherr, Anand Subramoney, Elias Hajek, Darjan Salaj, Robert Legenstein, and Wolfgang Maass. A solution to the learning dilemma for recurrent networks of spiking neurons. *Nature communications*, 11(1):3625, 2020.
- [17] Jamie Lohoff, Anil Kaya, Florian Assmuth, and Emre Neftci. A truly sparse and general implementation of gradient-based synaptic plasticity. *arXiv preprint arXiv:2501.11407*, 2025.
- [18] Rie Johnson and Tong Zhang. Accelerating stochastic gradient descent using predictive variance reduction. *Advances in neural information processing systems*, 26, 2013.
- [19] David Kozak, Stephen Becker, Alireza Doostan, and Luis Tenorio. Stochastic subspace descent. *arXiv preprint arXiv:1904.01145*, 2019.
- [20] James C Spall. Multivariate stochastic approximation using a simultaneous perturbation gradient approximation. *IEEE transactions on automatic control*, 37(3):332–341, 1992.
- [21] Ila R Fiete, Michale S Fee, and H Sebastian Seung. Model of birdsong learning based on gradient estimation by dynamic perturbation of neural conductances. *Journal of neurophysiology*, 98(4): 2038–2057, 2007.
- [22] Mengye Ren, Simon Kornblith, Renjie Liao, and Geoffrey Hinton. Scaling forward gradient with local losses. *arXiv preprint arXiv:2210.03310*, 2022.
- [23] David Silver, Anirudh Goyal, Ivo Danihelka, Matteo Hessel, and Hado van Hasselt. Learning by directional gradient descent. In *International Conference on Learning Representations*, 2021.
- [24] Louis Fournier, Stéphane Rivaud, Eugene Belilovsky, Michael Eickenberg, and Edouard Oyallon. Can forward gradient match backpropagation? In *International Conference on Machine Learning*, pages 10249–10264. PMLR, 2023.
- [25] Sadhika Malladi, Tianyu Gao, Eshaan Nichani, Alex Damian, Jason D Lee, Danqi Chen, and Sanjeev Arora. Fine-tuning language models with just forward passes. *Advances in Neural Information Processing Systems*, 36:53038–53075, 2023.
- [26] Maxence Ernout, Rasmus Høier, and Jack Kendall. A cookbook for hardware-friendly implicit learning on static data. In *NeurIPS 2024 Workshop Machine Learning with new Compute Paradigms*, 2024.
- [27] Benjamin Scellier and Yoshua Bengio. Equilibrium propagation: Bridging the gap between energy-based models and backpropagation. *Frontiers in computational neuroscience*, 11:24, 2017.
- [28] Lev Semenovich Pontryagin. *Mathematical theory of optimal processes*. Routledge, 1985.
- [29] T Konstantin Rusch and Daniela Rus. Oscillatory state-space models. *arXiv preprint arXiv:2410.03943*, 2024.
- [30] T Konstantin Rusch and Siddhartha Mishra. Unicornn: A recurrent model for learning very long time dependencies. In *International Conference on Machine Learning*, pages 9168–9178. PMLR, 2021.
- [31] Ricky TQ Chen, Yulia Rubanova, Jesse Bettencourt, and David K Duvenaud. Neural ordinary differential equations. *Advances in neural information processing systems*, 31, 2018.
- [32] Mathieu Blondel and Vincent Roulet. The elements of differentiable programming. *arXiv preprint arXiv:2403.14606*, 2024.
- [33] Habib Ammari, W Wei, and Y Sanghyeon. Numerical methods for ordinary differential equations. *Notes of Course at ETH Zürich*, 2018.

- [34] Benjamin Walker, Andrew D McLeod, Tiexin Qin, Yichuan Cheng, Haoliang Li, and Terry Lyons. Log neural controlled differential equations: The lie brackets make a difference. *arXiv preprint arXiv:2402.18512*, 2024.
- [35] Anthony Bagnall, Hoang Anh Dau, Jason Lines, Michael Flynn, James Large, Aaron Bostrom, Paul Southam, and Eamonn Keogh. The uea multivariate time series classification archive, 2018. *arXiv preprint arXiv:1811.00075*, 2018.
- [36] Attila Reiss, Ina Indlekofer, Philip Schmidt, and Kristof Van Laerhoven. Deep ppg: Large-scale heart rate estimation with convolutional neural networks. *Sensors*, 19(14):3079, 2019.
- [37] Axel Laborieux, Maxence Ernoult, Benjamin Scellier, Yoshua Bengio, Julie Grollier, and Damien Querlioz. Scaling equilibrium propagation to deep convnets by drastically reducing its gradient estimator bias. *Frontiers in neuroscience*, 15:633674, 2021.
- [38] Zhengdao Chen, Jianyu Zhang, Martin Arjovsky, and Léon Bottou. Symplectic recurrent neural networks. *arXiv preprint arXiv:1909.13334*, 2019.
- [39] Samuel Greydanus, Misko Dzamba, and Jason Yosinski. Hamiltonian neural networks. *Advances in neural information processing systems*, 32, 2019.
- [40] Miles Cranmer, Sam Greydanus, Stephan Hoyer, Peter Battaglia, David Spergel, and Shirley Ho. Lagrangian neural networks. *arXiv preprint arXiv:2003.04630*, 2020.
- [41] Peter Toth, Danilo Jimenez Rezende, Andrew Jaegle, Sébastien Racanière, Aleksandar Botev, and Irina Higgins. Hamiltonian generative networks. *arXiv preprint arXiv:1909.13789*, 2019.
- [42] Simon Duane, Anthony D Kennedy, Brian J Pendleton, and Duncan Roweth. Hybrid monte carlo. *Physics letters B*, 195(2):216–222, 1987.
- [43] Radford M Neal et al. Mcmc using hamiltonian dynamics. *Handbook of markov chain monte carlo*, 2(11):2, 2011.
- [44] Nilesh Tripurani, Mark Rowland, Zoubin Ghahramani, and Richard Turner. Magnetic hamiltonian monte carlo. In *International Conference on Machine Learning*, pages 3453–3461. PMLR, 2017.
- [45] Son Nguyen, Lizhang Chen, Bo Liu, and Qiang Liu. Memory-efficient optimization with factorized hamiltonian descent. *arXiv preprint arXiv:2406.09958*, 2024.
- [46] Menachem Stern, Daniel Hexner, Jason W Rocks, and Andrea J Liu. Supervised learning in physical networks: From machine learning to learning machines. *Physical Review X*, 11(2):021045, 2021.
- [47] Axel Laborieux and Friedemann Zenke. Holomorphic equilibrium propagation computes exact gradients through finite size oscillations. *Advances in neural information processing systems*, 35:12950–12963, 2022.
- [48] Benjamin Scellier, Maxence Ernoult, Jack Kendall, and Suhas Kumar. Energy-based learning algorithms for analog computing: a comparative study. *Advances in Neural Information Processing Systems*, 36:52705–52731, 2023.
- [49] Timothy Nest and Maxence Ernoult. Towards training digitally-tied analog blocks via hybrid gradient computation. *Advances in Neural Information Processing Systems*, 37:83877–83914, 2024.
- [50] Benjamin Scellier. A fast algorithm to simulate nonlinear resistive networks. *arXiv preprint arXiv:2402.11674*, 2024.
- [51] Benjamin Scellier, Siddhartha Mishra, Yoshua Bengio, and Yann Ollivier. Agnostic physics-driven deep learning. *arXiv preprint arXiv:2205.15021*, 2022.

- [52] S. Abreu, I. Boikov, M. Goldmann, T. Jonuzi, A. Lupo, S. Masaad, L. Nguyen, E. Picco, G. Pourcel, A. Skalli, L. Talandier, B. Vettelschoss, E. A. Vlieg, A. Argyris, P. Bienstman, D. Brunner, J. Dambre, L. Daudet, J. D. Domenech, I. Fischer, F. Horst, S. Massar, C. R. Mirasso, B. J. Offrein, A. Rossi, M. C. Soriano, S. Sygletos, and S. K. Turitsyn. A photonics perspective on computing with physical substrates. *Reviews in Physics*, 12:100093, December 2024. ISSN 2405-4283. doi: 10.1016/j.revip.2024.100093.
- [53] Michael Schneider, Emily Toomey, Graham Rowlands, Jeff Shainline, Paul Tschirhart, and Ken Segall. SuperMind: A survey of the potential of superconducting electronics for neuromorphic computing. *Superconductor Science and Technology*, 35(5):053001, March 2022. ISSN 0953-2048. doi: 10.1088/1361-6668/ac4cd2.
- [54] J. Grollier, D. Querlioz, K. Y. Camsari, K. Everschor-Sitte, S. Fukami, and M. D. Stiles. Neuromorphic spintronics. *Nature Electronics*, 3(7):360–370, July 2020. ISSN 2520-1131. doi: 10.1038/s41928-019-0360-9.
- [55] Dan Hendrycks and Kevin Gimpel. Gaussian Error Linear Units (GELUs), June 2023.
- [56] Yann N. Dauphin, Angela Fan, Michael Auli, and David Grangier. Language Modeling with Gated Convolutional Networks. In *Proceedings of the 34th International Conference on Machine Learning*, pages 933–941. PMLR, July 2017.
- [57] James Bradbury, Roy Frostig, Peter Hawkins, Matthew James Johnson, Chris Leary, Dougal Maclaurin, George Necula, Adam Paszke, Jake VanderPlas, Skye Wanderman-Milne, and Qiao Zhang. JAX: composable transformations of Python+NumPy programs, 2018. URL <http://github.com/jax-ml/jax>.

## A Supplementary material

### Contents

---

|       |                                                              |    |
|-------|--------------------------------------------------------------|----|
| A.1   | <b>Theoretical results in continuous time</b>                | 15 |
| A.1.1 | Definitions & assumptions                                    | 15 |
| A.1.2 | Proof of the continuous adjoint state method (ASM)           | 16 |
| A.1.3 | Proof of Theorem 3.1                                         | 19 |
| A.1.4 | Connection to Hamiltonian Echo Backpropagation (HEB)         | 21 |
| A.2   | <b>Theoretical results in discrete time</b>                  | 24 |
| A.2.1 | Definitions & assumptions                                    | 24 |
| A.2.2 | Backpropagation Through Time (BPTT)                          | 26 |
| A.2.3 | Proof of Theorem 3.2                                         | 28 |
| A.2.4 | Proof of Theorem 3.3                                         | 32 |
| A.3   | <b>Models and algorithms details</b>                         | 34 |
| A.3.1 | Toy model                                                    | 34 |
| A.3.2 | HSSM architecture                                            | 35 |
| A.3.3 | Linear HRU Block                                             | 36 |
| A.3.4 | Nonlinear HRU                                                | 38 |
| A.3.5 | Differentiating the time-discretization                      | 38 |
| A.3.6 | Echo passes with automatic differentiation                   | 39 |
| A.4   | <b>Experimental details</b>                                  | 41 |
| A.4.1 | Datasets                                                     | 41 |
| A.4.2 | Simulation details and resource consumption                  | 41 |
| A.4.3 | Hyperparameters                                              | 41 |
| A.4.4 | Gradient re-scaling for dealing with numerical instabilities | 42 |

---



## A.1 Theoretical results in continuous time

**Summary.** In this section, we present all the results derived in *continuous* time. More precisely:

- We formally define our model (Def. A.1) and constrained optimization problem (Def. A.2) in *continuous time*.
- We state and prove our algorithm baseline, the *continuous adjoint state method* (ASM) for this constrained optimization problem (Theorem A.1). To ease the comparison of the continuous ASM with *Recurrent Hamiltonian Echo Learning* (RHEL) in continuous time, we state re-parametrized version of the continuous ASM where time is indexed *backwards* (Corollary A.1). Finally, we also state a variant of the continuous ASM when the loss function is only defined at the *final* timestep (Corollary A.2) to ease the comparison between the continuous ASM and *Hamiltonian Echo Backprop* (HEB, [10]).
- We introduce two technical results (Lemma A.1–A.2) which enable us to prove the *time-reversal invariance* property of our model (Lemma A.3). We then show that the direct consequence of this property is the *time-reversibility* of our model upon momentum flipping (Corollary A.3), the key mechanics which fundamentally underpins our algorithm. All these intermediate results allow us to finally introduce RHEL in continuous time and prove its equivalence with the continuous ASM (Theorem A.2).
- Lastly, we connect HEB [10] with the continuous ASM when the loss is defined only at the final time step (Theorem A.4). We highlight some key differences between HEB and RHEL.

### A.1.1 Definitions & assumptions

**Definition A.1** (Continuous Hamiltonian model). *Given  $\theta \in \mathbb{R}^{d_\theta}$ ,  $T \in \mathbb{R}_+^*$  and an input sequence  $t \rightarrow \mathbf{u}(t) \in (\mathbb{R}^{d_u})^{[-T,0]}$ , the continuous Hamiltonian model prediction  $t \rightarrow \Phi(t) \in (\mathbb{R}^{d_\Phi})^{[-T,0]}$  is, by definition, implicitly given as the solution of the following ODE:*

$$\Phi(-T) = \mathbf{x}, \quad \forall t \in [-T, 0] : \partial_t \Phi(t) = \mathbf{J} \cdot \nabla_\Phi H[\Phi(t), \theta, \mathbf{u}(t)], \quad \mathbf{J} := \begin{bmatrix} \mathbf{0} & \mathbf{I} \\ -\mathbf{I} & \mathbf{0} \end{bmatrix}.$$

We assume that:

1.  $H$  is time-reversal invariant:

$$\forall \Phi \in \mathbb{R}^{d_\Phi}, \quad \forall \theta \in \mathbb{R}^{d_\theta}, \quad \forall \mathbf{u} \in \mathbb{R}^{d_u} : \quad H[\Phi, \theta, \mathbf{u}] = H[\Sigma_z \Phi, \theta, \mathbf{u}], \quad \Sigma_z := \begin{bmatrix} \mathbf{I} & \mathbf{0} \\ \mathbf{0} & -\mathbf{I} \end{bmatrix}$$

2.  $\Phi \rightarrow H[\Phi, \cdot, \cdot]$  is twice continuously differentiable,
3.  $\theta \rightarrow H[\cdot, \theta, \cdot]$  is differentiable,
4.  $\nabla_{1,2}^2 H$  exists and is continuous with respect to  $t$ ,
5.  $\mathbf{u} \rightarrow H[\cdot, \cdot, \mathbf{u}]$  is continuous,
6.  $\Phi \rightarrow \nabla_1 H[\Phi, \cdot, \cdot]$  and  $\Phi \rightarrow \nabla_{2,1} H[\Phi, \cdot, \cdot]$  are Lipschitz continuous.

**Remark 1.** While some of these assumptions will be used explicitly in our derivations, they are all needed to guarantee the existence of partial derivatives of  $s$  as an implicit function of  $\mathbf{x}$  and  $\theta$  through Eq. (1) and we refer to [28] for such claims given these assumptions.

**Definition A.2** (Continuous constrained optimization problem). *Given a continuous Hamiltonian model (Def. A.1), we consider the following constrained optimization problem:*

$$\min_{\boldsymbol{\theta}} L := \int_{-T}^0 dt \ell[t, \boldsymbol{\Phi}(t), \boldsymbol{\theta}] \quad \text{s.t.} \quad \forall t \in [-T, 0] : \partial_t \boldsymbol{\Phi}(t) = \mathbf{J} \cdot \nabla_{\boldsymbol{\Phi}} H[\boldsymbol{\Phi}(t), \boldsymbol{\theta}, \mathbf{u}(t)],$$

where we assume that:

1.  $\ell$  is time-reversal invariant:

$$\forall \boldsymbol{\Phi} \in \mathbb{R}^{d_{\boldsymbol{\Phi}}}, \quad \forall \boldsymbol{\theta} \in \mathbb{R}^{d_{\boldsymbol{\theta}}}, \quad \forall t \in [-T, 0] : \quad \ell[t, \boldsymbol{\Phi}, \boldsymbol{\theta}] = \ell[t, \boldsymbol{\Sigma}_z \cdot \boldsymbol{\Phi}, \boldsymbol{\theta}]$$

2.  $t \rightarrow \ell[t, \cdot, \cdot]$  is continuous,
3.  $\boldsymbol{\theta} \rightarrow \ell[\cdot, \cdot, \boldsymbol{\theta}]$  is differentiable,
4.  $t \rightarrow \nabla_{\boldsymbol{\theta}} \ell[t, \cdot, \cdot]$  is continuous.
5.  $\boldsymbol{\Phi} \rightarrow \ell[\cdot, \boldsymbol{\Phi}, \cdot]$  is twice differentiable,

**Remark 2.** Note that while we did not assume in the main part of this manuscript that  $\ell$  depended on  $\boldsymbol{\theta}$ , we assume it in the appendix for the generality of our derivations.

### A.1.2 Proof of the continuous adjoint state method (ASM)

**Theorem A.1** (Continuous adjoint state method ([28])). *Given assumptions A.1–A.2, the gradients of  $L$  with respect to  $\boldsymbol{\theta}$  and  $\mathbf{x}$  are given by:*

$$d_{\boldsymbol{\theta}} L = \int_{-T}^0 g_{\boldsymbol{\theta}}(t) dt, \quad d_{\mathbf{x}} L = \boldsymbol{\lambda}(0)$$

with  $t \rightarrow g_{\boldsymbol{\theta}} \in (\mathbb{R}^{d_{\boldsymbol{\theta}}})^{[-T, 0]}$  defined as:

$$g_{\boldsymbol{\theta}}(t) := \nabla_{\boldsymbol{\theta}} \ell[t, \boldsymbol{\Phi}(t), \boldsymbol{\theta}] + \nabla_{\boldsymbol{\Phi}, \boldsymbol{\theta}}^2 H[\boldsymbol{\Phi}(t), \boldsymbol{\theta}, \mathbf{u}(t)] \cdot \mathbf{J}^{\top} \cdot \boldsymbol{\lambda}(t) \quad \forall t \in [-T, 0]$$

and  $\boldsymbol{\lambda}$  solving for the **adjoint ODE**:

$$\begin{cases} \boldsymbol{\lambda}(0) &= \mathbf{0} \\ \partial_t \boldsymbol{\lambda}(t) &= -\nabla_{\boldsymbol{\Phi}}^2 H[\boldsymbol{\Phi}(t), \boldsymbol{\theta}, \mathbf{u}(t)] \cdot \mathbf{J}^{\top} \cdot \boldsymbol{\lambda}(t) - \nabla_{\boldsymbol{\Phi}} \ell[t, \boldsymbol{\Phi}(t), \boldsymbol{\theta}] \end{cases}$$

*Proof of Theorem A.1.* With slight adaptations, our proof mostly follows that of [32] and also assume the existence of the partial derivatives of  $\boldsymbol{\Phi} = \boldsymbol{\Phi}(t, \mathbf{x}, \boldsymbol{\theta})$  as an *implicit* function of  $t, \mathbf{x}$  and  $\boldsymbol{\theta}$  – we defer to [28] for the proof of this claim.

We start off defining the Lagrangian associated with the constraint optimization problem:

$$\mathcal{L}(\boldsymbol{\Phi}, \boldsymbol{\lambda}, \boldsymbol{\theta}, \mathbf{u}) := \int_{-T}^0 dt \left( \ell[t, \boldsymbol{\Phi}(t, \boldsymbol{\theta}), \boldsymbol{\theta}] + \boldsymbol{\lambda}^{\top}(t) \cdot (\mathbf{J} \cdot \nabla_{\boldsymbol{\Phi}} H[\boldsymbol{\Phi}(t, \boldsymbol{\theta}), \boldsymbol{\theta}, \mathbf{u}(t)] - \partial_t \boldsymbol{\Phi}(t, \boldsymbol{\theta})) \right),$$

where  $t \rightarrow \boldsymbol{\lambda}(t) \in (\mathbb{R}^{d_{\boldsymbol{\Phi}}})^{[-T, 0]}$  denotes the Lagrangian multiplier associated to the constraint.

**Derivation of  $d_{\boldsymbol{\theta}} L$ .** For readability, we emphasize the dependence of  $\boldsymbol{\Phi}$  on  $t$  and  $\boldsymbol{\theta}$ , as we will leverage the existence of its partial derivatives with respect to these variables. Then, given Assumptions A.1, the total derivative of the Lagrangian with respect to  $\boldsymbol{\theta}$  exists and reads:

$$\begin{aligned} d_{\boldsymbol{\theta}} \mathcal{L} &= \int_{-T}^0 dt \left( \partial_{\boldsymbol{\theta}} \boldsymbol{\Phi}(t, \boldsymbol{\theta})^{\top} \cdot \nabla_{\boldsymbol{\Phi}} \ell[t, \boldsymbol{\Phi}(t, \boldsymbol{\theta}), \boldsymbol{\theta}] + \nabla_{\boldsymbol{\theta}} \ell[t, \boldsymbol{\Phi}(t, \boldsymbol{\theta}), \boldsymbol{\theta}] \right) \\ &\quad + \int_{-T}^0 dt \left[ \partial_{\boldsymbol{\theta}} \boldsymbol{\Phi}(t, \boldsymbol{\theta})^{\top} \cdot \nabla_{\boldsymbol{\Phi}}^2 H[\boldsymbol{\Phi}(t, \boldsymbol{\theta}), \boldsymbol{\theta}, \mathbf{u}(t)] \cdot \mathbf{J}^{\top} + \nabla_{\boldsymbol{\theta}, \boldsymbol{\Phi}}^2 H[\boldsymbol{\Phi}(t, \boldsymbol{\theta}), \boldsymbol{\theta}, \mathbf{u}(t)]^{\top} \cdot \mathbf{J}^{\top} - \partial_{\boldsymbol{\theta}, t}^2 \boldsymbol{\Phi}(t, \boldsymbol{\theta})^{\top} \right] \cdot \boldsymbol{\lambda}(t). \end{aligned}$$

We can transform the last term of the integrand with  $\partial_{\theta,t}\Phi(t, \theta)$  by applying Schwartz's theorem and integration by parts as:

$$\begin{aligned} - \int_{-T}^0 dt \partial_{\theta,t}\Phi(t, \theta)^\top \cdot \lambda(t) &= - \int_{-T}^0 dt \partial_{t,\theta}\Phi(t, \theta)^\top \cdot \lambda(t) \\ &= [-\partial_{\theta}\Phi(t, \theta)^\top \cdot \lambda(t)]_{-T}^0 + \int_{-T}^0 dt \partial_{\theta}\Phi(t, \theta)^\top \cdot \partial_t \lambda(t) \\ &= -\partial_{\theta}\Phi(0, \theta)^\top \cdot \lambda(0) + \int_{-T}^0 dt \partial_{\theta}\Phi(t, \theta)^\top \cdot \partial_t \lambda(t) \end{aligned}$$

where the contribution of the first term at  $t = -T$  vanishes because:

$$\Phi(-T, \theta) = x \Rightarrow \partial_{\theta}\Phi(0, \theta) = 0$$

Plugging this back into  $d_{\theta}\mathcal{L}$  yields:

$$\begin{aligned} d_{\theta}\mathcal{L} &= -\partial_{\theta}\Phi(0, \theta)^\top \cdot \lambda(0) \\ &+ \int_{-T}^0 dt \partial_{\theta}\Phi(t, x, \theta)^\top \cdot [\nabla_{\Phi}\ell[t, \Phi(t, \theta), \theta] + \nabla_{\Phi}^2 H[\Phi(t, x, \theta), \theta] \cdot J^\top \cdot \lambda(t) + \partial_t \lambda(t)] \\ &+ \int_{-T}^0 dt (\nabla_{\theta}\ell[t, \Phi(t, \theta), \theta] + \nabla_{\Phi, \theta}^2 H[\Phi(t, x, \theta), \theta] \cdot J^\top \cdot \lambda(t)) \end{aligned}$$

Denoting  $\Phi_*$  the solution of the (primal) ODE and by defining  $\lambda_*$  as the solution of the adjoint ODE:

$$\partial_t \lambda_*(t) = -\nabla_{\Phi}^2 H[\Phi_*(t, \theta), \theta, u(t)] \cdot J^\top \cdot \lambda_*(t) - \nabla_{\Phi}\ell[t, \Phi_*(t, \theta), \theta], \quad \lambda_*(0) = 0$$

we have:

$$\begin{aligned} d_{\theta}L &:= d_{\theta} \int_{-T}^0 dt \ell[t, \Phi(t, \theta), \theta] \\ &= d_{\theta}\mathcal{L}(\Phi_*, \lambda_*, \theta) \\ &= \int_{-T}^0 dt (\nabla_{\theta}\ell[t, \Phi_*(t, \theta), \theta] + \nabla_{\Phi, \theta}^2 H[\Phi_*(t, x, \theta), \theta] \cdot J^\top \cdot \lambda_*(t)) \end{aligned}$$

**Derivation of  $d_x L$ .** Similarly, we can prove that:

$$\begin{aligned} d_x \mathcal{L} &= \partial_x \Phi(0, x)^\top \cdot [\nabla \ell[\Phi(0, x)] - \lambda(0)] \\ &+ \int_{-T}^0 dt \partial_x \Phi(t, x)^\top \cdot [\nabla_{\Phi}\ell[t, \Phi(t, \theta), \theta] + \nabla_{\Phi}^2 H[\Phi(t, x, \theta), \theta, u(t)] \cdot J^\top \cdot \lambda(t) + \partial_t \lambda(t)] \\ &+ \lambda(0) \end{aligned}$$

Using the same  $\Phi_*$  and  $\lambda_*$ , we get  $d_x L = \lambda(0)$ .  $\square$

**Reparametrization of the continuous adjoint method.** To ease the comparison of the continuous adjoint state method with our algorithm, we slightly reparametrize the variables introduced in Theorem A.1.

**Corollary A.1.** *Under the same assumptions as Theorem A.1, the gradients of  $L$  with respect to  $\theta$  and  $x$  are given by:*

$$d_{\theta}L = \int_0^T g_{\theta}(t) dt, \quad d_x L = \lambda(T)$$

with  $t \rightarrow g_{\theta} \in (\mathbb{R}^{d_{\theta}})^{[0, T]}$  defined as:

$$g_{\theta}(t) := \nabla_{\theta}\ell[-t, \Phi(-t), \theta] + \nabla_{\Phi, \theta}^2 H[\Phi(-t), \theta, u(-t)] \cdot J^\top \cdot \lambda(t) \quad \forall t \in [0, T]$$

and  $\lambda$  solving for the **adjoint ODE**:

$$\begin{cases} \lambda(0) &= 0 \\ \partial_t \lambda(t) &= \nabla_{\Phi}^2 H[\Phi(-t), \theta, u(-t)] \cdot J^\top \cdot \lambda(t) + \nabla_{\Phi}\ell[-t, \Phi(-t), \theta] \end{cases}$$

*Proof of Corollary A.1.* Immediately stems from Theorem A.1.  $\square$

**Edge case: loss at the final timestep only.** The backward ODE can be differently parametrized when a loss is defined only at the last time step. We need this formulation for later convenience.

**Corollary A.2.** *Under the same assumptions as Theorem A.1, and assuming:*

$$\ell[t, \cdot, \cdot] = 0 \quad \forall t \in [0, T), \quad \ell[T, \cdot, \cdot] := \ell_T,$$

*the gradients of  $\ell_T$  with respect to  $\theta$  and  $x$  are given by:*

$$d_{\theta} \ell_T[\Phi(0), \theta] = \nabla_{\theta} \ell_T[\Phi(0), \theta] + \int_0^T g_{\theta}(t) dt, \quad d_x L = \lambda(T)$$

*with  $t \rightarrow g_{\theta} \in (\mathbb{R}^{d_{\theta}})^{[0, T]}$  defined as:*

$$g_{\theta}(t) := \nabla_{\Phi, \theta}^2 H[\Phi(-t), \theta, u(-t)] \cdot J^{\top} \cdot \lambda(t) \quad \forall t \in [0, T]$$

*and  $\lambda$  solving for the **adjoint** ODE:*

$$\begin{cases} \lambda(0) &= \nabla_{\Phi} \ell_T[\Phi(0), \theta] \\ \partial_t \lambda(t) &= \nabla_{\Phi}^2 H[\Phi(-t), \theta, u(-t)] \cdot J^{\top} \cdot \lambda(t) \end{cases}$$

*Proof of Corollary A.2.* Starting from the Lagrangian:

$$\mathcal{L}(\Phi, \lambda, \theta, u) := \ell_T[\Phi(0), \theta] + \int_{-T}^0 dt \left( \lambda^{\top}(t) \cdot (J \cdot \nabla_{\Phi} H[\Phi(t), \theta, u(t)] - \partial_t \Phi(t, \theta)) \right),$$

the proof reads in the exact same fashion as that of Theorem A.1.  $\square$

### A.1.3 Proof of Theorem 3.1

**Technical Lemmas.** We first introduce two technical Lemmas which will be needed for the derivation of our main result.

**Lemma A.1** (Block-wise Pauli matrices and associated properties). *Defining  $\Sigma_x, \Sigma_y, \Sigma_z \in \mathbb{R}^{d_\Phi \times d_\Phi}$  as:*

$$\Sigma_x := \begin{bmatrix} \mathbf{0} & I_{d_\Phi/2} \\ I_{d_\Phi/2} & \mathbf{0} \end{bmatrix}, \quad \Sigma_y := \begin{bmatrix} \mathbf{0} & -iI_{d_\Phi/2} \\ iI_{d_\Phi/2} & \mathbf{0} \end{bmatrix}, \quad \Sigma_z := \begin{bmatrix} I_{d_\Phi/2} & \mathbf{0} \\ \mathbf{0} & -I_{d_\Phi/2} \end{bmatrix}$$

where  $i$  denotes the imaginary unit, the following equalities hold:

1.  $\mathbf{J} = i\Sigma_y$
2.  $\Sigma_x^2 = \Sigma_y^2 = \Sigma_z^2 = I_{d_\Phi/2}$ ,
3.  $\Sigma_x \cdot \Sigma_y = i\Sigma_z$ ,  $\Sigma_y \cdot \Sigma_z = i\Sigma_x$ ,  $\Sigma_z \cdot \Sigma_x = i\Sigma_y$ ,
4.  $\Sigma_i \cdot \Sigma_j = -\Sigma_j \cdot \Sigma_i$  for any  $i \neq j \in \{x, y, z\}$ .

*Proof of Lemma A.1.* Because of the block-wise structure of  $\Sigma_x, \Sigma_y, \Sigma_z$ , these equalities can be easily checked.  $\square$

**Lemma A.2.** *Under the assumptions of Def. A.1, the following equalities hold for all  $\Phi, \theta, u$ :*

$$\begin{aligned} \nabla_\Phi H[\Phi, \theta, u] &= \Sigma_z \cdot \nabla_{\Phi^*} H[\Phi^*, \theta, u], \\ \nabla_\Phi^2 H[\Phi, \theta, u] &= \Sigma_z \cdot \nabla_{\Phi^*}^2 H[\Phi^*, \theta, u] \cdot \Sigma_z, \\ \nabla_{\Phi, \theta} H[\Phi, \theta, u] &= \nabla_{\Phi^*, \theta} H[\Phi^*, \theta, u] \cdot \Sigma_z \end{aligned}$$

*Proof of Lemma A.2.* The above equalities can be simply obtained by differentiating through the time-reversal invariance hypothesis – which is possible because of the differentiability of  $H$  with respect to  $\Phi$  and  $\theta$  – and using the chain rule. Namely, given some  $\Phi, \theta, u$ :

$$\begin{aligned} \nabla_\Phi H[\Phi, \theta, u] &= \nabla_\Phi (H[\Phi^*, \theta, u]) \\ &= (\partial_\Phi \Phi^*)^\top \cdot \nabla_{\Phi^*} H[\Phi^*, \theta, u] \\ &= \Sigma_z^\top \cdot \nabla_{\Phi^*} H[\Phi^*, \theta, u] = \Sigma_z \cdot \nabla_{\Phi^*} H[\Phi^*, \theta, u], \end{aligned}$$

since  $\Phi^* := \Sigma_z \cdot \Phi$ . The other equalities are derived in the same way.  $\square$

**Time-reversal invariance.** We highlight here how the assumption  $H[\Phi, \cdot, \cdot] = H[\Sigma_z \cdot \Phi, \cdot, \cdot]$  given inside Def. A.1 entails time-reversal invariance of the dynamics – up to time-reversal the input sequence.

**Lemma A.3** (Time-reversal invariance of the dynamics). *Under the assumptions given in Def. A.1, if  $\Phi$  the solution of the ODE:*

$$\partial_t \Phi(t) = \mathbf{J} \cdot \nabla_\Phi H[\Phi(t), \theta, u(t)],$$

the function  $\tilde{\Phi}^* : t \rightarrow \Sigma_z \cdot \Phi(-t) = [\phi^\top(-t), -\pi^\top(-t)]^\top$  is solution of the same ODE with the time-reversed input sequence  $t \rightarrow \tilde{u}(t) := u(-t)$ .

*Proof.* Let  $t \in [-T, 0]$ . We have:

$$\begin{aligned} \partial_t \tilde{\Phi}^*(t) &= -\Sigma_z \cdot \partial_t \Phi(-t) && \text{(Def. of } \tilde{\Phi}^* \text{ and chain rule)} \\ &= -\Sigma_z \cdot \mathbf{J} \cdot \nabla_\Phi H[\Phi(-t), \theta, u(-t)] && \text{(by assumption)} \\ &= +\mathbf{J} \cdot \Sigma_z \cdot \nabla_\Phi H[\Phi(-t), \theta, u(-t)] && \text{(Lemma A.1)} \\ &= \mathbf{J} \cdot \Sigma_z^2 \cdot \nabla_{\Phi^*} H[\Phi^*(-t), \theta, u(-t)] && \text{(Lemma A.2)} \\ &= \mathbf{J} \cdot \nabla_{\Phi^*} H[\tilde{\Phi}^*(t), \theta, u(-t)] && \text{(Lemma A.1)} \end{aligned}$$

$\square$

**Time reversibility.** A direct consequence of the time-reversal of the dynamics under consideration (Lemma A.3) is *time reversibility*: upon flipping the momentum of  $\Phi$  at time  $t = 0$  ( $\pi(0) \leftarrow -\pi(0)$ ) and presenting the input sequence in reversed order, the system evolves backward to its initial state.

**Corollary A.3** (Reversibility of the dynamics). *Under the same assumptions as Lemma A.3, we define  $\Phi^e$  as the solution of the ODE:*

$$\Phi^e(0) = \Phi^*(0), \quad \partial_t \Phi^e(t) = \mathbf{J} \cdot \nabla_{\Phi^e} H[\Phi^e(t), \boldsymbol{\theta}, \mathbf{u}(-t)] \quad \forall t \in [0, T].$$

Then:

$$\forall t \in [0, T] : \quad \Phi^e(t) = \tilde{\Phi}^*(t)$$

*Proof.*  $\Phi^e$  and  $\tilde{\Phi}^*$  satisfy: i) the same initial conditions ( $\tilde{\Phi}^*(0) = \Phi^*(0) = \Phi^e(0)$ ), ii) the same ODE (Lemma A.3), therefore by unicity of the solution of the ODE, they are equal at all time over the domain of definition of  $\Phi$ .  $\square$

**Main result.** We are now ready to state and demonstrate our main result in continuous time.

**Theorem A.2** (Equivalence between RHEL and the continuous ASM). *Under the assumptions of Def. A.1 and Def. A.2, let  $\Phi$  be the solution of the ODE for  $t \in [-T, 0]$ :*

$$\Phi(-T) = \mathbf{x}, \quad \partial_t \Phi(t) = \mathbf{J} \cdot \nabla_{\Phi} H[\Phi(t), \boldsymbol{\theta}, \mathbf{u}(t)].$$

Given  $\Phi$ , let  $\Phi^e$  be defined as the solution of the other ODE for  $t \in [0, T]$ :

$$\Phi^e(0) = \Phi^*(0), \quad \partial_t \Phi^e(t, \epsilon) = \mathbf{J} \nabla_{\Phi^e} H[\Phi^e(t, \epsilon), \boldsymbol{\theta}, \mathbf{u}(-t)] - \epsilon \mathbf{J} \nabla_{\Phi^e} \ell[-t, \Phi^e(t, \epsilon), \boldsymbol{\theta}].$$

Defining:

$$\Delta_{\boldsymbol{\theta}}^{\text{RHEL}}(t, \epsilon) := \nabla_{\boldsymbol{\theta}} \ell[-t, \Phi^e(t, \epsilon), \boldsymbol{\theta}] + \frac{1}{2\epsilon} (\nabla_{\boldsymbol{\theta}} H[\Phi^e(t, \epsilon), \boldsymbol{\theta}, \mathbf{u}(-t)] - \nabla_{\boldsymbol{\theta}} H[\Phi^e(t, -\epsilon), \boldsymbol{\theta}, \mathbf{u}(-t)]),$$

$$\Delta_{\Phi}^{\text{RHEL}}(t, \epsilon) := \frac{1}{2\epsilon} \Sigma_{\mathbf{x}} \cdot (\Phi^e(t, \epsilon) - \Phi^e(t, -\epsilon)),$$

we have:

$$\forall t \in [0, T], \quad \lambda(t) = \lim_{\epsilon \rightarrow 0} \Delta_{\Phi}^{\text{RHEL}}(t, \epsilon), \quad g_{\boldsymbol{\theta}}(t) = \lim_{\epsilon \rightarrow 0} \Delta_{\boldsymbol{\theta}}^{\text{RHEL}}(t, \epsilon)$$

where  $\lambda$  and  $g_{\boldsymbol{\theta}}$  are defined in Corollary A.1.

*Proof.* Defining  $\Delta_{\Phi}^{\text{RHEL}}(t) := \lim_{\epsilon \rightarrow 0} \Delta_{\Phi}^{\text{RHEL}}(t, \epsilon)$  and  $\Delta_{\boldsymbol{\theta}}^{\text{RHEL}}(t) := \lim_{\epsilon \rightarrow 0} \Delta_{\boldsymbol{\theta}}^{\text{RHEL}}(t, \epsilon)$ , note that:

$$\begin{aligned} \Delta_{\Phi}^{\text{RHEL}}(t) &= \Sigma_{\mathbf{x}} \cdot \partial_{\epsilon} \Phi^e(t, \epsilon)|_{\epsilon=0} \\ \Delta_{\boldsymbol{\theta}}^{\text{RHEL}}(t) &= \nabla_{\boldsymbol{\theta}} \ell[-t, \Phi^e(t, 0), \boldsymbol{\theta}] + \partial_{\epsilon} (\nabla_{\boldsymbol{\theta}} H[\Phi^e(t, \epsilon), \boldsymbol{\theta}, \mathbf{u}(-t)])|_{\epsilon=0} \end{aligned}$$

**Derivation of  $\lambda(t) = \lim_{\epsilon \rightarrow 0} \Delta_{\Phi}^{\text{RHEL}}(t, \epsilon)$ .** Given  $t \in [0, T]$ , differentiating the ODE satisfied by  $\Phi^e$  with respect to  $\epsilon$  at  $\epsilon = 0$  yields:

$$\begin{aligned} \partial_t (\partial_{\epsilon} \Phi^e(t, \epsilon)|_{\epsilon=0}) &= \partial_{\epsilon} (\partial_t \Phi^e(t, \epsilon))|_{\epsilon=0} && \text{(Schwartz Theorem)} \\ &= \partial_{\epsilon} (\mathbf{J} \cdot \nabla_{\Phi^e} H[\Phi^e(t, \epsilon), \boldsymbol{\theta}, \mathbf{u}(-t)] - \epsilon \mathbf{J} \nabla_{\Phi^e} \ell[t, \Phi^e(t, \epsilon), \boldsymbol{\theta}])|_{\epsilon=0} \\ &= \mathbf{J} \cdot \nabla_{\Phi^e}^2 H[\Phi^e(t, 0)] \cdot \partial_{\epsilon} \Phi^e(t, \epsilon)|_{\epsilon=0} - \mathbf{J} \nabla_{\Phi^e} \ell[t, \Phi^e(t, 0), \boldsymbol{\theta}]. \end{aligned}$$

By Lemma A.3:

$$\Phi^e(t, 0) = \tilde{\Phi}^*(t) \quad \forall t \in [0, T],$$

therefore:

$$\begin{aligned} \partial_t (\partial_{\epsilon} \Phi^e(t, \epsilon)|_{\epsilon=0}) &= \mathbf{J} \cdot \nabla_{\Phi^*}^2 H[\Phi^*(-t)] \cdot \partial_{\epsilon} \Phi^e(t, \epsilon)|_{\epsilon=0} - \mathbf{J} \nabla_{\Phi^*} \ell[t, \Phi^*(-t), \boldsymbol{\theta}] \\ &= \mathbf{J} \cdot \Sigma_z \cdot \nabla_{\Phi}^2 H[\Phi(-t, 0)] \cdot \Sigma_z \cdot \partial_{\epsilon} \Phi^e(t, \epsilon)|_{\epsilon=0} - \mathbf{J} \cdot \Sigma_z \cdot \nabla_{\Phi} \ell[t, \Phi(-t), \boldsymbol{\theta}] \end{aligned} \quad \text{(Lemma A.2)}$$

Additionally, note that we have by Lemma A.1:

$$\mathbf{J} \cdot \Sigma_z = -\Sigma_x, \quad \Sigma_z = \mathbf{J} \cdot \Sigma_x,$$

so that:

$$\partial_t (\partial_\epsilon \Phi^e(t, \epsilon)|_{\epsilon=0}) = -\Sigma_x \cdot \nabla_{\Phi}^2 H[\Phi(-t, 0)] \cdot (\mathbf{J} \cdot \Sigma_x) \cdot \partial_\epsilon \Phi^e(t, \epsilon)|_{\epsilon=0} + \Sigma_x \cdot \nabla_{\Phi} \ell[t, \Phi(-t), \theta] \quad (19)$$

Left multiplying Eq. (19) on both sides by  $\Sigma_x$  yields:

$$\begin{aligned} \partial_t (\Sigma_x \cdot \partial_\epsilon \Phi^e(t, \epsilon)|_{\epsilon=0}) &= -\Sigma_x^2 \cdot \nabla_{\Phi}^2 H[\Phi(-t, 0)] \cdot \mathbf{J} \cdot (\Sigma_x \cdot \partial_\epsilon \Phi^e(t, \epsilon)|_{\epsilon=0}) + \Sigma_x^2 \cdot \nabla_{\Phi} \ell[t, \Phi(-t), \theta] \\ &= -\nabla_{\Phi}^2 H[\Phi(-t, 0)] \cdot \mathbf{J} \cdot (\Sigma_x \cdot \partial_\epsilon \Phi^e(t, \epsilon)|_{\epsilon=0}) + \nabla_{\Phi} \ell[t, \Phi(-t), \theta] \quad (\text{Lemma A.1}) \\ &= \nabla_{\Phi}^2 H[\Phi(-t, 0)] \cdot \mathbf{J}^\top \cdot (\Sigma_x \cdot \partial_\epsilon \Phi^e(t, \epsilon)|_{\epsilon=0}) + \nabla_{\Phi} \ell[t, \Phi(-t), \theta] \quad (\mathbf{J}^\top = -\mathbf{J}) \end{aligned}$$

Finally, note that because  $\Phi^e(0) = \Phi^*$  does not depend on  $\epsilon$ , we have that:

$$\partial_t (\Sigma_x \cdot \partial_\epsilon \Phi^e(0, \epsilon)|_{\epsilon=0}) = 0,$$

so that all in all,  $\Delta_{\Phi}^{\text{RHEL}}$  satisfies:

$$\begin{cases} \Delta_{\Phi}^{\text{RHEL}}(0) &= \mathbf{0} \\ \partial_t \Delta_{\Phi}^{\text{RHEL}}(t) &= \nabla_{\Phi}^2 H[\Phi(-t), \theta, \mathbf{u}(-t)] \cdot \mathbf{J}^\top \cdot \Delta_{\Phi}^{\text{RHEL}}(t) + \nabla_{\Phi} \ell[-t, \Phi(-t), \theta] \end{cases}$$

Therefore  $\Delta_{\Phi}^{\text{RHEL}}$  and  $\lambda$  (as defined in Corollary A.1) satisfy the same initial conditions and the same ODE, therefore they are equal at all times.

**Derivation of  $g_\theta(t) = \lim_{\epsilon \rightarrow 0} \Delta_{\theta}^{\text{RHEL}}(t, \epsilon)$ .** Note that by Lemma A.3 and time-reversal invariance of  $\ell$ :

$$\begin{aligned} \Delta_{\theta}^{\text{RHEL}}(t) &= \nabla_{\theta} \ell[-t, \Phi^*(-t, 0), \theta] + \partial_\epsilon (\nabla_{\theta} H[\Phi^e(t, \epsilon), \theta, \mathbf{u}(-t)])|_{\epsilon=0} \\ &= \nabla_{\theta} \ell[-t, \Phi(-t, 0), \theta] + \partial_\epsilon (\nabla_{\theta} H[\Phi^e(t, \epsilon), \theta, \mathbf{u}(-t)])|_{\epsilon=0} \end{aligned}$$

As the first term of  $\Delta_{\theta}^{\text{RHEL}}(t)$  and  $g_\theta(t)$  coincide, the remainder of the derivation focuses on the second term of  $\Delta_{\theta}^{\text{RHEL}}(t)$ . Given  $t \in [0, T]$ , we have:

$$\begin{aligned} \nabla_{\Phi, \theta}^2 H[\Phi(-t), \theta, \mathbf{u}(t)] \cdot \mathbf{J}^\top \cdot \lambda(t) &= \nabla_{\Phi^e, \theta}^2 H[\Phi^e(t, 0), \theta, \mathbf{u}(t)] \cdot \Sigma_z \cdot \mathbf{J}^\top \cdot \lambda(t) \quad (\text{Lemma A.2}) \\ &= -\nabla_{\Phi^e, \theta}^2 H[\Phi^e(t, 0), \theta, \mathbf{u}(t)] \cdot \Sigma_z \cdot i\Sigma_y \cdot \lambda(t) \quad (\mathbf{J} = i\Sigma_y) \\ &= +\nabla_{\Phi^e, \theta}^2 H[\Phi^e(t, 0), \theta, \mathbf{u}(t)] \cdot i\Sigma_y \cdot \Sigma_z \cdot \lambda(t) \quad (\text{Lemma A.1}) \\ &= -\nabla_{\Phi^e, \theta}^2 H[\Phi^e(t, 0), \theta, \mathbf{u}(t)] \cdot \Sigma_x \cdot \lambda(t) \quad (\text{Lemma A.1}) \\ &= -\nabla_{\Phi^e, \theta}^2 H[\Phi^e(t, 0), \theta, \mathbf{u}(t)] \cdot \Sigma_x^2 \cdot \partial_\epsilon \Phi^e(t, \epsilon)|_{\epsilon=0} \quad (\lambda = \Delta_{\Phi}^{\text{RHEL}}) \\ &= -\nabla_{\Phi^e, \theta}^2 H[\Phi^e(t, 0), \theta, \mathbf{u}(t)] \cdot \partial_\epsilon \Phi^e(t, \epsilon)|_{\epsilon=0} \quad (\text{Lemma A.1}) \\ &= -\partial_\epsilon (\nabla_{\theta} H[\Phi^e(t, \epsilon), \theta, \mathbf{u}(t)])|_{\epsilon=0}, \end{aligned}$$

which finishes to prove  $g_\theta(t) = \Delta_{\theta}^{\text{RHEL}}(t)$  for  $t \in [0, T]$ .  $\square$

#### A.1.4 Connection to Hamiltonian Echo Backpropagation (HEB)

**Remark 3.** The above setup and implementation of RHEL is not exactly that of Hamiltonian Echo Backprop (HEB, [10]). In particular:

- the loss function in HEB is only defined at the final time step,
- the interaction with  $\ell$  does not happen simultaneously with  $H$ ,
- finally, we would like to recover the HEB formula giving the gradient estimate of the loss with respect to the initial state of the neurons.

In the following corollary, we make slight algorithmic adjustments to match the seminal HEB implementation as much as possible **while preserving the generality of the sequence modelling setting**, i.e. dependence of  $H$  with  $\theta$  and  $\mathbf{u}$ . Note that in the seminal HEB work,  $H$  does not depend on a static set of parameters  $\theta$  nor on an input sequence.  $\mathbf{u}$ .

**Corollary A.4.** Under the assumptions of Def. A.1 and Def. A.2, and assuming additionally:

$$\ell[t, \cdot, \cdot] = 0 \quad \forall t \in [0, T], \quad \ell[T, \cdot, \cdot] := \ell_T,$$

let  $\Phi$  be the solution of the ODE, for  $t \in [-T, 0]$ :

$$\Phi(-T) = \mathbf{x}, \quad \partial_t \Phi(t) = \mathbf{J} \cdot \nabla_{\Phi} H[\Phi(t), \theta, \mathbf{u}(t)],$$

and the solution of another ODE, for  $t \in [0, \epsilon]$ :

$$\partial_t \Phi(t) = \mathbf{J} \nabla_{\Phi} \ell_T[\Phi(t), \theta].$$

Let  $\Phi^e$  be the solution of the following ODE, for  $t \in [\epsilon, T]$ :

$$\Phi^e(0, \epsilon) = \Phi(\epsilon)^*, \quad \partial_t \Phi^e(t, \epsilon) = \mathbf{J} \nabla_{\Phi^e} H[\Phi^e(t, \epsilon), \theta, \mathbf{u}(-t)],$$

Defining:

$$\begin{aligned} \Delta_{\theta}^{\text{RHEL}}(t, \epsilon) &:= \frac{1}{2\epsilon} (\nabla_{\theta} H[\Phi^e(t, \epsilon), \theta, \mathbf{u}(-t)] - \nabla_{\theta} H[\Phi^e(t, -\epsilon), \theta, \mathbf{u}(-t)]), \\ \Delta_{\Phi}^{\text{RHEL}}(t, \epsilon) &:= \frac{1}{2\epsilon} \Sigma_x \cdot (\Phi^e(t, \epsilon) - \Phi^e(t, -\epsilon)), \end{aligned}$$

we have:

$$\forall t \in [0, T], \quad \lambda(t) = \lim_{\epsilon \rightarrow 0} \Delta_{\Phi}^{\text{RHEL}}(t, \epsilon), \quad g_{\theta}(t) = \lim_{\epsilon \rightarrow 0} \Delta_{\theta}^{\text{RHEL}}(t, \epsilon)$$

where  $\lambda$  and  $g_{\theta}$  are defined in Corollary A.2. In particular:

$$-i\epsilon d_{\mathbf{x}^*}^w \ell_T[\Phi(0), \theta] = \Phi^e(T, \epsilon)^* - \Phi(-T) + \mathcal{O}(\epsilon^2)$$

where  $d_{\mathbf{x}^*}^w \equiv d_{\Phi}$  denotes the total Wirtinger derivative with respect to  $\mathbf{x}^*$  and  $i$  the imaginary unit.

*Proof of Corollary A.4.* The derivation is almost exactly similar to that of Theorem 3.1 with two key differences:

- the version of the continuous ASM against which this version of RHEL is compared is different (Corollary A.2),
- the interaction of  $\Phi$  with  $\ell$  and  $H$  do not happen simultaneously but on disjoint intervals.

We will simply show that the interaction with  $\ell$  and conjugation  $\Phi \rightarrow \Phi^*$  yields the correct initial conditions and defer to the proof of Theorem 3.1 for the remainder. We will also use the same notations and denote  $\Delta_{\Phi}^{\text{RHEL}}(t) := \lim_{\epsilon \rightarrow 0} \Delta_{\Phi}^{\text{RHEL}}(t, \epsilon)$ ,  $\Delta_{\theta}^{\text{RHEL}}(t) := \lim_{\epsilon \rightarrow 0} \Delta_{\theta}^{\text{RHEL}}(t, \epsilon)$ .

**Derivation of  $\lambda(t) = \lim_{\epsilon \rightarrow 0} \Delta_{\Phi}^{\text{RHEL}}(t, \epsilon)$ .** Integrating the ODE satisfied by  $\Phi$  between 0 and  $T$  yields:

$$\begin{aligned} \Phi(\epsilon) &= \Phi(0) + \int_0^{\epsilon} dt \mathbf{J} \cdot \nabla_{\Phi} \ell[\Phi(t), \theta] \\ &= \Phi(0) + \epsilon \mathbf{J} \cdot \nabla_{\Phi} \ell[\Phi(0), \theta] + \mathcal{O}(\epsilon^2) \end{aligned}$$

Therefore, the initial state of  $\Phi^e$  can be written as:

$$\begin{aligned} \Phi^e(\epsilon) &= \Phi^*(0) + \epsilon \Sigma_z \cdot \mathbf{J} \cdot \nabla_{\Phi} \ell[\Phi(0), \theta] + \mathcal{O}(\epsilon^2) \\ &= \Phi^*(0) + \epsilon \Sigma_x \cdot \nabla_{\Phi} \ell[\Phi(0), \theta] + \mathcal{O}(\epsilon^2) \end{aligned} \quad (\text{Lemma A.1}).$$

By differentiating the last equality with respect to  $\epsilon$  at  $\epsilon = 0$ , we obtain:

$$\Delta_{\Phi}^{\text{RHEL}}(0) = \nabla_{\Phi} \ell[\Phi(0), \theta].$$



Proceeding exactly as in the proof of Theorem A.2, we obtain that:

$$\begin{cases} \Delta_{\Phi}^{\text{RHEL}}(0) &= \nabla_{\Phi} \ell[\Phi(0), \theta], \\ \partial_t \Delta_{\Phi}^{\text{RHEL}}(t) &= \nabla_{\Phi}^2 H[\Phi(-t), \theta, \mathbf{u}(-t)] \cdot \mathbf{J}^{\top} \cdot \Delta_{\Phi}^{\text{RHEL}}(t) \end{cases}$$

Therefore  $\Delta_{\Phi}^{\text{RHEL}}$  and  $\lambda$  (as defined in Corollary A.2) satisfy the same initial conditions and the same ODE, therefore they are equal at all times.

**Derivation of  $g_{\theta}(t) = \lim_{\epsilon \rightarrow 0} \Delta_{\theta}^{\text{RHEL}}(t, \epsilon)$ .** See proof of Theorem A.2.

**Connection to HEB formula.** In particular, we have:

$$\begin{aligned} d_{\mathbf{x}} \ell_T[\Phi(0), \theta] &= \lambda(T) = \Sigma_x \cdot \partial_{\epsilon} (\Phi(T, \epsilon))|_{\epsilon=0} \\ &= \frac{1}{\epsilon} \Sigma_x (\Phi^e(T, \epsilon) - \Phi^e(T, 0)) + \mathcal{O}(\epsilon) \\ &= \frac{1}{\epsilon} \Sigma_x (\Phi^e(T, \epsilon) - \Phi^*(T)) + \mathcal{O}(\epsilon) && \text{(Lemma A.3)} \\ &= -\frac{i}{\epsilon} \Sigma_y \cdot \Sigma_z \cdot (\Phi^e(T, \epsilon) - \Phi^*(T)) + \mathcal{O}(\epsilon) && \text{(Lemma A.1)} \\ &= -\frac{i}{\epsilon} \Sigma_y \cdot (\Phi^e(T, \epsilon)^* - \Phi^*(T)) + \mathcal{O}(\epsilon) \end{aligned}$$

Left multiplying on both sides by  $i\epsilon \Sigma_y$  and noticing that  $id_{\Phi^*}^w \equiv -i\Sigma_y \cdot d_{\Phi}$ , we finally obtain:

$$-i\epsilon d_{\Phi^*}^w \ell_T[\Phi(0), \theta] = \Phi^e(T, \epsilon)^* - \Phi^*(T) + \mathcal{O}(\epsilon^2)$$

□

## A.2 Theoretical results in discrete time

**Summary.** In this section, we introduce all the results derived in *discrete* time. More precisely:

- We first formally define *Hamiltonian Recurrent Units* (HRUs, Definition A.3). HRUs can be regarded as the discrete-time counterpart of the continuous model introduced in the previous section (Definition A.1), namely as an *explicit* and *symplectic* integrator of the continuous Hamiltonian model which preserves the time-reversal invariance and time-reversibility properties in discrete time. We also introduce the constrained optimization problem naturally associated with HRUs (Definition A.4), which is the discrete time counterpart of the constrained continuous optimization problem introduced in the previous section (Definition A.2).
- We then formally define *Hamiltonian State Space Models* (HSSMs) as stacks of HRUs (Definition A.5) and the *multilevel* constrained optimization problem which is naturally associated to these models (Definition A.6).
- We state and prove our algorithmic baseline, *Backpropagation Through Time* (BPTT), through the lens of the *Lagrangian* formalism to establish a clear connection with the continuous ASM. We first introduce and derive BPTT in its general form (Theorem A.3) and then apply it more specifically to a HRU as defined in Definition A.3 (Corollary A.5).
- As we did in continuous time, we introduce a series of technical Lemmas needed to extend RHEL in discrete time. We first demonstrate the time-reversibility of HRUs on a single time step (Lemma A.4), which then enables us to extend the time reversibility property derived in continuous time (Corollary A.3) to *discrete* time (Corollary A.6). After introducing one last technical result (Lemma A.5), we then state and prove RHEL in discrete time when applied to HRUs (Corollary A.7). As the algorithm prescribed by Corollary A.7 includes solving an *implicit equation*, we finally introduce a slight practical (*i.e.* fully explicit) variant of RHEL in discrete time (Corollary A.8).
- Lastly, we show how to estimate gradients end-to-end in HSSMs by using *RHEL-chaining* (Theorem A.4). We also highlight that in practice, when using feedforward transformations across HRUs, the algorithm prescribed by Theorem A.4 implicitly requires to chain RHEL through HRUs and *automatic differentiation* through these feedforward transformations (Remark 8). This remark fundamentally underpins the actual algorithmic implementation of RHEL which was used throughout our experiments.

### A.2.1 Definitions & assumptions

**Definition A.3** (Hamiltonian Recurrent Unit). *Given  $\theta \in \mathbb{R}^{d_\theta}$ ,  $K \in \mathbb{N}^*$  and an input sequence  $(\mathbf{u}_k)_{k \in [-K, 0]} \in (\mathbb{R}^{d_u})^K$ , the Hamiltonian Recurrent Unit (HRU) prediction is given by:*

$$\Phi_{k+1} = \mathcal{M}_{H,\delta}[\Phi_k, \theta, \mathbf{u}_k] \quad \forall k = -K \cdots -1,$$

with  $H := T + V$  and:

$$\mathcal{M}_{H,\delta} := \mathcal{M}_{T,\delta/2} \circ \mathcal{M}_{V,\delta} \circ \mathcal{M}_{T,\delta/2}, \quad \mathcal{M}_{T,\delta} := \Phi + \delta \mathbf{J} \cdot \nabla_\Phi T, \quad \mathcal{M}_{V,\delta} := \Phi + \delta \mathbf{J} \cdot \nabla_\Phi V$$

We assume that:

1.  $H$  is separable, *i.e.*  $V$  and  $T$  only depend on  $\phi$  and  $\pi$  respectively:

$$V[\Phi, \theta, \mathbf{u}] = V[\phi, \theta, \mathbf{u}], \quad T[\Phi, \theta, \mathbf{u}] = T[\pi, \theta, \mathbf{u}]$$

2.  $T$  and  $V$  are time-reversal invariant:

$$\forall \Phi \in \mathbb{R}^{d_\Phi}, \forall \theta \in \mathbb{R}^{d_\theta}, \forall \mathbf{u} \in \mathbb{R}^{d_u} : \quad \begin{cases} T[\Phi, \theta, \mathbf{u}] = T[\Sigma_z \cdot \Phi, \theta, \mathbf{u}], \\ V[\Phi, \theta, \mathbf{u}] = V[\Sigma_z \cdot \Phi, \theta, \mathbf{u}] \end{cases}$$

3.  $T$  and  $V$  are twice differentiable with respect to  $\Phi$ ,  $\theta$  and  $\mathbf{u}$ .

**Remark 4.** Note that  $\mathcal{M}_{H,\delta}$  is simply a Leapfrog integrator associated with  $H$ . We justify each of our design choices below:

- **3 steps-parametrization.** We write the Leapfrog integrator in a three-steps fashion to yield a reversible integrator.
- **Separability of the Hamiltonian.** In the case where  $\phi$  and  $\phi$  can be separated out in the Hamiltonian function, the Leapfrog integrator becomes explicit [33].
- **$T$  and  $V$  as functions of  $\Phi$ .** Although  $T$  and  $V$  only depend on  $\pi$  and  $\phi$  respectively, we choose to define them as functions of  $\Phi$  so that the proof of RHEL in the continuous case seamlessly translates to the discrete case.

**Definition A.4** (Constrained optimization problem in discrete time). *Given a continuous Hamiltonian model (Def. A.1), we consider the following constrained optimization problem:*

$$\min_{\theta} L := \sum_{k=-K+1}^0 \ell[\Phi_k, k] \quad \text{s.t.} \quad \Phi_{k+1} = \mathcal{M}_{H,\delta}[\Phi_k, \theta, \mathbf{u}_k] \quad \forall k = -K \cdots -1$$

where we assume that:

1.  $\ell$  is time-reversal invariant:

$$\forall \Phi \in \mathbb{R}^{d_\Phi}, \forall \theta \in \mathbb{R}^{d_\theta}, \forall k = -K, \dots, 0: \quad \ell_k[\Phi, \theta] = \ell_k[\Sigma_z \cdot \Phi, \theta]$$

2.  $\ell$  is twice differentiable with respect to  $\Phi$  and  $\theta$ .

**Definition A.5** (Hamiltonian State Space Models). *Given  $(\theta^{(1)}, \dots, \theta^{(N)}) \in (\mathbb{R}^{d_\theta})^N$ ,  $K \in \mathbb{N}^*$  and an input sequence  $(\mathbf{u}_k)_{k \in [-K, 0]} \in (\mathbb{R}^{d_u})^K$ , a Hamiltonian State Space Model (HSSM) is defined as the composition of HRUs defined in Def. A.3 as:*

$$\Phi^{(0)} := \bar{\mathbf{u}}, \quad \forall \ell \in \llbracket 0, N-1 \rrbracket, \quad \forall k \in \llbracket -K, 0 \rrbracket: \quad \Phi_{k+1}^{(\ell+1)} = \mathcal{M}_{H^{(\ell)}, \delta}^{(\ell)}[\Phi_k^{(\ell+1)}, \theta^{(\ell)}, \Phi_k^{(\ell)}],$$

or in a vectorized fashion as:

$$\Phi^{(0)} := \bar{\mathbf{u}}, \quad \forall \ell \in \llbracket 0, N-1 \rrbracket: \quad \bar{\Phi}^{(\ell+1)} = \mathcal{M}_{H^{(\ell)}, \delta}^{(\ell)}[\bar{\Phi}^{(\ell+1)}, \theta^{(\ell)}, \bar{\Phi}^{(\ell)}],$$

**Definition A.6** (Multilevel optimization problem in discrete time). *Given a HSSM (Def. A.5), we consider the following constrained optimization problem:*

$$\min_{\theta} L := \sum_{k=-K+1}^0 \ell[\Phi_k, k] \quad \text{s.t.} \quad \Phi^{(0)} := \bar{\mathbf{u}},$$

$$\forall \ell \in \llbracket 0, N-1 \rrbracket: \quad \bar{\Phi}^{(\ell+1)} = \mathcal{M}_{H^{(\ell)}, \delta}^{(\ell)}[\bar{\Phi}^{(\ell+1)}, \theta^{(\ell)}, \bar{\Phi}^{(\ell)}]$$

where we assume that  $\ell$  satisfies the same assumptions as in Def. A.4.

## A.2.2 Backpropagation Through Time (BPTT)

**General form.** We first state and prove Backpropagation Through Time (BPTT) for any integrator  $\mathcal{M}_{H,\delta}$ .

**Theorem A.3** (Backpropagation Through Time (BPTT)). *Given assumptions in Def. A.3–A.4, the gradients of the loss with respect to the parameters  $\theta$  and the inputs  $\mathbf{u}_{-k}$  are given by:*

$$d_{\theta}L = \sum_{k=0}^{K-1} g_{\theta}(k), \quad d_{\mathbf{u}_{-(k+1)}}L = g_{\mathbf{u}}(k) \quad \forall k \in \llbracket 0, K-1 \rrbracket,$$

with:

$$\begin{cases} g_{\theta}(k) &= \nabla_2 \ell[\Phi_{-k}, \theta, -k] + \partial_2 \mathcal{M}_{H,\delta}[\Phi_{-(k+1)}, \theta, \mathbf{u}_{-(k+1)}]^\top \cdot \lambda_k \\ g_{\mathbf{u}}(k) &= \partial_3 \mathcal{M}_{H,\delta}[\Phi_{-(k+1)}, \theta, \mathbf{u}_{-(k+1)}]^\top \cdot \lambda_k, \end{cases}$$

and where  $(\lambda_k)$  satisfy the following recursion relationship:

$$\begin{cases} \lambda_0 = \nabla_1 \ell[\Phi_0, 0], \\ \lambda_{k+1} = \partial_1 \mathcal{M}_{H,\delta}(\Phi_{-(k+1)}, \theta, \mathbf{u}_{-(k+1)})^\top \cdot \lambda_k + \nabla_1 \ell[\Phi_{-(k+1)}, -(k+1)] \quad \forall k = 0, \dots, K-1 \end{cases}$$

*Proof of Theorem A.3.* BPTT can classically be derived through the application of the “chain rule” backward through the inference computational graph defined in Def. A.3. Another useful viewpoint though, which directly connects BPTT as the discrete counterpart of the continuous ASM and will be useful later in the appendix, is to derive it through *the method of Lagrangian multipliers*. Namely, the Lagrangian associated to the constrained optimization problem in Def. A.4 reads as:

$$\mathcal{L}(\Phi, \lambda, \theta, \mathbf{u}) = \sum_{k=0}^{K-1} \ell[\Phi_{-k}, \theta, -k] + \lambda_k^\top \cdot (\mathcal{M}_{H,\delta}[\Phi_{-(k+1)}, \theta, \mathbf{u}_{-(k+1)}] - \Phi_{-k})$$

Extremizing  $\mathcal{L}$  with respect to  $\Phi$  and  $\lambda$  yield  $\Phi_{k,*}$  and  $\lambda_{k,*}$ :

$$\forall k = 0, \dots, K-1 : \quad \partial_{\lambda_k} \mathcal{L}(\Phi_*, \lambda_*, \theta, \mathbf{u}) = \mathcal{M}_{H,\delta}[\Phi_{-(k+1),*}, \theta, \mathbf{u}_{-(k+1)}] - \Phi_{-k,*} = 0,$$

$$\partial_{\Phi_0} \mathcal{L}(\Phi_*, \lambda_*, \theta, \mathbf{u}) = \nabla_1 \ell[\Phi_{0,*}, \theta, 0] - \lambda_{0,*} = 0$$

$$\forall k = 1, \dots, K-1 : \quad \partial_{\Phi_{-k}} \mathcal{L}(\Phi_*, \lambda_*, \theta, \mathbf{u}) = \nabla_1 \ell[\Phi_{-k}, \theta, -k] + \partial_1 \mathcal{M}_{H,\delta}[\Phi_{-k}, \theta, \mathbf{u}_{-k}]^\top \cdot \lambda_{k-1} - \lambda_k = 0,$$

Finally, the total derivative of  $L$  with respect to  $\theta$  reads as:

$$\begin{aligned} d_{\theta}L &= d_{\theta} \mathcal{L}(\Phi_*, \lambda_*, \theta, \mathbf{u}) \\ &= \partial_{\theta} \mathcal{L}(\Phi_*, \lambda_*, \theta, \mathbf{u}) + \underbrace{\partial_{\theta} \Phi_*^\top \cdot \partial_{\Phi} \mathcal{L}(\Phi_*, \lambda_*, \theta, \mathbf{u})}_{=0} + \underbrace{\partial_{\theta} \lambda_*^\top \cdot \partial_{\lambda} \mathcal{L}(\Phi_*, \lambda_*, \theta, \mathbf{u})}_{=0} \\ &= \sum_{k=0}^{K-1} \nabla_2 \ell[\Phi_{-k}, \theta, -k] + \partial_2 \mathcal{M}_{H,\delta}[\Phi_{-(k+1)}, \theta, \mathbf{u}_{-(k+1)}]^\top \cdot \lambda_k \end{aligned}$$

The total derivative of  $L$  with respect to  $\mathbf{u}_{-k}$  is derived in the exact same fashion.  $\square$

**Remark 5.** Note that using the vectorized notations introduced in subsection 3.3, the Lagrangian of the constrained optimization problem defined in Def. A.4 re-writes:

$$\mathcal{L} = \mathbf{1}^\top \cdot \ell[\tilde{\Phi}, \theta] + \text{Tr} \left[ \left( \mathcal{M}_{H,\delta}[\tilde{\Phi}, \theta, \tilde{\mathbf{u}}] - \tilde{\Phi} \right) \cdot \tilde{\lambda}^\top \right]$$

with  $\text{Tr}$  denoting the trace matrix operator and:

$$\begin{aligned} \mathbf{1} &:= \begin{bmatrix} 1 \\ \vdots \\ 1 \end{bmatrix} \in \mathbb{R}^{K \times 1}, \quad \ell[\tilde{\Phi}, \theta] := \begin{bmatrix} \ell[\Phi_0, \theta, 0] \\ \vdots \\ \ell[\Phi_{-(K-1)}, \theta, -(K-1)] \end{bmatrix} \in \mathbb{R}^{K \times 1}, \\ \mathcal{M}_{H,\delta}[\tilde{\Phi}, \theta, \tilde{\mathbf{u}}] &:= \begin{bmatrix} \mathcal{M}_{H,\delta}[\Phi_0, \theta, \mathbf{u}] \\ \vdots \\ \mathcal{M}_{H,\delta}[\Phi_{-(K-1)}, \theta, \mathbf{u}] \end{bmatrix} \in \mathbb{R}^{K \times d_{\Phi}} \end{aligned}$$

**Detailed BPTT.** For the needs of our derivation of RHEL in discrete time, we now introduce a finer-grained version of BPTT given model assumptions given in Def. A.3.

**Corollary A.5** (Detailed BPTT). *Given assumptions in Def. A.3–A.4, the gradients of the loss with respect to the parameters  $\theta$  and the inputs  $\mathbf{u}_{-k}$  are given by:*

$$d_{\theta}L = \sum_{k=0}^{K-1} g_{\theta}(k), \quad d_{\mathbf{u}_{-k}}L = g_{\mathbf{u}}(k),$$

with:

$$\begin{aligned} g_{\theta}(k) &:= \nabla_2 \ell[\Phi_{-k}, \theta, -k] + \frac{\delta}{2} \nabla_{1,2}^2 T[\Phi_{-(k+1/3)}, \theta, \mathbf{u}_{-(k+1)}] \cdot \mathbf{J}^{\top} \cdot \lambda_k \\ &+ \delta \nabla_{1,2}^2 V[\Phi_{-(k+2/3)}, \theta, \mathbf{u}_{-(k+1)}] \cdot \mathbf{J}^{\top} \cdot \lambda_{k+1/3} + \frac{\delta}{2} \nabla_{1,2}^2 T[\Phi_{-(k+1)}, \theta, \mathbf{u}_{-(k+1)}] \cdot \mathbf{J}^{\top} \cdot \lambda_{k+2/3}, \end{aligned}$$

$$\begin{aligned} g_{\mathbf{u}}(k) &:= \frac{\delta}{2} \nabla_{1,3}^2 T[\Phi_{-(k+1/3)}, \theta, \mathbf{u}_{-(k+1)}] \cdot \mathbf{J}^{\top} \cdot \lambda_k \\ &+ \delta \nabla_{1,3}^2 V[\Phi_{-(k+2/3)}, \theta, \mathbf{u}_{-(k+1)}] \cdot \mathbf{J}^{\top} \cdot \lambda_{k+1/3} + \frac{\delta}{2} \nabla_{1,3}^2 T[\Phi_{-(k+1)}, \theta, \mathbf{u}_{-(k+1)}] \cdot \mathbf{J}^{\top} \cdot \lambda_{k+2/3}, \end{aligned}$$

and where  $(\lambda_k)$  satisfy the following recursion relationship, with  $\lambda_0 = \nabla_{\Phi} \ell[\Phi_0]$  and  $\forall k \in [0, K-1]$ :

$$\begin{cases} \lambda_{k+1/3} &= \lambda_k + \frac{\delta}{2} \nabla_1^2 T[\Phi_{-(k+1/3)}, \theta, \mathbf{u}_{-(k+1)}] \cdot \mathbf{J}^{\top} \cdot \lambda_k \\ \lambda_{k+2/3} &= \lambda_{k+1/3} + \delta \nabla_1^2 V[\Phi_{-(k+2/3)}, \theta, \mathbf{u}_{-(k+1)}] \cdot \mathbf{J}^{\top} \cdot \lambda_{k+1/3} \\ \lambda_{k+1} &= \lambda_{k+2/3} + \frac{\delta}{2} \nabla_1^2 T[\Phi_{-(k+1)}, \theta, \mathbf{u}_{-(k+1)}] \cdot \mathbf{J}^{\top} \cdot \lambda_{k+2/3} + \nabla_1 \ell[\Phi_{-(k+1)}, \theta] \end{cases}$$

*Proof of Corollary A.5.* Direct application of Theorem A.3 with the inference computational graph details defined inside Def. A.3.  $\square$

### A.2.3 Proof of Theorem 3.2

**Time reversibility in discrete time.** We first derive the discrete counterpart of Lemma A.3 as a technical pre-requisite for the extension of RHEL to the discrete-time setting.

**Lemma A.4** (Reversibility of  $\mathcal{M}_{H,\delta}$ ). *Under the assumptions of Def. A.3–A.4:*

$$\forall \Phi_k \in \mathbb{R}^{d_\Phi}, \forall \theta \in \mathbb{R}^{d_\theta}, \forall \mathbf{u} \in \mathbb{R}^{d_u} : \quad \Phi_{k+1} = \mathcal{M}_{H,\delta}[\Phi_k, \theta, \mathbf{u}] \Rightarrow \mathcal{M}_{H,\delta}[\Phi_{k+1}^*, \theta, \mathbf{u}] = \Phi_k^*$$

*Proof of Lemma A.4.* Let  $\Phi_k, \Phi_{k+1} \in \mathbb{R}^{d_\Phi}$  be such that:

$$\Phi_{k+1} = \mathcal{M}_{H,\delta}[\Phi_k, \theta, \mathbf{u}]$$

which rewrites, given Def. A.3:

$$\mathcal{M}_{H,\delta} : \begin{cases} \phi_{k+1/3} &= \phi_k + \frac{\delta}{2} \nabla_\pi T[\pi_k, \theta, \mathbf{u}] \\ \pi_{k+1/3} &= \pi_k \\ \phi_{k+2/3} &= \phi_{k+1/3} \\ \pi_{k+2/3} &= \pi_{k+1/3} - \delta \nabla_\phi V[\phi_{k+1/3}, \theta, \mathbf{u}] \\ \phi_{k+1} &= \phi_{k+2/3} + \frac{\delta}{2} \nabla_\pi T[\pi_{k+2/3}, \theta, \mathbf{u}] \\ \pi_{k+1} &= \pi_{k+2/3} \end{cases} \quad (20)$$

It becomes apparent from Eq. (20) that  $\mathcal{M}_{H,\delta}$  is invertible with respect to its first argument and that inverting  $\mathcal{M}_{H,\delta}$  amounts to change  $\delta$  to  $-\delta$ :

$$\mathcal{M}_{H,\delta}^{-1} : \begin{cases} \phi_{k+2/3} &= \phi_{k+1} - \frac{\delta}{2} \nabla_\pi T[\pi_{k+1}, \theta, \mathbf{u}] \\ \pi_{k+2/3} &= \pi_{k+1} \\ \phi_{k+1/3} &= \phi_{k+2/3} \\ \pi_{k+1/3} &= \pi_{k+2/3} + \delta \nabla_\phi V[\phi_{k+2/3}, \theta, \mathbf{u}] \\ \phi_k &= \phi_{k+1/3} - \frac{\delta}{2} \nabla_\pi T[\pi_{k+1/3}, \theta, \mathbf{u}] \\ \pi_k &= \pi_{k+1/3} \end{cases} \quad (21)$$

and therefore:

$$\Phi_k = \mathcal{M}_{H,\delta}^{-1}[\Phi_{k+1}, \theta, \mathbf{u}] = \mathcal{M}_{H,-\delta}[\Phi_{k+1}, \theta, \mathbf{u}],$$

Denoting  $\pi^* := -\pi$ , note that by time-reversal invariance hypothesis in Def. A.3 and Lemma A.2, we have that  $\nabla_\pi T[\pi, \theta, \mathbf{u}] = -\nabla_{\pi^*} T[\pi^*, \theta, \mathbf{u}]$ . Therefore, Eq. (21) rewrites as:

$$\begin{cases} \phi_{k+2/3} &= \phi_{k+1} + \frac{\delta}{2} \nabla_{\pi^*} T[\pi_{k+1}^*, \theta, \mathbf{u}] \\ \pi_{k+2/3}^* &= \pi_{k+1}^* \\ \phi_{k+1/3} &= \phi_{k+2/3} \\ \pi_{k+1/3}^* &= \pi_{k+2/3}^* - \delta \nabla_\phi V[\phi_{k+2/3}, \theta, \mathbf{u}] \\ \phi_k &= \phi_{k+1/3} + \frac{\delta}{2} \nabla_{\pi^*} T[\pi_{k+1/3}^*, \theta, \mathbf{u}] \\ \pi_k^* &= \pi_{k+1/3}^* \end{cases}, \quad (22)$$

where equations bearing on  $\pi$  have been multiplied on both sides by  $-1$ . Finally note that Eq. (22) simply rewrites as:

$$\mathcal{M}_{H,\delta}[\Phi_{k+1}^*, \theta, \mathbf{u}] = \Phi_k^*$$

□

**Corollary A.6.** *Under the assumptions of Def. A.3, if  $\Phi$  satisfies the following recursive equations:*

$$\Phi_{-K} = \mathbf{x}, \quad \forall k = -K, \dots, -1 : \quad \Phi_{k+1} = \mathcal{M}_{H,\delta}[\Phi_k, \theta, \mathbf{u}_k]$$

*and  $\Phi^e$  is subsequently defined as:*

$$\Phi_0^e = \Phi_0^*, \quad \forall t = 0, \dots, K-1 : \quad \Phi_{k+1}^e = \mathcal{M}_{H,\delta}[\Phi_k^e, \theta, \mathbf{u}_{-(k+1)}]$$

*Then:*

$$\forall t = 0, \dots, K-1 : \quad \Phi_k^e = \Phi_{-k}^*$$

*Proof of Corollary A.6.* This result is immediately obtained by iterating Lemma A.4 over the whole trajectory. □

**A technical pre-requisite.** Finally, we need one last technical Lemma to handle subtleties pertaining to Jacobian evaluation which only occur in discrete time.

**Lemma A.5.** *Under the assumptions of Def. A.3, if we have, for some  $\theta \in \mathbb{R}^{d_\theta}$  and  $u \in \mathbb{R}^{d_u}$ :*

$$\Phi_{k+1} = \mathcal{M}_{H,\delta}[\Phi_k, \theta, u],$$

*then:*

$$\begin{cases} T[\Phi_{k+1/3}, \theta, u] = T[\Phi_k, \theta, u], \\ V[\Phi_{k+2/3}, \theta, u] = V[\Phi_{k+1/3}, \theta, u] \\ T[\Phi_{k+1}, \theta, u] = T[\Phi_{k+2/3}, \theta, u] \end{cases}$$

*Proof.* This can be seen by simply writing  $\mathcal{M}_{H,\delta}$  explicitly for  $\phi$  and  $\pi$ :

$$\mathcal{M}_{H,\delta} : \begin{cases} \phi_{k+1/3} &= \phi_k + \frac{\delta}{2} \nabla_\pi T[\pi_k, \theta, u] \\ \pi_{k+1/3} &= \pi_k \\ \phi_{k+2/3} &= \phi_{k+1/3} \\ \pi_{k+2/3} &= \pi_{k+1/3} - \delta \nabla_\phi V[\phi_{k+1/3}, \theta, u] \\ \phi_{k+1} &= \phi_{k+2/3} + \frac{\delta}{2} \nabla_\pi T[\pi_{k+2/3}, \theta, u] \\ \pi_{k+1} &= \pi_{k+2/3} \end{cases}$$

□

**Discrete-time RHEL.** We are now ready to state the main result of this section.

**Corollary A.7.** *Under the assumptions of Def. A.3–A.4, let  $(\Phi_k)_k$  satisfy the recursive equation:*

$$\Phi_{-K} = x, \quad \Phi_{k+1} = \mathcal{M}_{H,\delta}[\Phi_k, \theta, u_k] \quad \forall k = -K, \dots, -1,$$

*and let  $\Phi^e$  satisfy:*

$$\begin{cases} \Phi_0^e(\epsilon) = \Phi_0^* + \epsilon \Sigma_x \cdot \nabla_\Phi \ell[\Phi_0, \theta, 0], \\ \forall k = 0, \dots, K-1 : \\ \quad \Phi_{k+1/3}^e(\epsilon) = \mathcal{M}_{T,\delta/2}[\Phi_k^e(\epsilon), \theta, u_{-(k+1)}] \\ \quad \Phi_{k+2/3}^e(\epsilon) = \mathcal{M}_{V,\delta}[\Phi_{k+1/3}^e(\epsilon), \theta, u_{-(k+1)}] \\ \quad \Phi_{k+1}^e(\epsilon) = \mathcal{M}_{T,\delta/2}[\Phi_{k+2/3}^e(\epsilon), \theta, u_{-(k+1)}] - \epsilon J \cdot \nabla_{\Phi^e} \ell[\Phi_{k+1}^e, \theta, -(k+1)], \end{cases}$$

*Then defining:*

$$\begin{aligned} H^{1/2}[\Phi_k^e(\epsilon), \theta, u_{-(k+1)}] &:= \frac{1}{2} \left( H[\Phi_{k+1/3}^e(\epsilon), \theta, u_{-(k+1)}] + H[\Phi_{k+2/3}^e(\epsilon), \theta, u_{-(k+1)}] \right), \\ \Delta_\theta^{\text{RHEL}}(k, \epsilon) &:= \nabla_2 \ell[\Phi_k^e(\epsilon), \theta, -k] \\ &\quad - \frac{\delta}{2\epsilon} \left( \nabla_2 H^{1/2}[\Phi_k^e(\epsilon), \theta, u_{-(k+1)}] - \nabla_2 H^{1/2}[\Phi_k^e(-\epsilon), \theta, u_{-(k+1)}] \right), \\ \Delta_u^{\text{RHEL}}(k, \epsilon) &:= -\frac{\delta}{2\epsilon} \left( \nabla_3 H^{1/2}[\Phi_k^e(\epsilon), \theta, u_{-(k+1)}] - \nabla_3 H^{1/2}[\Phi_k^e(-\epsilon), \theta, u_{-(k+1)}] \right), \\ \Delta_\Phi^{\text{RHEL}}(k, \epsilon) &:= \frac{1}{2\epsilon} \Sigma_x \cdot (\Phi_k^e(\epsilon) - \Phi_k^e(-\epsilon)), \end{aligned}$$

*we have:*

$$\begin{aligned} \forall k = 0, \dots, K-1 : \quad \lambda_k &= \lim_{\epsilon \rightarrow 0} \Delta_\theta^{\text{RHEL}}(k, \epsilon), \quad g_\theta(k) = \lim_{\epsilon \rightarrow 0} \Delta_\theta^{\text{RHEL}}(k, \epsilon), \\ g_u(k) &= \lim_{\epsilon \rightarrow 0} \Delta_u^{\text{RHEL}}(k, \epsilon), \end{aligned}$$

*where  $(\lambda_k)_{k \in \llbracket 0, K \rrbracket}$ ,  $(g_\theta(k))_{k \in \llbracket 0, K-1 \rrbracket}$  and  $(g_u(k))_{k \in \llbracket 0, K-1 \rrbracket}$  are defined inside Corollary (A.5).*

*Proof of Corollary A.7.* Let  $k \in [0, K - 1]$ . We define:

$$\Delta_{\Phi}^{\text{RHEL}}(k) := \lim_{\epsilon \rightarrow 0} \Delta_{\Phi}^{\text{RHEL}}(k, \epsilon) = \Sigma_x \cdot \partial_{\epsilon} \Phi(k, \epsilon)|_{\epsilon=0}$$

$$\Delta_{\theta}^{\text{RHEL}}(k) := \lim_{\epsilon \rightarrow 0} \Delta_{\theta}^{\text{RHEL}}(k, \epsilon) = \nabla_2 \ell[\Phi_k^e(0), \theta, -k] - \delta \partial_{\epsilon} \left( \nabla_2 H^{1/2}[\Phi_k^e(\epsilon), \theta, \mathbf{u}_{-(k+1)}] \right) \Big|_{\epsilon=0}$$

$$\Delta_{\mathbf{u}}^{\text{RHEL}}(k) := \lim_{\epsilon \rightarrow 0} \Delta_{\mathbf{u}}^{\text{RHEL}}(k, \epsilon) = -\delta \partial_{\epsilon} \left( \nabla_3 H^{1/2}[\Phi_k^e(\epsilon), \theta, \mathbf{u}_{-(k+1)}] \right) \Big|_{\epsilon=0}$$

**Derivation of  $\lambda_k = \lim_{\epsilon \rightarrow 0} \Delta_{\theta}^{\text{RHEL}}(k, \epsilon)$ .** We proceed exactly as in Theorem A.2 with some subtle adaptations which we highlight. Differentiating the dynamics of  $\Phi^e$  between  $k$  and  $k + 1/3$  and proceeding as in the proof of Theorem A.2 using Lemma A.1, Lemma A.2 and Corollary A.6 (as the discrete counterpart of Lemma A.3 which was used for Theorem A.2), we obtain:

$$\Delta_{\Phi}^{\text{RHEL}}(k + 1/3) = \Delta_{\Phi}^{\text{RHEL}}(k) + \frac{\delta}{2} \nabla_{\Phi}^2 T[\Phi_{-k}, \theta, \mathbf{u}_{-(k+1)}] \cdot \mathbf{J}^{\top} \cdot \Delta_{\Phi}^{\text{RHEL}}(k)$$

However note that this does not correctly match the dynamics satisfied by  $\lambda$  inside Corollary A.5 between  $k$  and  $k + 1/3$ : the Hessian  $\nabla_{\Phi}^2 T$  should instead be evaluated at  $\Phi_{-(k+1/3)}$ . Fortunately, using Lemma A.5:

$$\nabla_{\Phi}^2 T[\Phi_{-k}, \theta, \mathbf{u}_{-(k+1)}] = \nabla_{\Phi}^2 T[\Phi_{-(k+1/3)}, \theta, \mathbf{u}_{-(k+1)}]$$

Therefore we get:

$$\Delta_{\Phi}^{\text{RHEL}}(k + 1/3) = \Delta_{\Phi}^{\text{RHEL}}(k) + \frac{\delta}{2} \nabla_{\Phi}^2 T[\Phi_{-(k+1/3)}, \theta, \mathbf{u}_{-(k+1)}] \cdot \mathbf{J}^{\top} \cdot \Delta_{\Phi}^{\text{RHEL}}(k)$$

Proceeding the same way on  $\Phi_{k+2/3}^e$  and  $\Phi_{k+1}^e$ , we get altogether:

$$\begin{cases} \Delta_{\Phi}^{\text{RHEL}}(k + 1/3) = \Delta_{\Phi}^{\text{RHEL}}(k) + \frac{\delta}{2} \nabla_{\Phi}^2 T[\Phi_{-(k+1/3)}, \theta, \mathbf{u}_{-(k+1)}] \cdot \mathbf{J}^{\top} \cdot \Delta_{\Phi}^{\text{RHEL}}(k) \\ \Delta_{\Phi}^{\text{RHEL}}(k + 2/3) = \Delta_{\Phi}^{\text{RHEL}}(k + 1/3) + \delta \nabla_{\Phi}^2 V[\Phi_{-(k+2/3)}, \theta, \mathbf{u}_{-(k+1)}] \cdot \mathbf{J}^{\top} \cdot \Delta_{\Phi}^{\text{RHEL}}(k + 1/3) \\ \Delta_{\Phi}^{\text{RHEL}}(k + 1) = \Delta_{\Phi}^{\text{RHEL}}(k + 2/3) + \frac{\delta}{2} \nabla_{\Phi}^2 T[\Phi_{-(k+1)}, \theta, \mathbf{u}_{-(k+1)}] \cdot \mathbf{J}^{\top} \cdot \Delta_{\Phi}^{\text{RHEL}}(k + 2/3) \\ \quad + \nabla_{\Phi} \ell[\Phi_{-(k+1)}, \theta, -(k+1)] \end{cases}$$

$\Delta_{\Phi}^{\text{RHEL}}$  satisfying the same equations as  $(\lambda_k)_k$  given by BPTT, together with same initial conditions:

$$\Delta_{\Phi}^{\text{RHEL}}(0) = \nabla_{\Phi} \ell[\Phi_0, \theta, 0]$$

yields the desired equality.

**Derivation of  $g_{\theta}(k) = \lim_{\epsilon \rightarrow 0} \Delta_{\theta}^{\text{RHEL}}(k, \epsilon)$ .** Proceeding in the same way as in the derivation of Theorem A.2, starting from the expression of  $g_{\theta}(k)$  derived in Corollary A.5, using  $\Delta_{\Phi}^{\text{RHEL}}(k) = \lambda_k$  and paying attention to evaluating Jacobian at the right places using Lemma A.5, we obtain  $\forall k = 0, \dots, K - 1$ :

$$g_{\theta}(k) = \nabla_2 \ell[\Phi_k^e(0), \theta, -k] - \frac{\delta}{2} \partial_{\epsilon} \left\{ \nabla_{\theta} T[\Phi_k^e(\epsilon), \theta, \mathbf{u}_{-(k+1)}] + 2 \nabla_{\theta} V[\Phi_{k+1/3}^e(\epsilon), \theta, \mathbf{u}_{-(k+1)}] + \nabla_{\theta} T[\Phi_{k+2/3}^e(\epsilon), \theta, \mathbf{u}_{-(k+1)}] \right\} \Big|_{\epsilon=0}.$$

There again, using Lemma A.5:

$$\nabla_{\theta} T[\Phi_k^e(\epsilon), \theta, \mathbf{u}_{-(k+1)}] = \nabla_{\theta} T[\Phi_{k+1/3}^e(\epsilon), \theta, \mathbf{u}_{-(k+1)}], \quad \nabla_{\theta} V[\Phi_{k+1/3}^e(\epsilon), \theta, \mathbf{u}_{-(k+1)}] = \nabla_{\theta} V[\Phi_{k+2/3}^e(\epsilon), \theta, \mathbf{u}_{-(k+1)}],$$

therefore:

$$\begin{aligned} g_{\theta}(k) &= \nabla_2 \ell[\Phi_k^e(0), \theta, -k] - \frac{\delta}{2} \partial_{\epsilon} \left\{ \nabla_{\theta} T[\Phi_{k+1/3}^e(\epsilon), \theta, \mathbf{u}_{-(k+1)}] + \nabla_{\theta} V[\Phi_{k+1/3}^e(\epsilon), \theta, \mathbf{u}_{-(k+1)}] \right. \\ &\quad \left. + \nabla_{\theta} V[\Phi_{k+2/3}^e(\epsilon), \theta, \mathbf{u}_{-(k+1)}] + \nabla_{\theta} T[\Phi_{k+2/3}^e(\epsilon), \theta, \mathbf{u}_{-(k+1)}] \right\} \Big|_{\epsilon=0} \\ &= \nabla_2 \ell[\Phi_k^e(0), \theta, -k] - \frac{\delta}{2} \partial_{\epsilon} \left( H[\Phi_{k+1/3}^e(\epsilon), \theta, \mathbf{u}_{-(k+1)}] + H[\Phi_{k+2/3}^e(\epsilon), \theta, \mathbf{u}_{-(k+1)}] \right) \Big|_{\epsilon=0} \\ &= \nabla_2 \ell[\Phi_k^e(0), \theta, -k] - \delta \partial_{\epsilon} \left( H^{1/2}[\Phi_k^e(\epsilon), \theta, \mathbf{u}_{-(k+1)}] \right) \Big|_{\epsilon=0} = \lim_{\epsilon \rightarrow 0} \Delta_{\theta}^{\text{RHEL}}(k, \epsilon) \end{aligned}$$



**Derivation of  $g_u(k) = \lim_{\epsilon \rightarrow 0} \Delta_u^{\text{RHEL}}(k, \epsilon)$ .** Strictly identical to the above paragraph.  $\square$

**Remark 6.** Note that the echo dynamics prescribed by Corollary A.7 are *implicit*:

$$\begin{cases} \Phi_0^e(\epsilon) = \Phi_0^* + \epsilon \Sigma_x \cdot \nabla_{\Phi} \ell[\Phi_0, \theta, 0], \\ \forall k = 0, \dots, K-1 : \\ \quad \Phi_{k+1/3}^e(\epsilon) = \mathcal{M}_{T, \delta/2}[\Phi_k^e(\epsilon), \theta, \mathbf{u}_{-(k+1)}] \\ \quad \Phi_{k+2/3}^e(\epsilon) = \mathcal{M}_{V, \delta}[\Phi_{k+1/3}^e(\epsilon), \theta, \mathbf{u}_{-(k+1)}] \\ \quad \Phi_{k+1}^e(\epsilon) = \mathcal{M}_{T, \delta/2}[\Phi_{k+2/3}^e(\epsilon), \theta, \mathbf{u}_{-(k+1)}] - \epsilon \mathbf{J} \cdot \nabla_{\Phi^e} \ell[\Phi_{k+1}^e, \theta, -(k+1)], \end{cases}$$

where  $\Phi_{k+1}^e$  appears, as highlighted in red, on both sides of the last step. A straightforward way to make the echo dynamics explicit while preserving the theoretical guarantees of Corollary A.7 is to linearize the nudging signal, namely using instead the following set of equations:

$$\begin{cases} \Phi_0^e(\epsilon) = \Phi_0^* + \epsilon \Sigma_x \cdot \nabla_{\Phi} \ell[\Phi_0, \theta, 0], \\ \forall k = 0, \dots, K-1 : \\ \quad \Phi_{k+1/3}^e(\epsilon) = \mathcal{M}_{T, \delta/2}[\Phi_k^e(\epsilon), \theta, \mathbf{u}_{-(k+1)}] \\ \quad \Phi_{k+2/3}^e(\epsilon) = \mathcal{M}_{V, \delta}[\Phi_{k+1/3}^e(\epsilon), \theta, \mathbf{u}_{-(k+1)}] \\ \quad \Phi_{k+1}^e(\epsilon) = \mathcal{M}_{T, \delta/2}[\Phi_{k+2/3}^e(\epsilon), \theta, \mathbf{u}_{-(k+1)}] + \epsilon \Sigma_x \cdot \nabla_{\Phi^e} \ell[\Phi_{-(k+1)}, \theta, -(k+1)], \end{cases}$$

Note that the  $\epsilon \mathbf{J}$  of the original implicit equation becomes  $-\epsilon \Sigma_x$  in its linearized counterpart.

This remark leads us to a slight variant of Corollary A.7.

**Corollary A.8.** Under the assumptions of Def. A.3–A.4, let  $(\Phi_k)_k$  satisfy the recursive equation:

$$\Phi_{-K} = x, \quad \Phi_{k+1} = \mathcal{M}_{H, \delta}[\Phi_k, \theta, \mathbf{u}_k] \quad \forall k = -K, \dots, -1,$$

and let  $\Phi^e$  satisfy:

$$\begin{cases} \Phi_0^e(\epsilon) = \Phi_0^* + \epsilon \Sigma_x \cdot \mathbf{y}_0, \\ \forall k = 0, \dots, K-1 : \\ \quad \Phi_{k+1/3}^e(\epsilon) = \mathcal{M}_{T, \delta/2}[\Phi_k^e(\epsilon), \theta, \mathbf{u}_{-(k+1)}] \\ \quad \Phi_{k+2/3}^e(\epsilon) = \mathcal{M}_{V, \delta}[\Phi_{k+1/3}^e(\epsilon), \theta, \mathbf{u}_{-(k+1)}] \\ \quad \Phi_{k+1}^e(\epsilon) = \mathcal{M}_{T, \delta/2}[\Phi_{k+2/3}^e(\epsilon), \theta, \mathbf{u}_{-(k+1)}] + \epsilon \Sigma_x \cdot \mathbf{y}_{-(k+1)}, \end{cases}$$

where  $\mathbf{y} \in \mathbb{R}^{K \times d_{\Phi}}$  does not depend on  $\Phi$ . Then the same conclusions as Corollary A.7 hold, with  $(\lambda_k)_{k \in \llbracket 0, K-1 \rrbracket}$  satisfying  $\lambda_0 = \mathbf{y}_0$  and  $\forall k \in \llbracket 0, K-1 \rrbracket$ :

$$\begin{cases} \lambda_{k+1/3} &= \lambda_k + \frac{\delta}{2} \nabla_1^2 T[\Phi_{-(k+1/3)}, \theta, \mathbf{u}_{-(k+1)}] \cdot \mathbf{J}^\top \cdot \lambda_k \\ \lambda_{k+2/3} &= \lambda_{k+1/3} + \delta \nabla_1^2 V[\Phi_{-(k+2/3)}, \theta, \mathbf{u}_{-(k+1)}] \cdot \mathbf{J}^\top \cdot \lambda_{k+1/3} \\ \lambda_{k+1} &= \lambda_{k+2/3} + \frac{\delta}{2} \nabla_1^2 T[\Phi_{-(k+1)}, \theta, \mathbf{u}_{-(k+1)}] \cdot \mathbf{J}^\top \cdot \lambda_{k+2/3} + \mathbf{y}_{-(k+1)} \end{cases}$$

*Proof Corollary A.8.* Identical to the proof of Corollary A.7.  $\square$

#### A.2.4 Proof of Theorem 3.3

**Theorem A.4.** Assuming a HSSM (Def. A.5) and the optimization problem depicted in Def. A.6, we have:

$$\forall \ell = 0, \dots, N-1 : \quad d_{\theta^{(\ell)}} L = \lim_{\epsilon \rightarrow 0} \Delta \theta^{(\ell)}(\epsilon),$$

where  $\Delta \theta^{(\ell)}(\epsilon)$  can be recursively computed backwards, starting from the **top-most block** as:

$$\begin{aligned} \bar{\Phi}^{e,(N)}(\epsilon) &= \mathcal{M}_{H^{(N-1)}, \delta}[\bar{\Phi}^{e,(N)}(\epsilon), \theta^{(N-1)}, \tilde{\Phi}^{(N-1)}] + \epsilon \Sigma_x \cdot \nabla_{\Phi^{(N)}} L \\ \Delta \theta^{(N-1)}(\epsilon) &= \sum_{k=0}^{K-1} \nabla_2 \ell[\Phi_k^e, \theta^{(N-1)}, -k] \\ &\quad - \frac{\delta}{2\epsilon} \sum_{k=0}^{K-1} \left( \nabla_2 H^{1/2}[\Phi_k^{e,(L)}(\epsilon), \theta^{(N-1)}, \Phi_{-k}^{(N-1)}] - \nabla_2 H^{1/2}[\Phi_k^{e,(N)}(-\epsilon), \theta^{(N-1)}, \Phi_{-k}^{(N-1)}] \right), \\ \bar{\Delta}_{\Phi^{(N-1)}}(\epsilon) &= -\frac{\delta}{2\epsilon} \left( \nabla_3 H^{1/2}[\bar{\Phi}^{e,(N)}(\epsilon), \theta^{(N-1)}, \tilde{\Phi}^{(N-1)}] - \nabla_3 H^{1/2}[\bar{\Phi}^{e,(L)}(-\epsilon), \theta^{(N-1)}, \tilde{\Phi}^{(N-1)}] \right) \end{aligned}$$

and subsequently for **upstream blocks**, i.e.  $\forall \ell = N-2, \dots, 0$ :

$$\begin{aligned} \bar{\Phi}^{e,(\ell+1)}(\epsilon) &= \mathcal{M}_{H^{(\ell)}, \delta}[\bar{\Phi}^{e,(\ell+1)}(\epsilon), \theta^{(\ell)}, \tilde{\Phi}^{(\ell)}] + \epsilon \Sigma_x \cdot \tilde{\Delta}_{\Phi^{(\ell+1)}}(\epsilon) \\ \Delta \theta^{(\ell)}(\epsilon) &= -\frac{\delta}{2\epsilon} \sum_{k=0}^{K-1} \left( \nabla_2 H^{1/2}[\Phi_k^{e,(\ell+1)}(\epsilon), \theta^{(\ell)}, \Phi_{-k}^{(\ell)}] - \nabla_2 H^{1/2}[\Phi_k^{e,(\ell+1)}(-\epsilon), \theta^{(\ell)}, \Phi_{-k}^{(\ell)}] \right) \\ \Delta_{\Phi^{(\ell)}}(\epsilon) &= -\frac{\delta}{2\epsilon} \left( \nabla_3 H^{1/2}[\bar{\Phi}^{e,(\ell+1)}(\epsilon), \theta^{(\ell)}, \tilde{\Phi}^{(\ell)}] - \nabla_3 H^{1/2}[\bar{\Phi}^{e,(\ell+1)}(-\epsilon), \theta^{(\ell)}, \tilde{\Phi}^{(\ell)}] \right) \end{aligned}$$

*Proof.* Re-using the vectorized notations introduced in subsection 3.3 and used in Remark 5, the Lagrangian associated to the optimization problem depicted in Def. A.6 reads:

$$\begin{aligned} \mathcal{L} &= \mathbf{1}^\top \cdot \ell[\tilde{\Phi}^{(L)}, \theta^{(L-1)}] + \sum_{\ell=0}^{L-1} \text{Tr} \left[ \left( \mathcal{M}_{H^{(\ell)}, \delta}[\tilde{\Phi}^{(\ell+1)}, \theta^{(\ell)}, \tilde{\mathbf{u}}^{(\ell)}] - \tilde{\Phi}^{(\ell+1)} \right) \cdot \left( \bar{\lambda}^{(\ell+1)} \right)^\top \right] \\ &= \sum_{k=0}^{K-1} \ell[\Phi_{-k}^{(L)}, \theta^{(L)}, -k] + \sum_{\ell=0}^{L-1} \left( \lambda_k^{(\ell+1)} \right)^\top \cdot \left( \mathcal{M}_{H^{(\ell)}, \delta}[\Phi_{-(k+1)}^{(\ell+1)}, \theta^{(\ell)}, \Phi_{-(k+1)}^{(\ell)}] - \Phi_{-k}^{(\ell+1)} \right) \end{aligned}$$

We proceed by induction on the block index starting from  $\ell = L$ .

**Initialization** ( $\ell = L$ ). Let  $\Phi_k^{(N)}$  and  $\lambda_k^{(N)}$  for  $k \in \llbracket 0, K-1 \rrbracket$  be the critical points of  $\mathcal{L}$ . By Theorem A.3:

$$d_{\theta^{(N-1)}} L = \sum_{k=0}^{K-1} \nabla_2 \ell[\Phi_{-k}^{(N)}, \theta^{(N-1)}, -k] + \partial_2 \mathcal{M}_{H^{(N-1)}, \delta} \left[ \Phi_{-(k+1)}^{(N)}, \theta^{(N-1)}, \Phi_{-(k+1)}^{(N-1)} \right]^\top \cdot \lambda_k^{(N)},$$

with  $(\lambda_k^{(N)})_{k \in \llbracket 0, K-1 \rrbracket}$  satisfying the following recursion relationship:

$$\begin{cases} \lambda_0^{(N)} = \nabla_1 \ell[\Phi_0^{(N)}, \theta^{(N-1)}, 0], \\ \forall k = 0, \dots, K-1 : \\ \lambda_{k+1}^{(N)} = \partial_1 \mathcal{M}_{H^{(N-1)}, \delta} \left[ \Phi_{-(k+1)}^{(N)}, \theta^{(N-1)}, \Phi_{-(k+1)}^{(N-1)} \right]^\top \cdot \lambda_k^{(N)} + \nabla_1 \ell \left[ \Phi_{-(k+1)}^{(N)}, \theta, -(k+1) \right] \end{cases}$$

Given the definition of the dynamics of  $\Phi^{e,(N)}$  by hypothesis, we can directly apply Corollary A.7 to obtain:

$$\begin{aligned} d_{\theta^{(N-1)}} L &= \sum_{k=0}^{K-1} \nabla_2 \ell \left[ \Phi_{-k}^{(N)}, \theta^{(N-1)}, -k \right] - \delta \partial_\epsilon \left( \nabla_2 H^{(N-1), 1/2} \left[ \Phi_k^{e,(N)}(\epsilon), \theta^{(L-1)}, \Phi_{-(k+1)}^{(N-1)} \right] \right) \Big|_{\epsilon=0}, \\ d_{\Phi_{k-1}^{(N-1)}} L &= \partial_3 \mathcal{M}_{H^{(N-1)}, \delta} \left[ \Phi_{-(k+1)}^{(N)}, \theta^{(N-1)}, \Phi_{-(k+1)}^{(N-1)} \right]^\top \cdot \lambda_k^{(N)} \\ &= -\delta \partial_\epsilon \left( \nabla_3 H^{(N-1), 1/2} \left[ \Phi_k^{e,(N)}(\epsilon), \theta^{(L-1)}, \Phi_{-(k+1)}^{(N-1)} \right] \right) \Big|_{\epsilon=0} \end{aligned}$$

**Induction** ( $\ell + 1 \rightarrow \ell$ ). Let us assume that the desired property is satisfied at layer  $\ell + 1$ . We denote again  $\Phi_k^{(\ell)}$  and  $\lambda_k^{(\ell)}$  for  $k \in \llbracket 0, K - 1 \rrbracket$  the critical point of  $\mathcal{L}$ . We have, for  $k \in \llbracket 0, K - 1 \rrbracket$ :

$$\begin{cases} \lambda_0^{(\ell)} = \partial_3 \mathcal{M}_{H^{(\ell)}, \delta} \left[ \Phi_0^{(\ell+1)}, \theta^{(\ell)}, \Phi_0^{(\ell)} \right]^\top \cdot \lambda_0^{(\ell+1)} \\ \forall k = 0, \dots, K - 1 : \\ \lambda_{k+1}^{(\ell)} = \partial_1 \mathcal{M}_{H^{(\ell-1)}, \delta} \left[ \Phi_{-(k+1)}^{(\ell)}, \theta^{(\ell-1)}, \Phi_{-(k+1)}^{(\ell-1)} \right]^\top \cdot \lambda_k^{(\ell)} + \partial_3 \mathcal{M}_{H^{(\ell)}, \delta} \left[ \Phi_{-(k+1)}^{(\ell+1)}, \theta^{(\ell)}, \Phi_{-(k+1)}^{(\ell)} \right]^\top \cdot \lambda_k^{(\ell+1)} \end{cases}$$

Using the induction hypothesis at layer  $(\ell + 1)$ :

$$\begin{aligned} \partial_3 \mathcal{M}_{H^{(\ell)}, \delta} \left[ \Phi_{-(k+1)}^{(\ell+1)}, \theta^{(\ell)}, \Phi_{-(k+1)}^{(\ell)} \right]^\top \cdot \lambda_k^{(\ell+1)} &= -\delta \partial_\epsilon \left( \nabla_3 H^{(\ell), 1/2} \left[ \Phi_k^{e, (\ell+1)}(\epsilon), \theta^{(\ell)}, \Phi_{-(k+1)}^{(\ell)} \right] \right) \Big|_{\epsilon=0} \\ &= \lim_{\epsilon \rightarrow 0} \Delta_{\Phi_{(k+1)}^{(\ell)}}(\epsilon) \end{aligned}$$

Therefore on the one hand, denoting  $\Delta_{\Phi^{(\ell)}} := \lim_{\epsilon \rightarrow 0} \Delta_{\Phi^{(\ell)}}(\epsilon) \in \mathbb{R}^{K \times d_\Phi}$ , the dynamics on  $\lambda$  rewrite:

$$\begin{cases} \lambda_0^{(\ell)} = \Delta_{\Phi_0^{(\ell)}} \\ \forall k = 0, \dots, K - 1 : \\ \lambda_{k+1}^{(\ell)} = \partial_1 \mathcal{M}_{H^{(\ell-1)}, \delta} \left[ \Phi_{-(k+1)}^{(\ell)}, \theta^{(\ell-1)}, \Phi_{-(k+1)}^{(\ell-1)} \right]^\top \cdot \lambda_k^{(\ell)} + \Delta_{\Phi_{(k+1)}^{(\ell)}} \end{cases}$$

On the other hand, the dynamics of  $\Phi^{e, (\ell)}$  read by *hypothesis*:

$$\begin{cases} \Phi_0^{(\ell)} = \left( \Phi_0^{(\ell)} \right)^* + \epsilon \Sigma_x \cdot \Delta_{\Phi_0^{(\ell)}}(\epsilon) \\ \forall k = 0, \dots, K - 1 : \\ \Phi_{k+1}^{e, (\ell)} = \mathcal{M}_{H^{(\ell-1)}, \delta} \left[ \Phi_{k+1}^{e, (\ell)}, \theta^{(\ell-1)}, \Phi_{-(k+1)}^{(\ell-1)} \right] + \epsilon \Sigma_x \cdot \Delta_{\Phi_{(k+1)}^{(\ell)}} \end{cases}$$

using Corollary A.8 with  $y = \Delta_{\Phi^{(\ell)}}$ , we conclude that:

$$d_{\theta^{(\ell)}} L = \lim_{\epsilon \rightarrow 0} \Delta_{\theta^{(\ell)}}(\epsilon)$$

□

**Remark 7.** Theorem A.4, and more generally our definition of HSSMs (Def. A.5) assume that the connectivity pattern of the HRU units is a linear chain. Note that while we chose this hypothesis for the sake of clarity of our results and their derivations, Theorem A.4 could be seamlessly extended to any Directed Acyclic Graph (DAG) of HRUs. This allows, for instance as a simple and realistic case, to use skip connections across HRUs within HSSMs.

**Remark 8.** Note that RHEL chaining as prescribed by Theorem A.4 **implicitly chains RHEL and automatic differentiation**. Indeed, if  $H$  explicitly parametrizes feedforward mappings across HRUs as:

$$H^{(\ell)} \left[ \Phi_k^{e, (\ell+1)}(\epsilon), \theta^{(\ell)}, \Phi_{-(k+1)}^{(\ell)} \right] = H^{(\ell)} \left[ \Phi_k^{e, (\ell+1)}(\epsilon), \theta_\alpha^{(\ell)}, F \left[ \Phi_{-(k+1)}^{(\ell)}, \theta_\beta^{(\ell)} \right] \right], \quad (23)$$

then, denoting  $\mathbf{u}^{(\ell)} := F \left[ \Phi_{-(k+1)}^{(\ell)}, \theta_\beta^{(\ell)} \right]$ , we have:

$$\begin{aligned} &\partial_\epsilon \left( \nabla_3 H^{(\ell), 1/2} \left[ \Phi_k^{e, (\ell+1)}(\epsilon), \theta^{(\ell)}, \Phi_{-(k+1)}^{(\ell)} \right] \right) \Big|_{\epsilon=0} \\ &= \partial_1 F \left[ \Phi_{-(k+1)}^{(\ell)}, \theta_\beta^{(\ell)} \right]^\top \cdot \partial_\epsilon \left( \nabla_3 H^{(\ell), 1/2} \left[ \Phi_k^{e, (\ell+1)}(\epsilon), \theta_\alpha^{(\ell)}, \mathbf{u}^{(\ell)} \right] \right) \Big|_{\epsilon=0} \\ &\approx \partial_1 F \left[ \Phi_{-(k+1)}^{(\ell)}, \theta_\beta^{(\ell)} \right]^\top \cdot \frac{1}{2\epsilon} \left( \nabla_3 H^{(\ell), 1/2} \left[ \Phi_k^{e, (\ell+1)}(\epsilon), \theta_\alpha^{(\ell)}, \mathbf{u}^{(\ell)} \right] - \nabla_3 H^{(\ell), 1/2} \left[ \Phi_k^{e, (\ell+1)}(-\epsilon), \theta_\alpha^{(\ell)}, \mathbf{u}^{(\ell)} \right] \right) \end{aligned}$$

The red part is done by automatic differentiation and the blue part by RHEL. This underpins the implementation of RHEL chaining we used in our own code.

### A.3 Models and algorithms details

**Summary.** In this section, we provide details about our models and algorithms. More precisely:

- In section A.3.1, we first describe our *toy model* used inside Fig. 2 in terms of its Hamiltonian, resulting continuous-time dynamics and RHEL gradient estimators as prescribed by Theorem 3.1.
- In section A.3.2, we provide details about the HSSMs which we used in our experiments. We describe in greater details HSSMs made up of *linear* HRU blocks (section A.3.3). We describe their Hamiltonian, their resulting dynamics, the associated gradient estimators prescribed by RHEL and explain how *parallel scan* can be used on these models, especially when applying RHEL. Similarly, we describe HSSMs made up of *nonlinear* HRU blocks (section A.3.4).
- In section A.3.5, we show how the time discretization  $\delta$  can itself be trained by absorbing it into the definition of the Hamiltonian function.
- Finally, in the light of Remark 8, we highlight in section A.3.6 how our implementation hybridizes *Automatic Differentiation* (AD) and RHEL using Algorithms 3–4.

#### A.3.1 Toy model

In this section, we provide the gradient estimators for the parameters of the toy model. The toy model is a simple network of six mechanically coupled oscillators. Each oscillator is described by a state  $\Phi^i = (\phi^i, \pi^i)$  where  $\phi^i \in \mathbb{R}$  is the position of the oscillator and  $\pi^i \in \mathbb{R}$  is its momentum. It has a mass parameter  $m_i$  and spring parameter  $k_i$ . Any pair of oscillators  $(i, j)$  is coupled via the spring parameter  $k_{ij}$ . The input to the models is a time-varying external force  $\mathbf{u}(t) \in \mathbb{R}$  coupled to oscillator 1. During the echo passes, the nudging force is modelled by a spring coupling with parameter  $\epsilon \in \mathbb{R}$  to an external force  $y(t) \in \mathbb{R}$ . The Hamiltonian of the system is given by:

$$H[\Phi, \theta, \mathbf{u}] = \sum_i \frac{(\pi^i)^2}{2m_i} + \frac{1}{2} \sum_i k_i (\phi^i)^2 + \frac{1}{2} \sum_i \sum_{j>i} k_{ij} (\phi^j - \phi^i)^2 + \mathbf{u} \phi^1 \quad (24)$$

Which gives the following equations of motion:

$$\begin{cases} \dot{\phi}^i = \frac{\pi^i}{m_i} & \text{for all } i \in \{1, 6\} \\ \dot{\pi}^i = -k_i \phi^i + \sum_{j, j \neq i} k_{ij} (\phi^j - \phi^i), & i \in \{2, 3, 5, 6\} \\ \dot{\pi}^1 = -k_1 \phi^1 + \sum_{j, j \neq 1} k_{1j} (\phi^j - \phi^1) + \mathbf{u} \\ \dot{\pi}^4 = -k_4 \phi^4 + \sum_{j, j \neq 4} k_{4j} (\phi^j - \phi^4) - \delta_e \epsilon (\phi^4 - y) \end{cases}$$

where  $\delta_e$  is the indicator function of the echo pass, it's equal to 1 during the echo pass and 0 otherwise.

**RHEL gradient estimators of the model parameters.** For the mass  $m_i$ , we have:

$$\begin{aligned} \Delta_{m_i}^{RHEL} &= -\frac{1}{2\epsilon} (\nabla_{m_i} H[\Phi^e(t, \epsilon), \theta, \mathbf{u}] - \nabla_{m_i} H[\Phi^e(t, -\epsilon), \theta, \mathbf{u}]) \\ &= \frac{1}{2\epsilon} \left( \frac{(\pi^i(t, \epsilon))^2}{2m_i^2} - \frac{(\pi^i(t, -\epsilon))^2}{2m_i^2} \right) \end{aligned} \quad (25)$$

For the spring parameters  $k_i$ , we have:

$$\begin{aligned} \Delta_{k_i}^{RHEL} &= -\frac{1}{2\epsilon} (\nabla_{k_i} H[\Phi^e(t, \epsilon), \theta, \mathbf{u}] - \nabla_{k_i} H[\Phi^e(t, -\epsilon), \theta, \mathbf{u}]) \\ &= -\frac{1}{2\epsilon} ((\phi^i(t, \epsilon))^2 - (\phi^i(t, -\epsilon))^2) \end{aligned} \quad (26)$$

$$(27)$$

For the coupling parameters  $k_{ij}$ , we have:

$$\begin{aligned}\Delta_{k_{ij}}^{RHEL} &= -\frac{1}{2\epsilon} (\nabla_{k_{ij}} H[\Phi^e(t, \epsilon), \theta, \mathbf{u}] - \nabla_{k_{ij}} H[\Phi^e(t, -\epsilon), \theta, \mathbf{u}]) \\ &= -\frac{1}{2\epsilon} ((\phi^j(t, \epsilon) - \phi^i(t, \epsilon))^2 - (\phi^j(t, -\epsilon) - \phi^i(t, -\epsilon))^2)\end{aligned}\quad (28)$$

**RHEL gradient estimators of the state sensitivities** For the state position sensitivities, we have:

$$\begin{aligned}\Delta_{\phi^i}^{RHEL} &= -\frac{1}{2\epsilon} \Sigma_x (\Phi^e(t, \epsilon) - \Phi^e(t, -\epsilon)) \\ &= -\frac{1}{2\epsilon} (\pi^i(t, \epsilon) - \pi^i(t, -\epsilon))\end{aligned}\quad (29)$$

For the state momentum sensitivities, we have:

$$\begin{aligned}\Delta_{\pi^i}^{RHEL} &= -\frac{1}{2\epsilon} \Sigma_x (\Phi^e(t, \epsilon) - \Phi^e(t, -\epsilon)) \\ &= -\frac{1}{2\epsilon} (\phi^i(t, \epsilon) - \phi^i(t, -\epsilon))\end{aligned}\quad (30)$$

### A.3.2 HSSM architecture

In this section, we outline the detailed architecture of a full multi-layer HSSM architecture. For the inference, we re-use the same stacking architecture of recurrent blocks and feedforward elements as the LinOSS model [29]. The model starts by encoding an input sequence  $\bar{\mathbf{u}} \in \mathbb{R}^{d_u \times K}$  via an affine transformation. The transformed sequence then progresses through multiple HSSM blocks, linear (see Appendix A.3.3), or nonlinear (see Appendix A.3.4), directly followed by nonlinear transformations. These transformations include the Gaussian error linear unit (GELU) [55] and the Gated Linear Unit (GLU) [56], defined as  $\text{GLU}(\mathbf{x}) = \text{sigmoid}(\mathbf{W}_1 \mathbf{x}) \circ \mathbf{W}_2 \mathbf{x}$  where  $\mathbf{W}_{1,2}$  represent trainable weight matrices, accompanied by a residual connection. The sequence output from the final block undergoes a second affine transformation to produce the model output.

The full linear and nonlinear HSSM is further presented in Algorithm 2. For clarity, when applying operations to sequence elements denoted with an overline (e.g.,  $\bar{\mathbf{u}}$ ), these operations are implicitly broadcast across the time dimension. Specifically, for any function  $f$  applied to  $\bar{\mathbf{u}}$ , we have  $f(\bar{\mathbf{u}})_t = f(\mathbf{u}_t)$  for all time steps  $t \in \{1, 2, \dots, K\}$ .

For the inference of the linear HSSMs, we keep the same recurrent block as LinOSS with a slight change in the integrator (see A.3.3). For the nonlinear HSSM, we replace the recurrent block by a UniCORRN recurrent block [30] for which we use the same integrator as for the linear HSSM (see A.3.4).

---

#### Algorithm 2 HSSM model

---

```

1: Input: Input sequence  $\bar{\mathbf{u}}$ , model type type  $\in \{\text{linear}, \text{nonlinear}\}$ 
2: Output: HSSM output sequence  $\bar{\mathbf{u}}$ 
3:  $\bar{\mathbf{u}}^{(0)} \leftarrow \mathbf{W}_{enc} \bar{\mathbf{u}} + \mathbf{b}_{enc}$ 
4: for  $\ell = 1, \dots, N$  do
5:   if type = linear then
6:      $\bar{\Phi}^{(\ell)}, \_, \_ \leftarrow \text{LINEARHRU}(\bar{\mathbf{u}}^{(\ell-1)}, 0)$  ▷ Via parallel scan
7:   else
8:      $\bar{\Phi}^{(\ell)}, \_, \_ \leftarrow \text{NONLINEARHRU}(\bar{\mathbf{u}}^{(\ell-1)}, 0)$  ▷ Sequentially
9:   end if
10:   $\bar{\mathbf{x}}^{(\ell)} \leftarrow \mathbf{C} \bar{\Phi}^{(\ell)} + \mathbf{D} \bar{\mathbf{u}}^{\ell-1}$ 
11:   $\bar{\mathbf{x}}_g^{(\ell)} \leftarrow \text{GELU}(\bar{\mathbf{x}}^{(\ell)})$ 
12:   $\bar{\mathbf{u}}^{(\ell)} \leftarrow \text{GLU}(\bar{\mathbf{x}}_g^{(\ell)} + \bar{\mathbf{u}}^{(\ell-1)})$ 
13: end for
14:  $\bar{\mathbf{o}} \leftarrow \mathbf{W}_{dec} \bar{\mathbf{u}}^{(N)} + \mathbf{b}_{dec}$ 

```

---

### A.3.3 Linear HRU Block

**Hamiltonian of the recurrence.** The linear HRU block is the composition of a nonlinear spatial transformation and a linear recurrent transformation (see Eq. 17). Here we provide more details about the linear recurrence that is computed with the RHEL gradient estimator. The linear recurrence is defined by the following Hamiltonian:

$$\begin{aligned} H[\Phi, \theta, u] &= T[\pi, \theta, u] + V[\phi, \theta, u] \\ &= \frac{1}{2} \|\pi\|^2 + \left( \frac{1}{2} \phi^\top A \phi - \phi^\top B u \right) \end{aligned} \quad (31)$$

**Dynamics.** The dynamics of the linear HRU block are defined by the following equations:

$$\begin{cases} \dot{\phi} = \pi \\ \dot{\pi} = -A\phi + Bu \end{cases} \quad (32)$$

Which, after time-discretization with the integrator defined in A.2.1, gives the following equations:

$$\begin{cases} \phi_{k+1/3} = \phi_k + \frac{\delta}{2} \nabla_{\pi} T[\pi_k, \theta, u_k] \\ \quad = \phi_k + \frac{\delta}{2} \pi_k \\ \pi_{k+1/3} = \pi_k \\ \phi_{k+2/3} = \phi_{k+1/3} \\ \pi_{k+2/3} = \pi_{k+1/3} + \delta \nabla_{\phi} V[\phi_{k+1/3}, \theta, u_k] \\ \quad = \pi_{k+1/3} - \delta A \phi_{k+1/3} + \delta B u_k \\ \phi_{k+1} = \phi_{k+2/3} + \frac{\delta}{2} \nabla_{\pi} T[\pi_{k+2/3}, \theta, u_k] \\ \quad = \phi_{k+2/3} + \frac{\delta}{2} \pi_{k+2/3} \\ \pi_{k+1} = \pi_{k+2/3} \end{cases} \quad (33)$$

with the initial condition  $\Phi_{-K} = (\phi_{-K}^\top, \pi_{-K}^\top)^\top = x$ .

For the echo passes, the initial condition are  $\Phi_0^e = \Phi_0^* \pm \epsilon \Sigma_x \cdot \Delta_{\Phi} \ell[\Phi_0, 0]$ . The dynamics equations follow below, with modifications from equation 33 highlighted in blue::

$$\begin{cases} \phi_{k+1/3}^e = \phi_k^e + \frac{\delta}{2} \pi_k^e \\ \pi_{k+1/3}^e = \pi_k^e \\ \phi_{k+2/3}^e = \phi_{k+1/3}^e \\ \pi_{k+2/3}^e = \pi_{k+1/3}^e - \delta A \phi_{k+1/3}^e + \delta B u_{-(k+1)} \\ \phi_{k+1}^e = \phi_{k+2/3}^e + \frac{\delta}{2} \pi_{k+2/3}^e + \epsilon \nabla_{\pi} \ell[\Phi_{-(k+1)}] \\ \pi_{k+1}^e = \pi_{k+2/3}^e + \epsilon \nabla_{\phi} \ell[\Phi_{-(k+1)}] \end{cases} \quad (34)$$

**RHEL gradient estimators.** The gradient estimators of the parameters of the linear HRU are:

$$\begin{aligned}\Delta_{\mathbf{A}}^{RHEL}(k, \epsilon) &= -\frac{\delta}{2\epsilon} \left( \nabla_{\mathbf{A}} H^{1/2}[\Phi_k^e(\epsilon), \boldsymbol{\theta}, \mathbf{u}_{-(k+1)}] - \nabla_{\mathbf{A}} H^{1/2}[\Phi_k^e(-\epsilon), \boldsymbol{\theta}, \mathbf{u}_{-(k+1)}] \right) \\ &= -\frac{\delta}{4\epsilon} \left[ \left( \phi_{k+1/3}^e(\epsilon)^\top \phi_{k+1/3}^e(\epsilon) + \phi_{k+2/3}^e(\epsilon)^\top \phi_{k+2/3}^e(\epsilon) \right) \right. \\ &\quad \left. - \left( \phi_{k+1/3}^e(-\epsilon)^\top \phi_{k+1/3}^e(-\epsilon) + \phi_{k+2/3}^e(-\epsilon)^\top \phi_{k+2/3}^e(-\epsilon) \right) \right] \\ &= -\frac{\delta}{2\epsilon} \left[ \left( \phi_{k+1/3}^e(\epsilon) \right)^\top \left( \phi_{k+1/3}^e(\epsilon) \right) - \left( \phi_{k+1/3}^e(-\epsilon) \right)^\top \left( \phi_{k+1/3}^e(-\epsilon) \right) \right] \quad (35)\end{aligned}$$

$$\begin{aligned}\Delta_{\mathbf{B}}^{RHEL}(k, \epsilon) &= -\frac{\delta}{2\epsilon} \left( \nabla_{\mathbf{B}} H^{1/2}[\Phi_k^e(\epsilon), \boldsymbol{\theta}, \mathbf{u}_{-(k+1)}] - \nabla_{\mathbf{B}} H^{1/2}[\Phi_k^e(-\epsilon), \boldsymbol{\theta}, \mathbf{u}_{-(k+1)}] \right) \\ &= \frac{\delta}{4\epsilon} \left[ \left( \phi_{k+1/3}^e(\epsilon) + \phi_{k+2/3}^e(\epsilon) \right) \mathbf{u}_{-(k+1)}^\top \right. \\ &\quad \left. - \left( \phi_{k+1/3}^e(-\epsilon) + \phi_{k+2/3}^e(-\epsilon) \right) \mathbf{u}_{-(k+1)}^\top \right] \\ &= \frac{\delta}{2\epsilon} \left[ \phi_{k+1/3}^e(\epsilon) \mathbf{u}_{-(k+1)}^\top - \phi_{k+1/3}^e(-\epsilon) \mathbf{u}_{-(k+1)}^\top \right] \quad (36)\end{aligned}$$

The gradient estimator with respect to the input of the recurrent transformation is:

$$\begin{aligned}\Delta_{\mathbf{u}}^{RHEL}(k, \epsilon) &= -\frac{\delta}{2\epsilon} \left( \nabla_{\mathbf{u}} H^{1/2}[\Phi_k^e(\epsilon), \boldsymbol{\theta}, \mathbf{u}_{-(k+1)}] - \nabla_{\mathbf{u}} H^{1/2}[\Phi_k^e(-\epsilon), \boldsymbol{\theta}, \mathbf{u}_{-(k+1)}] \right) \\ &= \frac{\delta}{4\epsilon} \left[ \mathbf{B}^\top \left( \phi_{k+1/3}^e(\epsilon) + \phi_{k+2/3}^e(\epsilon) \right) \right. \\ &\quad \left. - \mathbf{B}^\top \left( \phi_{k+1/3}^e(-\epsilon) + \phi_{k+2/3}^e(-\epsilon) \right) \right] \\ &= \frac{\delta}{2\epsilon} \left[ \mathbf{B}^\top \phi_{k+1/3}^e(\epsilon) - \mathbf{B}^\top \phi_{k+1/3}^e(-\epsilon) \right] \quad (37)\end{aligned}$$

**Parallel Scan.** Similarly to the LinOSS model [29], we can compute the recurrence of Linear HRU with a parallel scan [5] to reduce the computational time. To implement the parallel scan, we need to put the discretized dynamics in a form that can be computed in parallel. For this we vectorize the Equation 33:

$$\begin{aligned}\Phi_{k+1} &= \begin{bmatrix} \mathbf{I} & \frac{\delta}{2}\mathbf{I} \\ \mathbf{0} & \mathbf{I} \end{bmatrix} \cdot \Phi_{k+2/3} \\ &= \begin{bmatrix} \mathbf{I} & \frac{\delta}{2}\mathbf{I} \\ \mathbf{0} & \mathbf{I} \end{bmatrix} \cdot \left( \begin{bmatrix} \mathbf{I} & \mathbf{0} \\ -\delta\mathbf{A} & \mathbf{I} \end{bmatrix} \cdot \Phi_{k+1/3} + \begin{bmatrix} \mathbf{0} \\ \delta\mathbf{B} \cdot \mathbf{u}_k \end{bmatrix} \right) \\ &= \begin{bmatrix} \mathbf{I} & \frac{\delta}{2}\mathbf{I} \\ \mathbf{0} & \mathbf{I} \end{bmatrix} \cdot \begin{bmatrix} \mathbf{I} & \mathbf{0} \\ -\delta\mathbf{A} & \mathbf{I} \end{bmatrix} \cdot \begin{bmatrix} \mathbf{I} & \frac{\delta}{2}\mathbf{I} \\ \mathbf{0} & \mathbf{I} \end{bmatrix} \cdot \Phi_k + \begin{bmatrix} \frac{\delta^2}{2}\mathbf{B} \cdot \mathbf{u}_k \\ \delta\mathbf{B} \cdot \mathbf{u}_k \end{bmatrix}\end{aligned}$$

which gives us the discrete dynamics in matrix form:

$$\Phi_{k+1} = M\Phi_k + F_k \quad (38)$$

with:

$$M = \begin{bmatrix} \mathbf{I} - \frac{\delta^2}{2}\mathbf{A} & \delta[\mathbf{I} - \frac{\delta^2}{4}\mathbf{A}] \\ -\delta\mathbf{A} & \mathbf{I} - \frac{\delta^2}{2}\mathbf{A} \end{bmatrix}, \quad F_k = \begin{bmatrix} \frac{\delta^2}{2}\mathbf{B} \cdot \mathbf{u}_k \\ \delta\mathbf{B} \cdot \mathbf{u}_k \end{bmatrix} \quad (39)$$

To compute the echo passes  $\Phi_k^e, k \in \{1, \dots, K\}$ , we only need to adapt  $F_k$  to get for the positive nudging  $+\epsilon$ :

$$F_k^e = \begin{bmatrix} \frac{\delta^2}{2}\mathbf{B} \cdot \mathbf{u}_{-(k+1)} + \epsilon \nabla_{\boldsymbol{\pi}} \ell[\Phi_{-(k+1)}] \\ \delta\mathbf{B} \cdot \mathbf{u}_{-(k+1)} + \epsilon \nabla_{\boldsymbol{\phi}} \ell[\Phi_{-(k+1)}] \end{bmatrix} \quad (40)$$

### A.3.4 Nonlinear HRU

**Hamiltonian of the recurrence.** The nonlinear HRU block is the composition of a nonlinear spatial transformation and a nonlinear recurrent transformation (see Eq. 18). The Hamiltonian of the nonlinear HRU block is defined by the following equations:

$$\begin{aligned} H[\Phi, \theta, u] &= T[\pi, \theta, u] + V[\phi, \theta, u] \\ &= \frac{1}{2}\|\pi\|^2 + \frac{\alpha}{2}\|\phi\|^2 + (\mathbf{A}^{-\top} \cdot \log(\cosh(\mathbf{A} \cdot \phi + \mathbf{B} \cdot u + \mathbf{b}))) \end{aligned} \quad (41)$$

**Dynamics.** The dynamics of the nonlinear HRU block are defined by the following equations:

$$\begin{cases} \dot{\phi} = \pi \\ \dot{\pi} = -(\tanh(\mathbf{A}\phi + \mathbf{B}u + \mathbf{b}) + \alpha\phi) \end{cases} \quad (42)$$

Which, after time-discretization with the integrator defined in A.2.1, gives the following equations:

$$\begin{cases} \phi_{k+1/3} = \phi_k + \frac{\delta}{2}\nabla_{\pi}T[\pi_k, \theta, u_k] \\ \quad = \phi_k + \frac{\delta}{2}\pi_k \\ \pi_{k+1/3} = \pi_k \\ \phi_{k+2/3} = \phi_{k+1/3} \\ \pi_{k+2/3} = \pi_{k+1/3} + \delta\nabla_{\phi}V[\phi_{k+1/3}, \theta, u_k] \\ \quad = \pi_{k+1/3} - \delta(\tanh(\mathbf{A}\phi_{k+1/3} + \mathbf{B}u_k + \mathbf{b}) + \alpha\phi_{k+1/3}) \\ \phi_{k+1} = \phi_{k+2/3} + \frac{\delta}{2}\nabla_{\pi}T[\pi_{k+2/3}, \theta, u_k] \\ \quad = \phi_{k+2/3} + \frac{\delta}{2}\pi_{k+2/3} \\ \pi_{k+1} = \pi_{k+2/3} \end{cases} \quad (43)$$

### A.3.5 Differentiating the time-discretization

In training HRUs, one can also train the multidimensional time-discretization  $\delta \in \mathbb{R}^{d_{\Phi} \times d_{\Phi}}$  that is assumed to be diagonal. For this, it suffices to reparametrize the integrator and the Hamiltonian, so that the time discretization becomes a parameter of the Hamiltonian. Hence the HRU equation 9 becomes:

$$\Phi_{k+1} = \mathcal{M}_{\hat{H},1}[\Phi_k, \theta, u_k, \delta] \quad \forall k = -K \dots -1, \quad (44)$$

with  $\hat{H}$  is a reparametrization of the Hamiltonian  $H$  such that  $\mathcal{M}_{\hat{H},1}[\Phi_k, \theta, u_k, \delta] = \mathcal{M}_{H,\delta}[\Phi_k, \theta, u_k]$ .

For instance, for the linear HRU, we can reparametrize the Hamiltonian as:

$$\hat{H}[\Phi, \theta, u, \delta] = \frac{1}{2}\pi^{\top}\delta\pi + \left(\frac{1}{2}\phi^{\top}(\delta\mathbf{A})\phi - \phi^{\top}(\delta\mathbf{B})u\right) \quad (45)$$

And for the nonlinear HRU, we can reparametrize the Hamiltonian as:

$$\hat{H}[\Phi, \theta, u, \delta] = \frac{1}{2}\pi^{\top}\delta\pi + \frac{\alpha}{2}\phi^{\top}\delta\phi + \frac{1}{2}(\delta\mathbf{A}^{-\top} \cdot \log(\cosh(\mathbf{A} \cdot \phi + \mathbf{B} \cdot u + \mathbf{b}))) \quad (46)$$

$$(47)$$

Note that to match with previous implementations of the nonlinear HRU [30], we used the following parametrization for the discretization step:

$$\hat{H}[\Phi, \theta, u, \delta] = \frac{1}{2}\pi^{\top}\sigma(\delta)\pi + \frac{\alpha}{2}\phi^{\top}\sigma(\delta)\phi + \frac{1}{2}(\sigma(\delta)\mathbf{A}^{-\top} \cdot \log(\cosh(\mathbf{A} \cdot \phi + \mathbf{B} \cdot u + \mathbf{b}))) \quad (48)$$

where  $\sigma(\delta)$  is a diagonal matrix with  $\sigma(\delta)_{ii} = 0.5 + 0.5 \tanh(\delta_{ii}/2)$ .



---

**Algorithm 3** Custom Reverse Mode Automatic Differentiation for an arbitrary HRU

---

```
1: @custom_autodiff
2: function HRU( $\theta_{HRU}, \bar{u}$ )
3:    $\bar{\Phi} \leftarrow \text{HRU-HELPER}(\mathbf{0}, \theta_{HRU}, \bar{u}, \mathbf{0})$ 
4:   return  $\bar{\Phi}$ 
5: end function

6: function FORWARDHRU( $\theta_{HRU}, \bar{u}$ )
7:    $\bar{\Phi} \leftarrow \text{HRU-HELPER}(\mathbf{0}, \theta_{HRU}, \bar{u}, \mathbf{0})$ 
8:   return  $\bar{\Phi}, \Phi_K$ 
9: end function

10: function BACKWARDHRU( $r, \bar{g}$ , input_forward)
11:    $\theta_{HRU}, \bar{u} \leftarrow \text{input\_forward}$ 
12:    $\tilde{u} \leftarrow \text{REVERSE}(\bar{u})$  ▷ Reverse the input sequence
13:    $\Phi_K \leftarrow r$ 
14:    $\Phi_{0, \pm\epsilon}^e \leftarrow \Sigma_x \Phi_K + \epsilon \Sigma_x g_K$ 
15:    $\bar{\Phi}_{\pm\epsilon}^e \leftarrow \text{HRU-HELPER}(\Phi_{0, \pm\epsilon}^e, \theta_{HRU}, \bar{u}, \bar{g})$ 
16:    $\Delta_{\theta_{HRU}}, \Delta_{\bar{u}} \leftarrow \text{LEARNINGHRU}(\bar{\Phi}_{\pm\epsilon}^e)$ 
17:   return  $\Delta_{\theta_{HRU}}, \Delta_{\bar{u}}$  ▷ Loss gradient with respect to the input of FORWARDHRU( $\cdot, \cdot$ )
18: end function

19: HRU.define_custom_autodiff(FORWARDHRU, BACKWARDHRU)
```

---

---

**Algorithm 4** Helper functions for custom Automatic Differentiation

---

```
1: function HRU-HELPER( $\Phi_0, \theta_{HRU}, \bar{u}, \bar{n}$ ) ▷ See Dynamics. in App. A.3.3 and A.3.4 for concrete examples
2:   Input: Initial State  $\Phi_0$ , Parameters of the recurrence  $\theta_{HRU}$ , Inputs  $\bar{u}$ , Nudging  $\bar{n}$ 
3:   Output: State sequence  $\bar{\Phi}$ 
4: end function

5: function LEARNINGHRU( $\bar{\Phi}_{\pm\epsilon}^e, \bar{u}$ ) ▷ See RHEL gradient estimator in App. A.3.3 for a concrete example
6:   Input: Echo passes  $\bar{\Phi}_{\pm\epsilon}^e$ , input sequence time-reversed  $\tilde{u}$ 
7:   Output: Loss gradient w.r.t to the input of FORWARDHRU( $\cdot, \cdot$ ):  $\Delta_{\theta_{HRU}}, \Delta_{\bar{u}}$ 
8: end function
```

---

### A.3.6 Echo passes with automatic differentiation

As mentioned in Remark 8, learning in HSSMs with RHEL involves chaining RHEL gradient estimators with Automatic Differentiation (AD). This design makes RHEL implementation readily compatible with modern automatic differentiation frameworks such as PyTorch or JAX. These libraries provide the capability to create custom functions that, when invoked within a computational graph, are automatically differentiated. Algorithm 3 demonstrates how to implement RHEL/AD chaining for the recurrent component of an HRU block.

To implement a custom autodiff function HRU, we must define two additional functions required by AD: FORWARDHRU and BACKWARDHRU. The FORWARDHRU function is invoked when the HRU function is called within a computational graph to be differentiated. The FORWARDHRU function returns two elements: its output for computing the remainder of the computational graph  $\bar{\Phi}$ , and a *residual* to be stored for the backward pass. For the HRU, the residual consists only of the final state  $\Phi_K$  from the forward pass. The BACKWARDHRU function is called during graph backpropagation. It receives as input the loss gradient from the upstream portion of the computational graph  $\bar{g}$ , the input from the forward pass  $\bar{u}$ , and the residual  $\Phi_K$ . The BACKWARDHRU function computes the gradient of the loss with respect to the HRU block parameters, which is subsequently used to update the HRU parameters.

It also computes the gradient of the loss with respect to the forward pass input  $\bar{u}$ , which is then used to propagate the loss gradient backward through the computational graph.

These three functions are then registered with the autodiff library using the `HRU.define_custom_autodiff` function, enabling the library to automatically differentiate the HRU function when it is called within a computational graph.

The three functions utilize two helper functions: `HRU-HELPER` and `LearningHRU`. The `HRU-HELPER` function implements the HRU dynamics and is employed in both the forward and backward passes. The `LearningHRU` function implements the RHEL gradient estimator and is used in the backward pass to compute the gradient of the loss with respect to the HRU block parameters.

## A.4 Experimental details

**Summary.** This last section provides additional experimental details. More precisely:

- We provide details about the datasets at use, the devices we used, as well as detailed hyperparameters.
- Importantly, we detail **how RHEL can be subject to numerical *underflow* and how we mitigated this issue** – see Fig. 5–6 for details.

### A.4.1 Datasets

This series of tasks was recently introduced as a subset of the University of East Anglia (UEA) datasets with the longest sequences for increased difficulty and recently used to benchmark the linear HSSM previously introduced [29]

The classification datasets are drawn from a recently introduced benchmark [34] that selects a subset of the University of East Anglia (UEA) datasets [35], specifically choosing those with the longest sequences to increase difficulty, and which has been recently employed to evaluate the linear HSSM model [29]. These datasets include EigenWorms (17,984 sequence length, 5 classes), SelfRegulationSCP1 (896 length, 2 classes), SelfRegulationSCP2 (1,152 length, 2 classes), EthanolConcentration (1,751 length, 4 classes), Heartbeat (405 length, 2 classes), and MotorImagery (3,000 length, 2 classes).

Additionally, we evaluate our HSSMs on the PPG-DaLiA dataset [36], a multivariate time series regression dataset designed for heart rate prediction using data collected from a wrist-worn device. It includes recordings from fifteen individuals, each with approximately 150 minutes of data sampled at a maximum rate of 128 Hz. The dataset consists of six channels: blood volume pulse, electrodermal activity, body temperature, and three-axis acceleration. After splitting the data, a sliding window of length 49920 and step size 4992 is applied, representing a challenging very long-range interaction task.

### A.4.2 Simulation details and ressource consumption

**Simulation details** The code to run the experiments is implemented using the JAX auto-differentiation framework [57]. All experiments were run on Nvidia V100 GPUs, except for the PPG experiments, which were run on Nvidia Tesla A100 GPUs due to larger memory demands.

**Ressource consumption.** For the linear HSSM, training time is approximately 30 minutes for both classification (SCP1 dataset) and regression (PPG dataset) tasks across all training algorithms (RHEL and BPPT). The nonlinear HSSM requires approximately 1.5 hours for classification tasks with both algorithms. For the regression tasks, both BPTT and RHEL require approximately between 10 and 20 hours.

RAM consumption remained consistent across all conditions (learning algorithms, datasets, and models), reaching 30GB for regression tasks and 24.15GB for classification (SCP1 dataset). This represents 75% pre-allocation of GPU memory by JAX. While the regression task exceeded the V100’s 32GB memory capacity, it fit within the A100’s 40GB RAM.

### A.4.3 Hyperparameters

We adopted the hyperparameters of [29] without modification, as their experiments utilized the IMEX integrator, which, like our approach, derives from the LeapFrog integrator. The hyperparameters are: learning rate (lr), number of layers (#blocks), number of hidden neurons (hidden dim), state-space dimension (state dim), and whether the time dimension is sent as input (include time). These hyperparameters were found by grid search and are presented in Table 3.

We note that the implementation of the linear HSSM models is based on the code of [29] and uses complex numbers for implementation, which corresponds to doubling the state-space dimension mentioned in Table 3. We found that this dimensional doubling was necessary to recover the performance reported in the original paper, so we maintained this approach. We did not need to apply the same doubling for the nonlinear HSSM covering the performance reported in the paper, so we kept

Table 3: Hyperparameters for the linear and nonlinear HSSM model

| Dataset   | lr      | hidden dim | state dim | #blocks | include time |
|-----------|---------|------------|-----------|---------|--------------|
| Worms     | 0.0001  | 64         | 16        | 2       | False        |
| SCP1      | 0.0001  | 64         | 256       | 6       | False        |
| SCP2      | 0.00001 | 64         | 256       | 6       | True         |
| Ethanol   | 0.00001 | 16         | 256       | 4       | False        |
| Heartbeat | 0.00001 | 64         | 16        | 2       | True         |
| Motor     | 0.0001  | 16         | 256       | 6       | True         |
| PPG       | 0.0001  | 64         | 16        | 2       | True         |

For the *RHEL* algorithm we have two additional hyperparameters: the nudging strength  $\epsilon$  and the scaling factor  $\gamma$  (see Appx. A.4.4). The nudging strength  $\epsilon$  was set to  $10^{-1}$  without prior tuning. For the scaling factor  $\gamma$  we did a grid search over the values  $\{10^0, 10^4, 10^8, 10^{12}\}$  for the regression task (PPG-DaLiA) and found that the best performing parameter was  $10^4$ . For the classification tasks, we did a grid search over the values  $\{10^0, 10^1, 10^2, 10^4\}$  and found that the best performing scaling was  $10^4$  for the averaged score.

We employ different initialization schemes for linear and nonlinear HSSM variants. For the linear HSSM model, we follow the same initialization scheme as [29]:

- $\mathbf{A} \sim \text{Uniform}(0, 1)$
- $\mathbf{B} \sim \text{Uniform}\left(-\frac{1}{\text{hidden\_dim}}, \frac{1}{\text{hidden\_dim}}\right)$
- $\mathbf{C} \sim \text{Uniform}\left(-\frac{1}{\text{state\_dim}}, \frac{1}{\text{state\_dim}}\right)$
- $\mathbf{D} \sim \mathcal{N}(0, 1)$
- $\delta \sim \text{Uniform}(0, 1)$

For the nonlinear HSSM model, we adopt the initialization scheme from [30]:

- $\mathbf{A} \sim \text{Uniform}(0.5, 1)$
- $\mathbf{B} \sim \text{Uniform}\left(-\frac{1}{\text{hidden\_dim}}, \frac{1}{\text{hidden\_dim}}\right)$
- $\mathbf{C} = 0$
- $\mathbf{D} \sim \mathcal{N}(0, 1)$
- $\mathbf{b} \sim \mathcal{N}(0, 1)$
- $\alpha \sim \text{Uniform}(0.1, 1.0)$
- $\delta \sim \text{Uniform}(-1, 1)$

#### A.4.4 Gradient re-scaling for dealing with numerical instabilities

**Analysis** From the theoretical results on RHEL, the nudging strength  $\epsilon$  should be chosen as small as possible to accurately estimate the gradient of the loss function. However, naive numerical implementation can encounter underflow issues. In RHEL, gradient information is encoded in small perturbations to the state  $\Phi$  that generate the echo trajectory  $\Phi^e$ . This presents numerical challenges when the perturbations and state values differ by several orders of magnitude.

In practice, when using finite precision representations (such as floating-point numbers), the perturbations may be lost due to discretization error, where small values cannot be accurately represented or distinguished from zero in finite precision arithmetic.

To address this numerical challenge, we employ a simple gradient rescaling method. We multiplicatively scale the output loss  $\ell[\cdot, \cdot]$  by a constant  $\gamma > 1$  during the echo dynamics computation. This amplification ensures that the perturbation-induced changes in the loss remain within the representable range of floating-point arithmetic. Subsequently, we divide the RHEL parameter gradient estimate  $(\Delta_{\theta}^{\text{RHEL}}(k, \epsilon))$  by the same scaling factor  $\gamma$  to recover the unbiased gradient.

We now demonstrate the effectiveness of this gradient rescaling method on both Linear and Nonlinear HSSM. To evaluate the effect of gradient scaling, we compare the gradients of RHEL and BPTT. Given a HSSM model, we randomly sample a tuple  $(\mathbf{u}_i, \mathbf{y}_i) \sim \mathcal{D}$  from the SCP1 dataset (see App. A.4.1) and compute the gradients of the loss function with respect to the parameters of the recurrence of the HRUs in Fig. 5A for the linear HSSM and Fig. 6A for the nonlinear HSSM.

To gain a more fine-grained understanding, we also compute the gradients with respect to the inputs of the third layer of the HSSM (see Eq. 17 and 18) in Fig. 5B and Fig. 6B. Compared to the parameter gradients, these input gradients are not time-averaged and hence reveal more detail about the underflow issues.

**Gradient rescaling recovers the underflow issues.** As a general pattern across both architectures, we observe that the unscaled ( $\gamma = 10^0$ ) parameter and input gradients tend to be biased: they have a norm ratio above 1.0 and cosine similarity below 1.0. We also note that for both the linear and nonlinear cases, there exists a scaling factor ( $10^6$  for linear and  $10^4$  for nonlinear) that recovers a nearly perfect match between RHEL and BPTT.

Additionally, for the input gradients that are not time-averaged (first column of Fig. 5B and Fig. 6B), there is a large amount of noise in the unscaled gradients, which is eliminated for the best-performing scaling factor ( $10^6$  for linear and  $10^4$  for nonlinear). We conjecture that this noise is due to the underflow effects described above.

**Linear HSSM: scaling improves gradient matching** For the linear analysis, we observe a general pattern: the more we scale the RHEL gradient, the better it matches BPTT, both for input gradients and parameter gradients (Fig. 5A and B).

**Nonlinear HSSM: optimal scaling balances underflow and linearization errors** For the nonlinear analysis, we also observe that scaling the RHEL gradient improves the match between RHEL and BPTT gradients. However, for higher values of the scaling factor ( $\gamma = 10^6$ ), we observe that both input and parameter gradients show a drop in matching quality. We conjecture that this is due to linearization errors. If the loss gradient is too large, the nudging it drives will be large, and the finite difference method used for the learning rule will no longer be valid.

**Solution Implemented.** For the experiments, we conducted a grid search over the scaling parameter  $\gamma$ . In our initial implementation, we applied gradient scaling without the corresponding downscaling step, effectively amplifying the gradient magnitude throughout training. This provided valuable insights into the sensitivity of RHEL dynamics to gradient scaling and improved performance. The complete rescaling procedure (including both upscaling and downscaling) will be implemented in the camera-ready version to provide a more comprehensive analysis of the numerical stabilization approach.

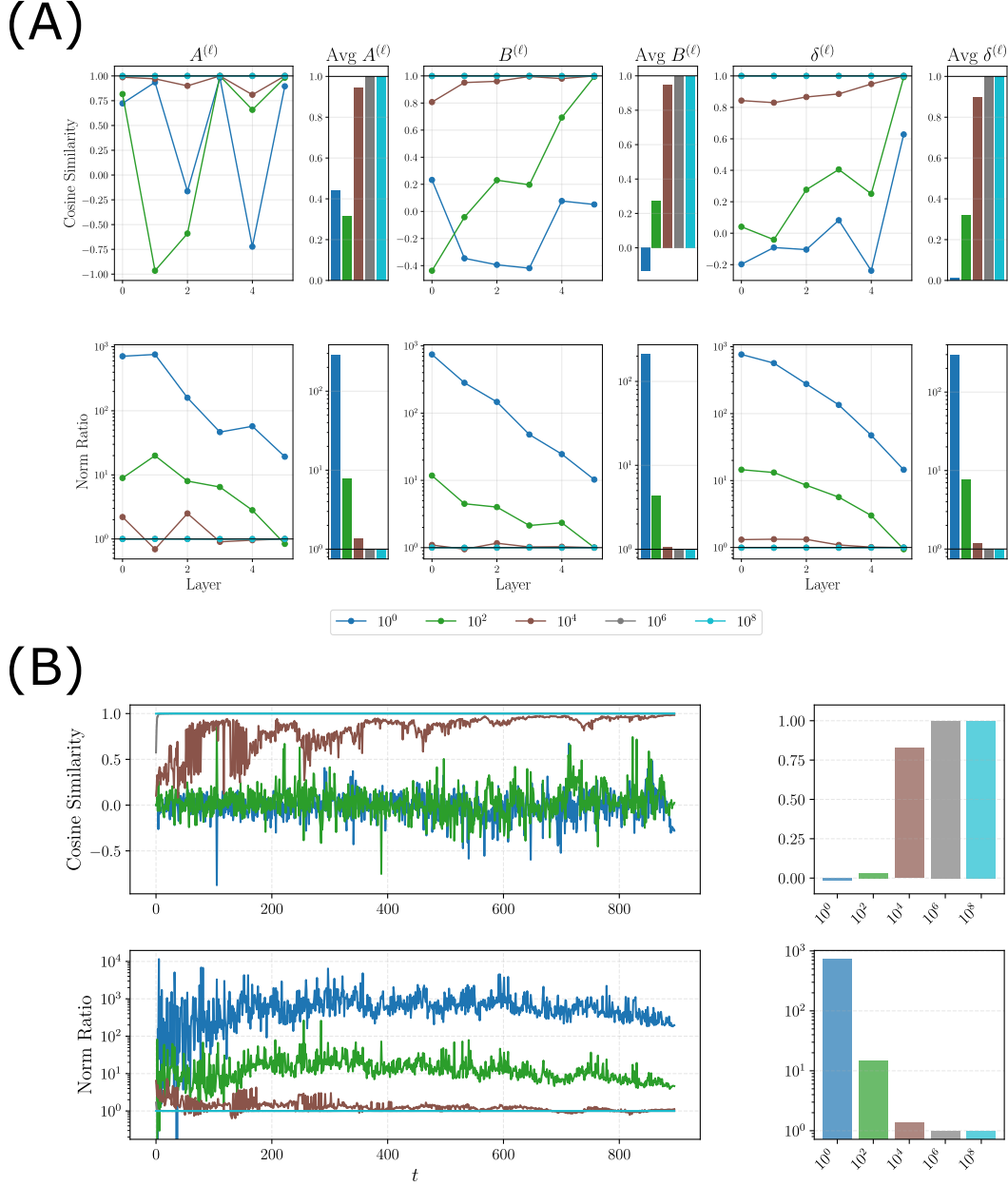
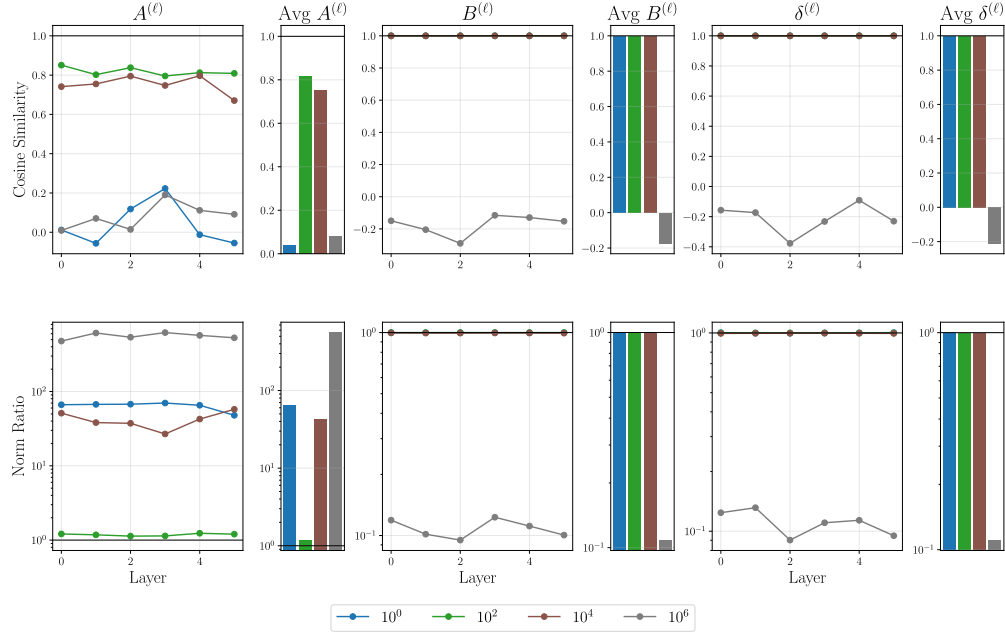


Figure 5: **(A): Parameters gradients comparison between RHEL and BPTT for HSSM.** Given some  $(u_i, y_i) \sim \mathcal{D}$ , we perform BPTT and RHEL on six blocks-deep linear HSSMs for different scaling factor  $\gamma$  of RHEL (different colors). We measure, per layer (line plot) and when averaged across layers (bar-plot), the cosine similarity (top panels) and norm ratio (bottom panels) between RHEL and BPTT parameters gradients of a linear HSSM (Eq. (17)). **(B): Inputs gradient comparison between RHEL and BPTT for HSSM.** Same setting as (A) but we focus on the gradients with respect to the inputs of the third layer ( $u^{(3)}$ , see Eq. 17). We measure, per time steps (line plot) and when averaged across time (bar-plot), the cosine similarity (top panels) and norm ratio (bottom panels) between RHEL and BPTT inputs gradients.

(A)



(B)

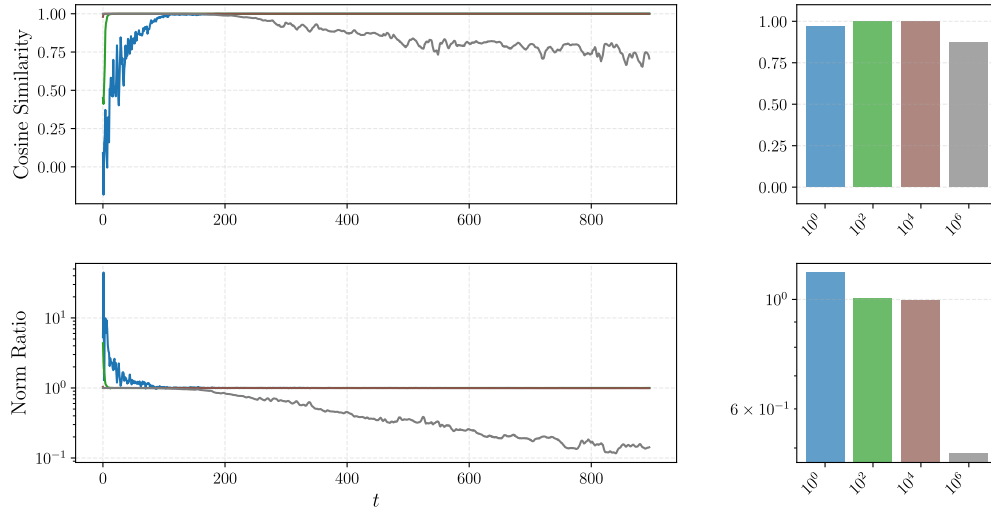


Figure 6: **(A): Parameters gradients comparison between RHEL and BPTT for HSSM.** Given some  $(\mathbf{u}_i, \mathbf{y}_i) \sim \mathcal{D}$ , we perform BPTT and RHEL on six blocks-deep nonlinear HSSMs for different scaling factor  $\gamma$  of RHEL (different colors). We measure, per layer (line plot) and when averaged across layers (bar-plot), the cosine similarity (top panels) and norm ratio (bottom panels) between RHEL and BPTT parameters gradients of a nonlinear HSSM (Eq. (18)). **(B): Inputs gradient comparison between RHEL and BPTT for HSSM.** Same setting as (A) but we focus on the gradients with respect to the inputs of the third layer ( $\mathbf{u}^{(3)}$ , see Eq. 18). We measure, per time steps (line plot) and when averaged across time (bar-plot), the cosine similarity (top panels) and norm ratio (bottom panels) between RHEL and BPTT inputs gradients. For (A) and (B),  $\gamma = 10^8$  was also tested but produced numerical instabilities leading to NaN values and is therefore omitted from the results.